

A web-based student placement system for Zamboanga City National High School West. This document provides a comprehensive overview of all source code files organized by module and functionality.

CORE PROJECT CONFIGURATION

Main Django Project Structure

settings.py - Primary Django configuration file for development

1. Database configuration

```
DATABASE_URL = os.environ.get('DATABASE_URL')
```

```
if DATABASE_URL:
```

```
    # Production (Railway + Supabase)
```

```
    DATABASES = {
```

```
        'default': dj_database_url.parse(
```

```
            DATABASE_URL,
```

```
            conn_max_age=0, # Important for Supabase PgBouncer
```

```
            ssl_require=True,
```

```
        )
```

```
    }
```

```
    DATABASES['default']['OPTIONS'] = {
```

```
        'sslmode': 'require',
```

```
    }
```

```
    DATABASES['default']['DISABLE_SERVER_SIDE_CURSORS'] = True
```

```
else:
```

```
    # Local development — reads from .env DB_* variables
```

```
    DATABASES = {
```

```
        'default': {
```

```
            'ENGINE': os.environ.get('DB_ENGINE', 'django.db.backends.postgresql'),
```

```
            'NAME': os.environ.get('DB_NAME', 'program_recommendation_db'),
```

```
            'USER': os.environ.get('DB_USER', 'postgres'),
```

```
            'PASSWORD': os.environ.get('DB_PASSWORD', ''),
```

```
            'HOST': os.environ.get('DB_HOST', 'localhost'),
```

```
            'PORT': os.environ.get('DB_PORT', '5432'),
```

```
        }
```

```
    }
```

2. Static files and media settings

```
STATIC_URL = '/static/'
```

```
STATICFILES_DIRS = [
```

```
    BASE_DIR / 'shared_assets' / 'static',
```

```
]
```

```
STATIC_ROOT = BASE_DIR / 'staticfiles'
```

```
# Media files (User uploaded content)
```

```
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

3. Email and Azure Document AI configuration

try:

```
from docs.django_settings_documentai import (
    GOOGLE_CLOUD_PROJECT as _GOOGLE_CLOUD_PROJECT,
    DOCUMENT_AI_PROCESSOR_ID as _DOCUMENT_AI_PROCESSOR_ID,
    DOCUMENT_AI_LOCATION as _DOCUMENT_AI_LOCATION,
    OCR_CONFIG as _OCR_CONFIG,
)
```

except Exception:

```
_GOOGLE_CLOUD_PROJECT = None
_DOCUMENT_AI_PROCESSOR_ID = None
_DOCUMENT_AI_LOCATION = None
_OCR_CONFIG = None
```

Environment overrides or defaults

```
GOOGLE_CLOUD_PROJECT = os.environ.get('GOOGLE_CLOUD_PROJECT') or
os.environ.get('GCP_PROJECT_ID') or _GOOGLE_CLOUD_PROJECT or '1094485135926'
DOCUMENT_AI_PROCESSOR_ID = os.environ.get('DOCUMENT_AI_PROCESSOR_ID') or
_DOCUMENT_AI_PROCESSOR_ID or 'a0cbcc2e3afe7ae0'
DOCUMENT_AI_LOCATION = os.environ.get('DOCUMENT_AI_LOCATION') or
_DOCUMENT_AI_LOCATION or 'us'
```

Consolidated OCR config consumed by GeminiAPIKeyOCR

```
OCR_CONFIG = _OCR_CONFIG or {
    'tolerance': 3.0,
    'use_document_ai': True,
    'async_processing': False,
    'batch_size': 10,
    'subject_tolerance': 0.70,
    'row_y_padding': 35,
    'column_pad': 50,
}
```

Ensure IDs are present

```
OCR_CONFIG.update({
    'project_id': GOOGLE_CLOUD_PROJECT,
    'processor_id': DOCUMENT_AI_PROCESSOR_ID,
    'location': DOCUMENT_AI_LOCATION,
})
```

settings_production.py - Production-specific Django settings

1. Database URL configuration from Railway

```

# Auto-detect Railway domain
RAILWAY_PUBLIC_DOMAIN = os.environ.get('RAILWAY_PUBLIC_DOMAIN', '') # e.g.
myapp.up.railway.app
RAILWAY_STATIC_URL = os.environ.get('RAILWAY_STATIC_URL', '') # e.g.
https://myapp.up.railway.app

from urllib.parse import urlparse

if RAILWAY_PUBLIC_DOMAIN and RAILWAY_PUBLIC_DOMAIN not in ALLOWED_HOSTS:
    ALLOWED_HOSTS.append(RAILWAY_PUBLIC_DOMAIN)

if RAILWAY_STATIC_URL:
    railway_host = urlparse(RAILWAY_STATIC_URL).netloc
    if railway_host and railway_host not in ALLOWED_HOSTS:
        ALLOWED_HOSTS.append(railway_host)

# Allow all Railway subdomains
ALLOWED_HOSTS.append('.up.railway.app')

# Railway internal health check host
ALLOWED_HOSTS.append('healthcheck.railway.app')

# Fallback — Railway handles host verification at proxy level
if not ALLOWED_HOSTS:
    ALLOWED_HOSTS = ['*']

# CSRF trusted origins
CSRF_TRUSTED_ORIGINS = os.environ.get('CSRF_TRUSTED_ORIGINS', '').split(',')
CSRF_TRUSTED_ORIGINS = [origin.strip() for origin in CSRF_TRUSTED_ORIGINS if
origin.strip()]

# Auto-add Railway URL to CSRF trusted origins
# Auto-add Railway URLs to CSRF trusted origins
if RAILWAY_STATIC_URL:
    url = RAILWAY_STATIC_URL
    if not url.startswith('http://') and not url.startswith('https://'):
        url = f'https://{url}'
    if url not in CSRF_TRUSTED_ORIGINS:
        CSRF_TRUSTED_ORIGINS.append(url)

if RAILWAY_PUBLIC_DOMAIN:
    railway_origin = f'https://{RAILWAY_PUBLIC_DOMAIN}'
    if railway_origin not in CSRF_TRUSTED_ORIGINS:
        CSRF_TRUSTED_ORIGINS.append(railway_origin)

```

2. OCR/ Google Cloud Configuration

```
GOOGLE_CLOUD_PROJECT = os.environ.get('GOOGLE_CLOUD_PROJECT', '')
DOCUMENT_AI_PROCESSOR_ID = os.environ.get('DOCUMENT_AI_PROCESSOR_ID', '')
DOCUMENT_AI_LOCATION = os.environ.get('DOCUMENT_AI_LOCATION', 'us')
```

```
# Gemini API Key (for OCR)
```

```
GEMINI_API_KEY = os.environ.get('GEMINI_API_KEY', '')
```

```
OCR_CONFIG = {
    'tolerance': 3.0,
    'use_document_ai': bool(DOCUMENT_AI_PROCESSOR_ID),
    'async_processing': False,
    'batch_size': 10,
    'subject_tolerance': 0.70,
    'row_y_padding': 35,
    'column_pad': 50,
    'project_id': GOOGLE_CLOUD_PROJECT,
    'processor_id': DOCUMENT_AI_PROCESSOR_ID,
    'location': DOCUMENT_AI_LOCATION,
}
```

urls.py - Root URL routing configuration

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static
```

```
urlpatterns = [
    # path('admin/', admin.site.urls),
    path('admin-portal/', include('admin_app.urls')),
    path("", include('enrollment_app.urls')),
    path('coordinator/', include('coordinator_app.urls')),
]
```

```
# Serve static and media files during development
```

```
if settings.DEBUG:
```

```
    urlpatterns += static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Project Configuration Files

manage.py - Django management utility script

```
def main():
    """Run administrative tasks."""
```

```

os.environ.setdefault("DJANGO_SETTINGS_MODULE",
"section_placement_system.settings")
try:
    from django.core.management import execute_from_command_line
except ImportError as exc:
    raise ImportError(
        "Couldn't import Django. Are you sure it's installed and "
        "available on your PYTHONPATH environment variable? Did you "
        "forget to activate a virtual environment?"
    ) from exc
execute_from_command_line(sys.argv)

if __name__ == "__main__":
    main()

```

requirements.txt - Python package dependencies

```

# Core Django
Django==4.2.16
asgiref==3.11.0
sqlparse==0.5.5

# Database
dj-database-url==3.1.1
psycopg2-binary==2.9.9

# Server
gunicorn==21.2.0
whitenoise==6.7.0

# Environment
python-dotenv==1.2.1

# Media / Cloud storage
cloudinary==1.44.1
django-cloudinary-storage==0.3.0

# REST API
djangorestframework==3.16.1

# HTTP / Networking
requests==2.32.3
httpx==0.28.1
httpcore==1.0.7
httplib2==0.31.1
urllib3==2.3.0
certifi==2025.1.31

```

charset-normalizer==3.4.1
idna==3.10
h11==0.14.0

Google / AI / Gemini

google-genai==1.59.0
google-generativeai==0.8.6
google-ai-generativelanguage==0.6.15
google-api-core==2.28.1
google-api-python-client==2.188.0
google-auth==2.47.0
google-auth-httpplib2==0.3.0
google-cloud-documentai==3.8.0
google-cloud-aiplatform==1.133.0
google-cloud-bigquery==3.40.0
google-cloud-core==2.5.0
google-cloud-resource-manager==1.16.0
google-cloud-storage==3.8.0
google-cloud-vision==3.11.0
google-crc32c==1.8.0
google-resumable-media==2.8.0
googleapis-common-protos==1.72.0
grpc-google-iam-v1==0.14.3
grpcio==1.76.0
grpcio-status==1.71.2
proto-plus==1.27.0
protobuf==5.29.5

OCR

easyocr==1.7.2
opencv-python-headless==4.10.0.84
pytesseract==0.3.13
pyclipper==1.4.0
python-bidi==0.6.7
arabic-reshaper==3.0.0

Data science

numpy==2.2.6
pandas==2.2.3
matplotlib==3.10.0
scipy==1.14.1
scikit-learn==1.6.0
scikit-image==0.26.0
seaborn==0.13.2
imbalanced-learn==0.14.1
shap==0.50.0
joblib==1.4.2

threadpoolctl==3.6.0
lightgbm==4.5.0
xgboost==2.1.3
sklearn-compat==0.1.5
slicer==0.0.8

Deep learning (easyocr dependency)

torch==2.9.1
torchvision==0.24.1
networkx==3.4.2
sympy==1.14.0
filelock==3.20.1
fsspec==2025.3.0
llvmlite==0.46.0
numba==0.63.0

Files / Documents

Pillow==10.4.0
python-docx==1.2.0
reportlab==4.4.7
openpyxl==3.1.5
pypdf==6.6.0
xlrd==2.0.1
lxml==6.0.2
tiffio==2025.2.18
imageio==2.37.2

Utilities

tqdm==4.67.1
pydantic==2.12.5
pydantic_core==2.41.5
typing_extensions==4.15.0
typing-inspection==0.4.2
packaging==24.2
six==1.17.0
python-dateutil==2.9.0.post0
pytz==2024.2
tzdata==2024.2
tzlocal==5.3.1
PyYAML==6.0.2
tenacity==9.1.2
cachetools==6.2.4
distro==1.9.0
Jinja2==3.1.5
MarkupSafe==3.0.2
cryptography==44.0.3
cffi==1.17.1

```
pycparser==2.22
pyasn1==0.6.1
pyasn1_modules==0.4.2
rsa==4.9.1
setuptools==75.8.0
sniffio==1.3.1
anyio==4.8.0
```

tailwind.config.js - Tailwind CSS configuration

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  // Scan ALL Django templates for Tailwind classes
  content: [
    './enrollment_app/templates/**/*.html',
    './admin_app/templates/**/*.html',
    './coordinator_app/templates/**/*.html',
  ],
  theme: {
    extend: {
      colors: {
        primary: "#991b1b",
        "primary-dark": "#7f1d1d",
        secondary: "#eab308",
        "secondary-dark": "#ca8a04",
        accent: "#b91c1c",
        "ai-primary": "#10b981",
        "ai-dark": "#059669",
        coordinator: "#991b1b",
        "coordinator-dark": "#7f1d1d",
      },
    },
    fontFamily: {
      sans: ["Poppins", "sans-serif"],
      display: ["Playfair Display", "serif"],
    },
    animation: {
      "fade-in": "fadeIn 0.3s ease-out",
      "slide-in-right": "slideInRight 0.3s ease-out",
      "modal-fade-in": "modalFadeIn 0.3s ease-out",
    },
    keyframes: {
      fadeIn: {
        from: { opacity: "0", transform: "translateY(10px)" },
        to: { opacity: "1", transform: "translateY(0)" },
      },
      slideInRight: {
        from: { transform: "translateX(100%)", opacity: "0" },
        to: { transform: "translateX(0)", opacity: "1" },
      },
    },
  },
}
```



```

    },
    modalFadelIn: {
      from: { opacity: "0", transform: "translateY(-20px) scale(0.95)" },
      to: { opacity: "1", transform: "translateY(0) scale(1)" },
    },
  },
},
},
plugins: [],
};

```

nixpacks.toml - Nix package configuration

Nixpacks configuration for Railway

```
providers = ["python"]
```

```
[phases.setup]
```

```
nixPkgsAppend = ["gcc", "zlib", "libjpeg", "libffi", "openssl"]
```

```
[phases.install]
```

```
cmds = [
```

```
  "python -m venv --copies /opt/venv",
```

```
  "/opt/venv/bin/pip install -r requirements.txt",
```

```
  "/opt/venv/bin/pip install --force-reinstall --no-deps google-genai==1.59.0"
```

```
]
```

```
[start]
```

```
cmd = "python manage.py collectstatic --noinput; python manage.py migrate --noinput; gunicorn
section_placement_system.wsgi:application --bind 0.0.0.0:$PORT --timeout 120 --workers 2 --
threads 2 --log-file -"
```

```
[variables]
```

```
PYTHONUNBUFFERED = "1"
```

railway.toml - Railway deployment configuration

```
[build]
```

```
builder = "nixpacks"
```

```
[deploy]
```

```
startCommand = "python manage.py collectstatic --noinput; python manage.py migrate --
noinput; gunicorn section_placement_system.wsgi:application --bind 0.0.0.0:$PORT --timeout
120 --workers 2 --threads 2 --log-file -"
```

```
healthcheckPath = "/"
```

```
healthcheckTimeout = 300
```

```
restartPolicyType = "on_failure"
```

```
restartPolicyMaxRetries = 5
```

4 main Django applications in this system:

1. ADMIN APP - Administrative functions for staff and teachers managing the system.

- models.py

SystemSettings - System configuration

```
class SystemSettings(models.Model):
```

```
    SETTING_TYPE_CHOICES = [
        ('header_logo_school', 'Header - School Logo'),
        ('carousel_slide_1_image', 'Carousel - Slide 1 Image'),
        ('carousel_slide_1_title', 'Carousel - Slide 1 Title'),
        ('carousel_slide_1_caption', 'Carousel - Slide 1 Caption'),
        ('carousel_slide_2_image', 'Carousel - Slide 2 Image'),
        ('carousel_slide_2_title', 'Carousel - Slide 2 Title'),
        ('carousel_slide_2_caption', 'Carousel - Slide 2 Caption'),
        ('carousel_slide_3_image', 'Carousel - Slide 3 Image'),
        ('carousel_slide_3_title', 'Carousel - Slide 3 Title'),
        ('carousel_slide_3_caption', 'Carousel - Slide 3 Caption'),
        ('partner_logo_1', 'Partner - Logo 1'),
        ('partner_logo_1_name', 'Partner - Logo 1 Name'),
        ('partner_logo_2', 'Partner - Logo 2'),
        ('partner_logo_2_name', 'Partner - Logo 2 Name'),
        ('partner_logo_3', 'Partner - Logo 3'),
        ('partner_logo_3_name', 'Partner - Logo 3 Name'),
        ('header_logo_region', 'Header - Region IX Logo'),
        ('header_logo_peninsula', 'Header - Zamboanga Peninsula Logo'),
        ('header_logo_matatag', 'Header - Matatag Logo'),
        ('header_caption', 'Header - Caption'),
        ('announcement_image', 'Announcement - Image'),
        ('announcement_caption', 'Announcement - Caption'),
        ('contact_address', 'Contact - Address'),
        ('contact_phone', 'Contact - Phone'),
        ('contact_email', 'Contact - Email'),
        ('contact_facebook', 'Contact - Facebook'),
        ('contact_hours', 'Contact - Operating Hours'),
        ('footer_copyright', 'Footer - Copyright'),
        ('footer_links', 'Footer - Links (JSON)'),
        ('footer_social', 'Footer - Social Media (JSON)'),
    ]
```

```
    setting_type = models.CharField(max_length=50, choices=SETTING_TYPE_CHOICES,
unique=True)
```

```
    setting_value = models.TextField(blank=True, null=True)
```

```
    image = models.ImageField(upload_to='system_settings/', blank=True, null=True)
```

```
    updated_by = models.ForeignKey(
```

```
        User, on_delete=models.SET_NULL, null=True, blank=True,
related_name='updated_settings'
```

```
)
```

```

updated_at = models.DateTimeField(auto_now=True)
created_at = models.DateTimeField(auto_now_add=True)

class Meta:
    ordering = ['setting_type']
    verbose_name = 'System Setting'
    verbose_name_plural = 'System Settings'
    db_table = 'system_settings'
    indexes = [models.Index(fields=['setting_type'])]

def __str__(self):
    return f"{self.get_setting_type_display()}"

def get_formatted_date(self):
    return self.updated_at.strftime('%b %d, %Y at %I:%M %p')

```

UserProfile - Extended user information with photos

```

class UserProfile(models.Model):
    USER_TYPE_CHOICES = [
        ('admin', 'Admin'),
        ('coordinator', 'Coordinator'),
    ]

    user = models.OneToOneField(User, on_delete=models.CASCADE, related_name='profile')
    user_type = models.CharField(max_length=20, choices=USER_TYPE_CHOICES)

    program = models.ForeignKey(
        Program, on_delete=models.SET_NULL, null=True, blank=True,
        related_name='userprofile_set'
    )
    position = models.ForeignKey(
        Position, on_delete=models.SET_NULL, null=True, blank=True,
        related_name='userprofile_set'
    )
    department = models.ForeignKey(
        Department, on_delete=models.SET_NULL, null=True, blank=True,
        related_name='userprofile_set'
    )

    employee_id = models.CharField(max_length=50, unique=True)
    photo = models.ImageField(upload_to='user_profiles/', blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'user_profile'

```

```

def __str__(self):
    return f"{self.user.username} - {self.user_type}"

def get_user_type_display(self):
    return {'admin': 'Admin', 'coordinator': 'Coordinator'}.get(self.user_type, self.user_type)

def get_program_name(self):
    return self.program.code if self.program else 'N/A'

def get_position_name(self):
    return self.position.name if self.position else 'N/A'

def get_department_name(self):
    return self.department.name if self.department else 'N/A'

def get_access_badges(self):
    return ['Admin'] if self.user_type == 'admin' else ['Coordinator']

def get_last_login_formatted(self):
    if not self.user.last_login:
        return 'Never'
    today = timezone.now().date()
    yesterday = today - datetime.timedelta(days=1)
    login_date = self.user.last_login.date()
    if login_date == today:
        return f"Today, {self.user.last_login.strftime('%I:%M %p')}}"
    elif login_date == yesterday:
        return f"Yesterday, {self.user.last_login.strftime('%I:%M %p')}}"
    days_ago = (today - login_date).days
    if days_ago < 7:
        return f"{days_ago} days ago"
    return self.user.last_login.strftime("%b %d, %Y")

def get_date_joined_formatted(self):
    return self.user.date_joined.strftime("%b %d, %Y")

```

StaffMember – Staff information

```

class StaffMember(models.Model):
    name = models.CharField(max_length=200)
    position = models.CharField(max_length=200)
    photo = models.ImageField(upload_to='staff_members/', blank=True, null=True)
    display_order = models.IntegerField(default=0)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

```

```

class Meta:
    ordering = ['display_order', 'name']
    verbose_name = 'Staff Member'
    verbose_name_plural = 'Staff Members'
    db_table = 'staff_member'
    indexes = [
        models.Index(fields=['display_order']),
        models.Index(fields=['is_active']),
    ]

```

```

def __str__(self):
    return f"{self.name} - {self.position}"

```

Teacher – Teacher records

```

class Teacher(models.Model):
    first_name = models.CharField(max_length=100)
    middle_name = models.CharField(max_length=100, blank=True, null=True)
    last_name = models.CharField(max_length=100)

    position = models.ForeignKey(
        Position, on_delete=models.SET_NULL, null=True, blank=True, related_name='teachers'
    )
    department = models.ForeignKey(
        Department, on_delete=models.SET_NULL, null=True, blank=True,
        related_name='teachers'
    )

    address = models.TextField(blank=True, null=True)
    email = models.EmailField(max_length=255, unique=True)

    is_adviser = models.BooleanField(default=False)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

```

```

class Meta:
    ordering = ['last_name', 'first_name']
    verbose_name = 'Teacher'
    verbose_name_plural = 'Teachers'
    db_table = 'teacher'
    indexes = [
        models.Index(fields=['last_name', 'first_name']),
        models.Index(fields=['email']),
        models.Index(fields=['is_adviser']),
    ]

```

```

def __str__(self):

```

```

        return self.get_full_name()

    def get_full_name(self):
        parts = [self.first_name or "",
                  self.middle_name or "", self.last_name or ""]
        return ' '.join(p for p in parts if p).strip()

    def clean(self):
        if self.first_name:
            self.first_name = self.first_name.strip()
        if self.middle_name:
            self.middle_name = self.middle_name.strip()
        if self.last_name:
            self.last_name = self.last_name.strip()
        if self.email:
            self.email = self.email.strip().lower()
        if not self.first_name:
            raise ValidationError({'first_name': 'First name is required.'})
        if not self.last_name:
            raise ValidationError({'last_name': 'Last name is required.'})

    def save(self, *args, **kwargs):
        self.full_clean()
        super().save(*args, **kwargs)

```

Building – School buildings

```

class Building(models.Model):
    name = models.CharField(max_length=100, unique=True)

    def __str__(self):
        return self.name

```

Room – Classroom information

```

class Room(models.Model):
    building = models.ForeignKey(Building, on_delete=models.CASCADE,
                                related_name='rooms')
    room_number = models.CharField(max_length=50)

    class Meta:
        unique_together = ('building', 'room_number')

    def __str__(self):
        return f"{self.room_number} in {self.building.name}"

```

Section – Student sections with advisers

```
class Section(models.Model):
    school_year = models.ForeignKey(
        'SchoolYear',
        on_delete=models.CASCADE,
        related_name='sections',
        null=True, blank=True,
    )
    program = models.ForeignKey(
        Program, on_delete=models.CASCADE, related_name='sections'
    )

    grade_level = models.ForeignKey(
        GradeLevel,
        on_delete=models.SET_NULL,
        null=True, blank=True,
        related_name='sections',
        help_text="Grade level this section handles (e.g. Grade 7, Grade 8)"
    )

    name = models.CharField(max_length=100)
    regular_track = models.CharField(
        max_length=20, blank=True, null=True,
        help_text="For Regular program sections: TOP5 or HETERO"
    )
    adviser = models.ForeignKey(
        Teacher,
        on_delete=models.SET_NULL,
        null=True, blank=True,
        related_name='advisory_sections',
    )
    building = models.CharField(max_length=50, blank=True, null=True)
    room = models.CharField(max_length=50, blank=True, null=True)
    max_students = models.PositiveIntegerField(default=40)
    current_students = models.PositiveIntegerField(default=0)
    masterlist_published = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

class Meta:
    ordering = ['school_year', 'program__code', 'grade_level__code', 'created_at']
    unique_together = [('school_year', 'program', 'grade_level', 'name')]
    db_table = 'section'
    indexes = [
        models.Index(fields=['school_year', 'program', 'name']),
        models.Index(fields=['school_year']),
        models.Index(fields=['adviser']),
    ]
```

```

        models.Index(fields=['grade_level']),
    ]

    def __str__(self):
        year_label = self.school_year.year_label if self.school_year else 'No Year'
        grade_label = self.grade_level.name if self.grade_level else 'No Grade'
        track_info = f"({self.regular_track})" if self.regular_track else ""
        return f"{year_label} - {grade_label} - {self.program.code}{track_info} - {self.name}"

    def update_current_students_count(self):
        """Recount enrolled students using the real FK — no string coercion needed."""
        from enrollment_app.models import ProgramSelection
        actual_count = ProgramSelection.objects.filter(
            assigned_section=self,
            admin_approved=True
        ).count()
        self.current_students = actual_count
        self.save(update_fields=['current_students'])
        return actual_count

    def get_actual_count(self):
        from enrollment_app.models import ProgramSelection
        return ProgramSelection.objects.filter(
            assigned_section=self,
            admin_approved=True
        ).count()

    def clean(self):
        if self.name:
            self.name = self.name.strip()
        if not self.name:
            raise ValidationError({'name': 'Section name is required.'})
        if self.max_students <= 0:
            raise ValidationError({'max_students': 'Maximum students must be positive.'})

    def save(self, *args, **kwargs):
        self.full_clean()
        super().save(*args, **kwargs)

```

Subject – School subjects

```

class Subject(models.Model):
    """
    Subject is scoped to Program only (not GradeLevel).
    Math in Grade 7 STE and Math in Grade 9 STE are the same Subject row.
    The grade dimension is carried by AcademicPerformance.grade_level.
    """

```



```

    program = models.ForeignKey(Program, on_delete=models.CASCADE,
related_name='subjects')
    name = models.CharField(max_length=150)
    code = models.CharField(max_length=50)
    description = models.TextField(blank=True, null=True)
    is_active = models.BooleanField(default=True)
    is_threshold_subject = models.BooleanField(
        default=False,
        help_text=(
            "STE only — if True, student must score >= 83 in this subject "
            "to avoid probation. Applies to Math, Science, English, Research "
            "and any future STE specialized subjects."
        )
    )
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['program__code', 'name']
        unique_together = [('program', 'code')]
        db_table = 'subject'
        indexes = [
            models.Index(fields=['program', 'code']),
            models.Index(fields=['is_active']),
        ]

    def __str__(self):
        return f"{self.code} - {self.name} ({self.program.code})"

    def clean(self):
        if self.name:
            self.name = self.name.strip()
        if self.code:
            self.code = self.code.strip().upper()
        if not self.name:
            raise ValidationError({'name': 'Subject name is required.'})
        if not self.code:
            raise ValidationError({'code': 'Subject code is required.'})

    def save(self, *args, **kwargs):
        self.full_clean()
        super().save(*args, **kwargs)

```

GradeLevel – Grade levels (7–12)

```

class GradeLevel(models.Model):
    """Represents a grade level (Grade 7, Grade 8, Grade 9, Grade 10)."""
    code = models.CharField(

```

```

        max_length=10, unique=True,
        help_text="Short code (e.g. G7, G8, G9, G10)"
    )
    name = models.CharField(
        max_length=50, unique=True,
        help_text="Full name (e.g. Grade 7, Grade 8)"
    )
    description = models.TextField(blank=True, null=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

```

```

class Meta:
    ordering = ['code']
    db_table = 'grade_level'
    verbose_name = 'Grade Level'
    verbose_name_plural = 'Grade Levels'
    indexes = [
        models.Index(fields=['code']),
        models.Index(fields=['is_active']),
    ]

```

```

def __str__(self):
    return self.name

```

DocumentRequirement – Documents needed per grade

```

class DocumentRequirement(models.Model):

```

```

    REQUIREMENT_TYPE_CHOICES = [
        ('mandatory', 'Mandatory'),
        ('optional', 'Optional'),
        ('conditional', 'Conditional'),
    ]

```

```

    APPLIES_TO_CHOICES = [
        ('all', 'All Enrollee Types'),
        ('new', 'New Students Only'),
        ('transferee', 'Transferees Only'),
        ('continuing', 'Continuing Students Only'),
        ('returnee', 'Returnees Only'),
    ]

```

```

    school_year = models.ForeignKey(
        SchoolYear,
        on_delete=models.CASCADE,
        related_name='document_requirements',
    )
    name = models.CharField(max_length=100)

```

```

description = models.TextField(blank=True, null=True)
requirement_type = models.CharField(
    max_length=20, choices=REQUIREMENT_TYPE_CHOICES, default='mandatory'
)

applies_to = models.CharField(
    max_length=20,
    choices=APPLIES_TO_CHOICES,
    default='all',
    help_text=(
        "Which enrollee type must submit this document. "
        "'All' covers new students and transferees. "
        "Continuing students never go through the document upload step "
        "— their docs carry over automatically."
    )
)

grade_level = models.ForeignKey(
    'GradeLevel',
    on_delete=models.SET_NULL,
    null=True,
    blank=True,
    related_name='document_requirements',
    help_text=(
        "If set, only students enrolling into this grade level see this requirement. "
        "Leave blank (NULL) for requirements that apply to all grade levels. "
        "Example: 'SF9 Gr 6 card' → set to Grade 7 (incoming Grade 7 students submit their
Gr 6 card)."
    )
)

file_format = models.CharField(
    max_length=100, default='pdf,jpg,jpeg,png',
    help_text="Allowed file formats (comma-separated)"
)

max_file_size_mb = models.DecimalField(
    max_digits=5, decimal_places=2, default=5.0,
    validators=[MinValueValidator(0.1), MaxValueValidator(50.0)],
)

is_active = models.BooleanField(default=True)
order = models.IntegerField(default=0)
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)
created_by = models.ForeignKey(
    User, on_delete=models.SET_NULL, null=True, blank=True,
    related_name='created_document_requirements',
)

```

```

class Meta:
    ordering = ['school_year', 'order', 'name']
    verbose_name = 'Document Requirement'
    verbose_name_plural = 'Document Requirements'
    db_table = 'document_requirement'
    unique_together = [('school_year', 'name')]
    indexes = [
        models.Index(fields=['school_year', 'is_active']),
        models.Index(fields=['requirement_type']),
        models.Index(fields=['order']),
        models.Index(fields=['applies_to']),
        models.Index(fields=['grade_level']),
    ]

    def __str__(self):
        year_label = self.school_year.year_label if self.school_year else 'No Year'
        return f"{year_label} - {self.name} ({self.get_requirement_type_display()})"

    def get_allowed_extensions(self):
        return [ext.strip() for ext in self.file_format.split(',')]

    def clean(self):
        if self.name:
            self.name = self.name.strip()
        if not self.name:
            raise ValidationError({'name': 'Document name is required.'})

    def save(self, *args, **kwargs):
        self.full_clean()
        super().save(*args, **kwargs)

```

LISStudent – Integration with LIS

```

class LISStudent(models.Model):
    """LIS Student data for LRN verification."""
    lrn = models.CharField(max_length=12, primary_key=True)
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    birth_date = models.DateField()
    last_school = models.CharField(max_length=255)

    class Meta:
        managed = True
        db_table = 'lis_students'

    def __str__(self):
        return f"{self.lrn} - {self.last_name}"

```

```

# for request move student
class ProgramMoveRequest(models.Model):
    STATUS_CHOICES = [
        ('pending', 'Pending'),
        ('approved', 'Approved'),
        ('rejected', 'Rejected'),
    ]

    # Programs that require DOST exam — cannot be moved INTO unless student has it
    DOST_REQUIRED_PROGRAMS = ['STE']

    # Move eligibility matrix: from_program -> list of allowed target programs
    ALLOWED_MOVES = {
        'STE': ['SPFL', 'REGULAR', 'SPTVE', 'OHSP', 'SNED'],
        'SPFL': ['SPTVE', 'REGULAR', 'OHSP', 'SNED'],
        'SPTVE': ['SPFL', 'REGULAR', 'OHSP', 'SNED'],
        'OHSP': ['REGULAR', 'SPFL', 'SPTVE', 'SNED'],
        'SNED': ['REGULAR', 'SPFL', 'SPTVE', 'OHSP'],
        'REGULAR': [], # REGULAR (both TOP5 and HETERO) cannot move to specialized
        programs
    }

    # TOP5 override — these can move to non-STE specialized programs
    REGULAR_TOP5_ALLOWED = ['SPFL', 'SPTVE', 'OHSP', 'SNED']

    student = models.ForeignKey(
        'enrollment_app.Student',
        on_delete=models.CASCADE,
        related_name='move_requests'
    )
    from_program_code = models.CharField(max_length=20)
    to_program_code = models.CharField(max_length=20)
    from_section = models.ForeignKey(
        'admin_app.Section',
        null=True, blank=True,
        on_delete=models.SET_NULL,
        related_name='move_requests_from'
    )
    reason = models.TextField()
    requested_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        related_name='submitted_move_requests'
    )

```

```

status = models.CharField(max_length=20, choices=STATUS_CHOICES,
default='pending')
reviewed_by = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.SET_NULL,
    null=True, blank=True,
    related_name='reviewed_move_requests'
)
review_notes = models.TextField(blank=True, default="")
created_at = models.DateTimeField(auto_now_add=True)
reviewed_at = models.DateTimeField(null=True, blank=True)

class Meta:
    ordering = ['-created_at']

def __str__(self):
    return f"{self.student.lrn}: {self.from_program_code} → {self.to_program_code} [{self.status}]"

@classmethod
def check_eligibility(cls, from_program_code, to_program_code, student=None):
    """
    Returns (is_eligible: bool, reason: str)
    Checks business rules for program transfer eligibility.
    """
    from_code = from_program_code.upper()
    to_code = to_program_code.upper()

    if from_code == to_code:
        return False, "Student is already in this program."

    # STE requires DOST exam — no one can move into STE
    if to_code in cls.DOST_REQUIRED_PROGRAMS:
        return False, (
            "Cannot move to STE. STE requires a passing DOST entrance exam "
            "which cannot be waived via program transfer."
        )

    # Check if from_program is REGULAR
    if from_code == 'REGULAR':
        # Check if student is TOP5 — TOP5 can move to non-STE specialized programs
        is_top5 = False
        if student:
            try:
                ps = student.program_selection
                if ps.assigned_section and hasattr(ps.assigned_section, 'regular_track'):
                    is_top5 = ps.assigned_section.regular_track == 'TOP5'

```

```

except Exception:
    pass

if is_top5:
    if to_code in cls.REGULAR_TOP5_ALLOWED:
        return True, ""
    return False, (
        f"REGULAR TOP5 students can only move to: "
        f"{', '.join(cls.REGULAR_TOP5_ALLOWED)}."
    )
else:
    # HETERO — cannot move to any specialized program
    return False, (
        "REGULAR (Hetero) students cannot transfer to specialized programs. "
        "Specialized programs require qualifications that cannot be satisfied "
        "through a transfer request."
    )

# All other programs — check the allowed moves matrix
allowed = cls.ALLOWED_MOVES.get(from_code, [])
if to_code in allowed:
    return True, ""

return False, (
    f"Transfer from {from_code} to {to_code} is not permitted. "
    f"Allowed destinations from {from_code}: "
    f"{', '.join(allowed) if allowed else 'None'}."
)

```

admin.py – Django admin interface customization

```

from django.contrib import admin
from .models import UserProfile, Position, Department, Program, Teacher, Subject, Section

```

```

@admin.register(Position)
class PositionAdmin(admin.ModelAdmin):
    list_display = ['name', 'description', 'get_user_count', 'is_active', 'created_at']
    list_filter = ['is_active', 'created_at']
    search_fields = ['name', 'description']
    readonly_fields = ['created_at', 'updated_at', 'get_user_count']

    def get_user_count(self, obj):
        return obj.get_user_count()
    get_user_count.short_description = 'Users Count'

```

```

@admin.register(Department)
class DepartmentAdmin(admin.ModelAdmin):
    list_display = ['name', 'description', 'get_user_count', 'is_active', 'created_at']

```

```
list_filter = ['is_active', 'created_at']
search_fields = ['name', 'description']
readonly_fields = ['created_at', 'updated_at', 'get_user_count']
```

```
def get_user_count(self, obj):
    return obj.get_user_count()
get_user_count.short_description = 'Users Count'
```

```
@admin.register(Program)
```

```
class ProgramAdmin(admin.ModelAdmin):
    list_display = ['code', 'name', 'description', 'get_user_count', 'is_active', 'created_at']
    list_filter = ['is_active', 'created_at']
    search_fields = ['code', 'name', 'description']
    readonly_fields = ['created_at', 'updated_at', 'get_user_count']
```

```
def get_user_count(self, obj):
    return obj.get_user_count()
get_user_count.short_description = 'Users Count'
```

```
@admin.register(UserProfile)
```

```
class UserProfileAdmin(admin.ModelAdmin):
    list_display = ['user', 'user_type', 'program', 'position', 'department']
    list_filter = ['user_type', 'program', 'position', 'department']
    search_fields = ['user__username', 'user__email', 'user__first_name', 'user__last_name']
    raw_id_fields = ['user']
```

```
@admin.register(Teacher)
```

```
class TeacherAdmin(admin.ModelAdmin):
    list_display = ['get_full_name', 'position', 'department', 'email', 'is_adviser', 'created_at']
    list_filter = ['is_adviser', 'department', 'position']
    search_fields = ['first_name', 'middle_name', 'last_name', 'email']
    readonly_fields = ['created_at', 'updated_at']
    autocomplete_fields = ['position', 'department']
```

```
def get_full_name(self, obj):
    return obj.get_full_name()
get_full_name.short_description = 'Name'
```

```
@admin.register(Subject)
```

```
class SubjectAdmin(admin.ModelAdmin):
    list_display = ['code', 'name', 'program', 'is_active', 'updated_at']
    list_filter = ['program', 'is_active']
    search_fields = ['code', 'name', 'description']
    autocomplete_fields = ['program']
    readonly_fields = ['created_at', 'updated_at']
```

```
@admin.register(Section)
```



```

class SectionAdmin(admin.ModelAdmin):
    list_display = ['name', 'program', 'adviser', 'building', 'room', 'max_students',
'current_students']
    list_filter = ['program', 'building']
    search_fields = ['name', 'program__code', 'adviser__last_name', 'adviser__first_name']
    autocomplete_fields = ['program', 'adviser']
    readonly_fields = ['created_at', 'updated_at']

```

decorators.py – Custom decorators for authorization

```

from django.shortcuts import redirect
from django.contrib.auth.decorators import login_required
from functools import wraps

```

```

def admin_required(view_func):
    """Decorator to ensure user is an admin"""
    @wraps(view_func)
    @login_required
    def wrapper(request, *args, **kwargs):
        if hasattr(request.user, 'profile') and request.user.profile.user_type == 'admin':
            return view_func(request, *args, **kwargs)
        return redirect('admin_app:login')
    return wrapper

```

```

def coordinator_required(view_func):
    """Decorator to ensure user is a coordinator"""
    @wraps(view_func)
    @login_required
    def wrapper(request, *args, **kwargs):
        if hasattr(request.user, 'profile') and request.user.profile.user_type == 'coordinator':
            return view_func(request, *args, **kwargs)
        return redirect('admin_app:login')
    return wrapper

```

urls.py – Admin app URL routing

```

from django.urls import path
from .views import (
    analytics_views,
    dashboard_views,
    enrollment_views,
    log_in_out_views,
    masterlist_views,
    reports_views,
    sections_views,
    settings_views,
    studentdetails_views,
    studentedit_views,
    move_request_views,

```

```

)
from .views import archive_views

from .views.batch_upload_views import download_teacher_template,
batch_upload_teachers

app_name = 'admin_app'

urlpatterns = [
    # Authentication
    path('login/', log_in_out_views.admin_login, name='login'),
    path('logout/', log_in_out_views.admin_logout, name='logout'),

    # Dashboard
    path("", dashboard_views.dashboard, name='dashboard'),
    path('dashboard/', dashboard_views.dashboard, name='dashboard_alt'),

    # Dashboard API Endpoints
    path('api/dashboard/header/', dashboard_views.dashboard_header_data,
name='api_dashboard_header'),
    path('api/dashboard/statistics/', dashboard_views.dashboard_statistics,
name='api_dashboard_statistics'),
    path('api/dashboard/notifications/', dashboard_views.dashboard_notifications,
name='api_dashboard_notifications'),
    path('api/dashboard/programs/', dashboard_views.dashboard_programs_overview,
name='api_dashboard_programs'),

    # Analytics
    path('analytics/', analytics_views.analytics, name='analytics'),
    path('api/analytics/header/', analytics_views.analytics_header_data,
name='api_analytics_header'),
    path('api/analytics/data/', analytics_views.analytics_data, name='analytics_data'),

    # Enrollment
    path('enrollment/', enrollment_views.enrollment_list, name='enrollment'),
    path('enrollment/<str:student_id>', enrollment_views.enrollment_detail,
name='enrollment_detail'),

    # Enrollment API
    path('api/enrollment/header/', enrollment_views.enrollment_header_data,
name='api_enrollment_header'),
    path('api/enrollment/summary/', enrollment_views.enrollment_summary,
name='api_enrollment_summary'),
    path('api/enrollment/requests/', enrollment_views.enrollment_requests,
name='api_enrollment_requests'),

```

```

# Sections
path('sections/', sections_views.sections_list, name='sections'),
path('api/sections/header/', sections_views.sections_header_data,
name='api_sections_header'),
path('sections/<str:program>', sections_views.sections_by_program,
name='sections_by_program'),
path('sections/<str:program>/<int:section_id>', sections_views.section_detail,
name='section_detail'),

# Sections/Subjects/Buildings API
path('api/programs/', sections_views.get_programs, name='api_get_programs'),
path('api/programs/all/', sections_views.get_all_programs, name='api_get_all_programs'),
path('api/programs/add/', sections_views.add_program, name='api_add_program'),
path('api/programs/<int:program_id>/update/', sections_views.update_program,
name='api_update_program'),
path('api/programs/<int:program_id>/delete/', sections_views.delete_program,
name='api_delete_program'),
path('api/programs/<int:program_id>/toggle-status/',
sections_views.toggle_program_status, name='api_toggle_program_status'),
path('api/teachers/', sections_views.get_teachers, name='api_get_teachers'),
path('api/buildings/', sections_views.get_buildings, name='api_get_buildings'),
path('api/rooms/', sections_views.get_rooms, name='api_get_rooms'),
path('api/subjects/', sections_views.get_subjects, name='api_get_subjects'),
path('api/subjects/add/', sections_views.add_subject, name='api_add_subject'),
path('api/subjects/<int:subject_id>/update/', sections_views.update_subject,
name='api_update_subject'),
path('api/subjects/<int:subject_id>/delete/', sections_views.delete_subject,
name='api_delete_subject'),
path('api/sections/', sections_views.get_sections, name='api_get_sections'),
path('api/sections/add/', sections_views.add_section, name='api_add_section'),
path('api/sections/<int:section_id>/update/', sections_views.update_section,
name='api_update_section'),
path('api/sections/<int:section_id>/delete/', sections_views.delete_section,
name='api_delete_section'),

# Masterlist
path('masterlist/', masterlist_views.masterlist, name='masterlist'),
path('masterlist/<int:section_id>', masterlist_views.masterlist_by_section,
name='masterlist_by_section'),

# Student Details & Edit
path('student/<str:lrn>', studentdetails_views.student_details, name='student_details'),
path('api/student/<str:lrn>/lift-probation/', studentdetails_views.lift_probation,
name='lift_probation'),
path('student/<str:student_id>/edit/', studentedit_views.student_edit,
name='student_edit'),

```

```

# student edit-move
path('api/move-requests/pending/', move_request_views.get_pending_move_requests,
name='api_pending_move_requests'),
path('api/move-requests/<int:move_request_id>/review/',
move_request_views.review_move_request, name='api_review_move_request'),
path('api/move-requests/sections/', move_request_views.get_sections_for_program,
name='api_sections_for_program'),
path('api/sections/', studentedit_views.get_sections_by_program, name='api_sections'),
path('api/admin-move/', studentedit_views.admin_move_student, name='api_admin_move'),

```

```

# Student Edit API Endpoints
path('api/student/<str:student_id>/details/', studentedit_views.get_student_details,
name='api_get_student_details'),
path('api/student/<str:student_id>/update/student-data/',
studentedit_views.update_student_data, name='api_update_student_data'),
path('api/student/<str:student_id>/update/family-data/',
studentedit_views.update_family_data, name='api_update_family_data'),
path('api/student/<str:student_id>/update/survey-data/',
studentedit_views.update_survey_data, name='api_update_survey_data'),
path('api/student/<str:student_id>/update/academic-data/',
studentedit_views.update_academic_data, name='api_update_academic_data'),
path('api/student/<str:student_id>/update/program-selection/',
studentedit_views.update_program_selection, name='api_update_program_selection'),
path('api/student/<str:student_id>/update/status/',
studentedit_views.update_enrollment_status, name='api_update_enrollment_status'),
path('api/student/<str:student_id>/upload/', studentedit_views.upload_student_file,
name='api_upload_student_file'),

```

```

# Reports
path('reports/', reports_views.reports, name='reports'),
path('api/reports/header/', reports_views.reports_header_data,
name='api_reports_header'),
path('reports/generate/', reports_views.generate_report, name='generate_report'),
path('api/reports/summary/', reports_views.reports_summary,
name='api_reports_summary'),
path('api/reports/preview/', reports_views.get_report_preview,
name='api_reports_preview'),
path('api/reports/export/enrolled/', reports_views.export_enrolled_students,
name='export_enrolled'),
path('api/reports/export/sections/', reports_views.export_section_list,
name='export_sections'),
path('api/reports/export/promotion/', reports_views.export_promotion_status,
name='export_promotion'),
path('api/reports/export/pending/', reports_views.export_pending_enrollments,
name='export_pending'),
path('api/reports/export/masterlist/', reports_views.export_masterlist,
name='export_masterlist'),

```

```

    path('api/reports/export/transferees/', reports_views.export_transferees,
name='export_transferees'),
    path('api/reports/export/probation/', reports_views.export_probation_list,
name='export_probation'),
    path('api/reports/export/ai-vs-manual/', reports_views.export_ai_vs_manual,
name='export_ai_vs_manual'),
    path('api/reports/export/no-section/', reports_views.export_no_section,
name='export_no_section'),
    path('api/reports/export/documents/', reports_views.export_document_compliance,
name='export_documents'),
    path('api/reports/export/activity-log/', reports_views.export_activity_log,
name='export_activity_log'),
    path('api/reports/export/enrollee-breakdown/',
reports_views.export_enrollee_type_breakdown, name='export_enrollee_breakdown'),
    path('api/reports/export/move-requests/', reports_views.export_move_requests,
name='export_move_requests'),

```

Settings

```

    path('settings/', settings_views.settings, name='settings'),
    path('api/settings/header/', settings_views.settings_header_data,
name='api_settings_header'),
    path('settings/users/', settings_views.manage_users, name='manage_users'),
    path('settings/content/', settings_views.manage_content, name='manage_content'),
    path('api/generate-promotion-statuses/', settings_views.generate_promotion_statuses,
name='api_generate_promotion_statuses'),

```

Settings API Endpoints

```

    path('api/users/', settings_views.get_users, name='api_get_users'),
    path('api/users/add/', settings_views.add_user, name='api_add_user'),
    # User CRUD API Endpoints (Settings > Users)
    path('api/users/<int:user_id>', settings_views.get_user_profile,
name='api_get_user_profile'),
    path('api/users/<int:user_id>/update/', settings_views.update_user_profile,
name='api_update_user_profile'),
    path('api/users/<int:user_id>/delete/', settings_views.delete_user_profile,
name='api_delete_user_profile'),
    path('api/positions/', settings_views.get_positions, name='api_get_positions'),
    path('api/positions/add/', settings_views.add_position, name='api_add_position'),
    path('api/positions/<int:position_id>/update/', settings_views.update_position,
name='api_update_position'),
    path('api/positions/<int:position_id>/delete/', settings_views.delete_position,
name='api_delete_position'),
    path('api/departments/', settings_views.get_departments, name='api_get_departments'),
    path('api/departments/add/', settings_views.add_department,
name='api_add_department'),
    path('api/departments/<int:department_id>/update/', settings_views.update_department,
name='api_update_department'),

```

```

    path('api/departments/<int:department_id>/delete/', settings_views.delete_department,
name='api_delete_department'),
    # Teacher Management API Endpoints
    path('api/settings/teachers/', settings_views.get_teachers_for_settings,
name='api_get_teachers_for_settings'),
    path('api/settings/teachers/add/', settings_views.add_teacher, name='api_add_teacher'),
    path('api/settings/teachers/<int:teacher_id>/update/', settings_views.update_teacher,
name='api_update_teacher'),
    path('api/settings/teachers/<int:teacher_id>/delete/', settings_views.delete_teacher,
name='api_delete_teacher'),
    # School Years API Endpoints
    path('api/school-years/', settings_views.get_school_years, name='api_get_school_years'),
    path('api/school-years/add/', settings_views.add_school_year,
name='api_add_school_year'),
    path('api/school-years/<int:school_year_id>/update/', settings_views.update_school_year,
name='api_update_school_year'),
    path('api/school-years/<int:school_year_id>/delete/', settings_views.delete_school_year,
name='api_delete_school_year'),

    # Document Requirements API Endpoints (Settings > Others)
    path('api/document-requirements/', settings_views.get_document_requirements,
name='api_get_document_requirements'),
    path('api/document-requirements/add/', settings_views.add_document_requirement,
name='api_add_document_requirement'),
    path('api/document-requirements/<int:requirement_id>/update/',
settings_views.update_document_requirement,
name='api_update_document_requirement'),
    path('api/document-requirements/<int:requirement_id>/delete/',
settings_views.delete_document_requirement, name='api_delete_document_requirement'),

    # Content Management API Endpoints
    path('api/content/settings/', settings_views.get_content_settings,
name='api_get_content_settings'),
    path('api/content/save/', settings_views.save_content_setting,
name='api_save_content_setting'),
    path('api/content/upload-image/', settings_views.upload_content_image,
name='api_upload_content_image'),
    path('api/staff/', settings_views.get_staff_members, name='api_get_staff_members'),
    path('api/staff/add/', settings_views.add_staff_member, name='api_add_staff_member'),
    path('api/staff/<int:staff_id>/update/', settings_views.update_staff_member,
name='api_update_staff_member'),
    path('api/staff/<int:staff_id>/delete/', settings_views.delete_staff_member,
name='api_delete_staff_member'),
    path('api/content/delete-image/', settings_views.delete_content_image,
name='delete_content_image'),

    # Buildings & Rooms API Endpoints

```

```

    path('api/buildings/', settings_views.get_buildings_with_rooms,
name='api_get_buildings_with_rooms'),
    path('api/buildings/add/', settings_views.add_building, name='api_add_building'),
    path('api/buildings/<int:building_id>/update/', settings_views.update_building,
name='api_update_building'),
    path('api/buildings/<int:building_id>/delete/', settings_views.delete_building,
name='api_delete_building'),
    # Note: api/rooms/ is handled by sections_views.get_rooms (line 42)
    path('api/rooms/add/', settings_views.add_room, name='api_add_room'),
    path('api/rooms/<int:room_id>/update/', settings_views.update_room,
name='api_update_room'),
    path('api/rooms/<int:room_id>/delete/', settings_views.delete_room,
name='api_delete_room'),

    # Grade Level API Endpoints (Settings > Others)
    path('api/grade-levels/', settings_views.get_grade_levels, name='api_get_grade_levels'),
    path('api/grade-levels/add/', settings_views.add_grade_level,
name='api_add_grade_level'),
    path('api/grade-levels/<int:grade_level_id>/update/', settings_views.update_grade_level,
name='api_update_grade_level'),
    path('api/grade-levels/<int:grade_level_id>/delete/', settings_views.delete_grade_level,
name='api_delete_grade_level'),

    # Activity Logs API Endpoint
    path('api/activity-logs/', settings_views.get_activity_logs, name='api_get_activity_logs'),

    # Archive
    path('archive/', archive_views.archive, name='archive'),
    path('api/archive/school-years/', archive_views.archive_school_years,
name='api_archive_school_years'),
    path('api/archive/enrollment/', archive_views.archive_enrollment,
name='api_archive_enrollment'),
    path('api/archive/sections/', archive_views.archive_sections,
name='api_archive_sections'),
    path('api/archive/academic/', archive_views.archive_academic,
name='api_archive_academic'),
    path('api/archive/probation/', archive_views.archive_probation,
name='api_archive_probation'),
    path('api/archive/export/enrollment/', archive_views.export_archive_enrollment,
name='api_archive_export_enrollment'),
    path('api/archive/export/sections/', archive_views.export_archive_sections,
name='api_archive_export_sections'),
    path('api/archive/export/academic/', archive_views.export_archive_academic,
name='api_archive_export_academic'),
    path('api/archive/export/probation/', archive_views.export_archive_probation,
name='api_archive_export_probation'),

```

```

    # Batch Upload URLs
    path('api/download-teacher-template/', download_teacher_template,
         name='download_teacher_template'),
    path('api/batch-upload-teachers/', batch_upload_teachers,
         name='batch_upload_teachers'),
]

```

Admin Views

log_in_out_views.py – User authentication (Login and logout functionality)

```

from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.http import JsonResponse
from admin_app.models import Program
import logging

```

```

logger = logging.getLogger(__name__)

```

```

def admin_login(request):

```

```

    """

```

```

    Unified login view for Admin and Coordinator

```

```

    - GET : render login page with program list
    - POST : authenticate user and validate role + program
    """

```

```

    # Already authenticated

```

```

    if request.user.is_authenticated:
        if hasattr(request.user, 'profile'):
            if request.user.profile.user_type == 'admin':
                return redirect('admin_app:dashboard')
            elif request.user.profile.user_type == 'coordinator':
                return redirect('coordinator:dashboard')
        return redirect('admin_app:dashboard')

```

```

    # POST (AJAX login)

```

```

    if request.method == 'POST':
        try:
            username = request.POST.get('username', "").strip()
            password = request.POST.get('password', "")
            user_type = request.POST.get('user_type', 'admin')
            program_id = request.POST.get('program', "").strip()

```

```

            logger.info(f"Login attempt - Username: {username}, Type: {user_type}, Program ID: {program_id}")

```



```

# Basic validation
if not username or not password:
    return JsonResponse({
        'success': False,
        'message': 'Username and password are required.'
    }, status=400)

# Authenticate
user = authenticate(request, username=username, password=password)
if user is None:
    logger.warning(f"Authentication failed for username: {username}")
    return JsonResponse({
        'success': False,
        'message': 'Invalid username or password.'
    }, status=401)

# Profile check
if not hasattr(user, 'profile'):
    logger.error(f"User {username} has no profile")
    return JsonResponse({
        'success': False,
        'message': 'User profile not found. Please contact administrator.'
    }, status=400)

# Role validation
if user.profile.user_type != user_type:
    logger.warning(f"Role mismatch for {username}. Expected: {user_type}, Got: {user.profile.user_type}")
    return JsonResponse({
        'success': False,
        'message': f'This account is not registered as a {user_type}. Please select the correct role.'
    }, status=403)

# Coordinator-specific checks
if user_type == 'coordinator':
    # Check if program was selected
    if not program_id:
        return JsonResponse({
            'success': False,
            'message': 'Please select a program.'
        }, status=400)

# Fetch program from DB
try:
    selected_program = Program.objects.get(id=program_id)

```

```

        logger.info(f"Selected program: {selected_program.code} (ID:
{selected_program.id})")
    except Program.DoesNotExist:
        logger.error(f"Program ID {program_id} does not exist")
        return JsonResponse({
            'success': False,
            'message': 'Selected program does not exist.'
        }, status=400)

    # Check if coordinator has a program assigned
    if not user.profile.program:
        logger.error(f"Coordinator {username} has no program assigned in profile")
        return JsonResponse({
            'success': False,
            'message': 'Your account has no program assigned. Please contact the
administrator.'
        }, status=403)

    # Verify coordinator is assigned to this program
    logger.info(f"User's assigned program: {user.profile.program.code} (ID:
{user.profile.program.id})")

    if user.profile.program.id != int(program_id):
        logger.warning(
            f"Program mismatch for {username}. "
            f"Selected: {selected_program.code}, "
            f"Assigned: {user.profile.program.code}"
        )
        return JsonResponse({
            'success': False,
            'message': f"You are not authorized for {selected_program.code}. "
            f"Your assigned program is {user.profile.program.code}."
        }, status=403)

    # Login success
    login(request, user)
    logger.info(f"Successful login for {username} as {user_type}")

    if user.profile.user_type == 'admin':
        redirect_url = '/admin-portal/dashboard/'
        dashboard_name = 'Admin Dashboard'
    else:
        redirect_url = '/coordinator/dashboard/'
        dashboard_name = f'{user.profile.program.code} Coordinator Dashboard'

    return JsonResponse({
        'success': True,

```

```

        'message': f'Welcome to {dashboard_name}!',
        'redirect_url': redirect_url,
        'user_type': user.profile.user_type,
        'username': user.get_full_name() or user.username
    })

except Exception as e:
    logger.exception(f'Login error: {str(e)}')
    return JsonResponse({
        'success': False,
        'message': f'An error occurred: {str(e)}'
    }, status=500)

# GET (render login page)
programs = Program.objects.filter(is_active=True).order_by('code')

return render(request, 'admin_app/login.html', {
    'programs': programs
})

@login_required
def admin_logout(request):
    """Logout view for both Admin and Coordinator"""
    logout(request)
    return redirect('admin_app:login')

def logout_page(request):
    """Render logout confirmation page"""
    return render(request, 'admin_app/logout.html')

dashboard_views.py – Main admin dashboard (System overview and statistics)
from django.shortcuts import render
from django.http import JsonResponse
from django.db.models import Count, Q, F
from django.utils import timezone
from django.contrib.auth.decorators import login_required
from admin_app.decorators import admin_required
from admin_app.models import (
    SchoolYear, UserProfile, Teacher, Section, Program
)
from enrollment_app.models import Student, StudentData, ProgramSelection,
StudentEnrollment
from django.db.models import Q
from datetime import datetime, timedelta

@admin_required
def dashboard(request):

```

```

"""
Main admin dashboard view
"""

# Get active school year
active_school_year = SchoolYear.get_active_school_year()

# Get user profile
try:
    user_profile = UserProfile.objects.select_related('program', 'position',
'department').get(user=request.user)
except UserProfile.DoesNotExist:
    user_profile = None

context = {
    'user': request.user,
    'user_profile': user_profile,
    'active_school_year': active_school_year,
}
return render(request, 'admin_app/dashboard.html', context)

@admin_required
def dashboard_header_data(request):
    """
    API endpoint for dashboard header data
    Returns: school year, user fullname, role, and photo/initials
    """

    # Get active school year
    active_school_year = SchoolYear.get_active_school_year()

    # Get user profile
    try:
        user_profile = UserProfile.objects.select_related('program', 'position',
'department').get(user=request.user)
    except UserProfile.DoesNotExist:
        user_profile = None

    # Get user's full name
    user = request.user
    full_name = f"{user.first_name} {user.last_name}" if user.first_name and user.last_name else
user.username

    # Get initials
    if user.first_name and user.last_name:
        initials = f"{user.first_name[0]}{user.last_name[0]}.upper()"
    elif user.first_name:
        initials = user.first_name[0].upper()
    elif user.last_name:

```

```

        initials = user.last_name[0].upper()
    else:
        initials = user.username[0].upper() if user.username else "U"

    # Get role
    role = user_profile.get_user_type_display() if user_profile else "Admin"

    # Get photo URL
    photo_url = None
    if user_profile and user_profile.photo:
        photo_url = user_profile.photo.url

    data = {
        'school_year': active_school_year.year_label if active_school_year else 'No Active Year',
        'full_name': full_name,
        'role': role,
        'initials': initials,
        'photo_url': photo_url,
        'program': user_profile.get_program_name() if user_profile else 'N/A',
    }

    return JsonResponse(data)

@admin_required
def dashboard_statistics(request):
    """
    API endpoint for dashboard statistics
    Returns: total teachers, students, programs, sections
    """

    # Get active school year
    active_school_year = SchoolYear.get_active_school_year()

    # Get statistics
    total_teachers = Teacher.objects.count()
    total_programs = Program.objects.count()

    if active_school_year:
        # Statistics for active school year
        total_students = Student.objects.filter(
            school_year=active_school_year,
            enrollment_status__in=['submitted', 'under_review', 'approved']
        ).count()
        total_sections = Section.objects.filter(school_year=active_school_year).count()

    # Grade level breakdown
    grade_7_students = Student.objects.filter(
        school_year=active_school_year,

```

```
        enrollment_status__in=['submitted', 'under_review', 'approved']
    ).count() # You might need to add grade_level field to filter properly
```

```
else:
```

```
    total_students = 0
    total_sections = 0
    grade_7_students = 0
```

```
data = {
    'total_teachers': total_teachers,
    'total_students': total_students,
    'total_programs': total_programs,
    'total_sections': total_sections,
    'grade_breakdown': {
        'grade_7': grade_7_students // 4 if grade_7_students else 0, # Simplified distribution
        'grade_8': grade_7_students // 4 if grade_7_students else 0,
        'grade_9': grade_7_students // 4 if grade_7_students else 0,
        'grade_10': grade_7_students // 4 if grade_7_students else 0,
    }
}
```

```
return JsonResponse(data)
```

```
@admin_required
```

```
def dashboard_notifications(request):
```

```
    """
```

```
    API endpoint for new student enrollment notifications
```

```
    Returns notifications grouped by program
```

```
    """
```

```
    # Get active school year
```

```
    active_school_year = SchoolYear.get_active_school_year()
```

```
    if not active_school_year:
```

```
        return JsonResponse({'notifications': [], 'total_count': 0})
```

```
    tab = request.GET.get('tab', 'pending')
```

```
    status_map = {
```

```
        'pending': 'submitted',
        'review': 'under_review',
        'approved': 'approved'
```

```
    }
```

```
    from enrollment_app.models import StudentEnrollment
```

```
    enrollments = StudentEnrollment.objects.filter(
```

```
        school_year=active_school_year,
```

```
        enrollment_status=status_map[tab]
```

```
    ).select_related(
```

```
        'student__program_selection',
```

```

        'student__student_data'
   )[:100]

# Convert to student-like structure for existing logic (safe prefetch)
new_students = []
for enrollment in enrollments:
    if hasattr(enrollment.student, 'program_selection'):
        new_students.append(enrollment.student)
    else:
        continue

# Group by program
notifications = []
program_counts = {}

for student in new_students:
    try:
        program_selection = student.program_selection
        program_code = program_selection.selected_program_code

        if program_code not in program_counts:
            program_counts[program_code] = {
                'count': 0,
                'students': []
            }

        program_counts[program_code]['count'] += 1

        # Get student details
        try:
            student_data = student.student_data
            student_name = student_data.full_name
        except:
            student_name = f"Student {student.lrn}"

        program_counts[program_code]['students'].append({
            'lrn': student.lrn,
            'name': student_name,
            'created_at': student.created_at.strftime('%b %d, %Y %l:%M %p'),
            'time_ago': get_time_ago(student.created_at)
        })
    except:
        continue

# Create notification list
for program_code, data in program_counts.items():
    try:

```

```

program = Program.objects.get(code=program_code)
icon_map = {
    'STE': 'fas fa-flask',
    'SPFL': 'fas fa-language',
    'SPTVE': 'fas fa-tools',
    'OHSP': 'fas fa-laptop-house',
    'SNED': 'fas fa-universal-access',
    'REGULAR': 'fas fa-users',
    'TOP 5': 'fas fa-trophy',
    'HETERO': 'fas fa-users',
}

notifications.append({
    'program_code': program_code,
    'program_name': program.name,
    'icon': icon_map.get(program_code, 'fas fa-users'),
    'count': data['count'],
    'students': data['students'][:5], # Limit to 5 most recent
    'message': f'{data["count"]} new {program_code} application{'s' if data["count"] > 1 else
"} pending review"
})
except Program.DoesNotExist:
    continue

# Sort by count (highest first)
notifications.sort(key=lambda x: x['count'], reverse=True)

total_count = sum(n['count'] for n in notifications)

return JsonResponse({
    'notifications': notifications,
    'total_count': total_count
})

@admin_required
def dashboard_programs_overview(request):
    """
    API endpoint for programs overview table
    Returns detailed program statistics
    """
    # Get active school year
    active_school_year = SchoolYear.get_active_school_year()

    if not active_school_year:
        return JsonResponse({'programs': []})

    # Get all active programs

```



```

programs = Program.objects.filter(is_active=True).order_by('code')

programs_data = []
total_approved = 0
total_pending = 0
total_rejected = 0

for program in programs:
    # Get program statistics
    program_students = ProgramSelection.objects.filter(
        school_year=active_school_year,
        selected_program_code=program.code
    ).select_related('student')

    total = program_students.count()
    approved = program_students.filter(student__enrollment_status='approved').count()
    pending = program_students.filter(student__enrollment_status__in=['submitted',
'under_review']).count()
    rejected = program_students.filter(student__enrollment_status='rejected').count()

    # Get sections for this program
    sections = Section.objects.filter(
        school_year=active_school_year,
        program=program
    )
    section_count = sections.count()
    capacity = sum(s.max_students for s in sections)

    # Calculate acceptance rate
    reviewed = approved + rejected
    acceptance_rate = (approved / reviewed * 100) if reviewed > 0 else 0

    # Determine trend (you can calculate from previous year if needed)
    trend = 'stable'
    trend_value = 0

    programs_data.append({
        'code': program.code,
        'name': program.name,
        'status': 'active',
        'total_applicants': total,
        'approved': approved,
        'pending': pending,
        'rejected': rejected,
        'capacity': capacity,
        'sections': section_count,
        'acceptance_rate': round(acceptance_rate, 1),

```

```

        'trend': trend,
        'trend_value': trend_value
    })

    total_approved += approved
    total_pending += pending
    total_rejected += rejected

# Calculate overall acceptance rate
total_reviewed = total_approved + total_rejected
overall_acceptance_rate = (total_approved / total_reviewed * 100) if total_reviewed > 0 else 0

return JsonResponse({
    'programs': programs_data,
    'summary': {
        'total_approved': total_approved,
        'total_pending': total_pending,
        'total_rejected': total_rejected,
        'overall_acceptance_rate': round(overall_acceptance_rate, 1)
    }
})

def get_time_ago(dt):
    """
    Helper function to get human-readable time difference
    """
    now = timezone.now()
    diff = now - dt

    if diff.days > 0:
        if diff.days == 1:
            return "1 day ago"
        elif diff.days < 7:
            return f"{diff.days} days ago"
        elif diff.days < 30:
            weeks = diff.days // 7
            return f"{weeks} week{'s' if weeks > 1 else ''} ago"
        else:
            months = diff.days // 30
            return f"{months} month{'s' if months > 1 else ''} ago"

    hours = diff.seconds // 3600
    if hours > 0:
        return f"{hours} hour{'s' if hours > 1 else ''} ago"

    minutes = diff.seconds // 60
    if minutes > 0:

```

```
    return f"{minutes} minute{'s' if minutes > 1 else ''} ago"
```

```
    return "Just now"
```

enrollment_views.py – Enrollment management (New student, transferee, continuing enrollment)

```
from django.shortcuts import render
from django.http import JsonResponse
from django.urls import reverse
from django.contrib.auth.decorators import login_required
from django.db.models import Q
```

```
from admin_app.decorators import admin_required
from admin_app.models import Program, SchoolYear, UserProfile
from enrollment_app.models import StudentEnrollment, StudentData, ProgramSelection,
SurveyData
from enrollment_app.models import Student
```

```
def _get_school_year_from_request(request):
    """Resolve school year from query parameter or fall back to active one."""
    school_year_id = request.GET.get('school_year')
    if school_year_id:
        return SchoolYear.objects.filter(id=school_year_id).first()
    return SchoolYear.get_active_school_year()
```

```
@admin_required
```

```
def enrollment_list(request):
    """Render enrollment page with dynamic filter options."""
    school_years = SchoolYear.objects.all().order_by('-id')
    programs = Program.objects.all().order_by('code')
    active_school_year = SchoolYear.get_active_school_year()
```

```
# Get user profile
```

```
try:
```

```
    user_profile = UserProfile.objects.select_related('program', 'position',
'department').get(user=request.user)
except UserProfile.DoesNotExist:
    user_profile = None
```

```
context = {
    'school_years': school_years,
    'programs': programs,
    'active_school_year': active_school_year,
    'user': request.user,
    'user_profile': user_profile,
    'active_page': 'enrollment',
}
```

```
return render(request, 'admin_app/enrollment.html', context)
```

```
@admin_required
```

```
def enrollment_detail(request, student_id):
```

```
    """Render enrollment detail page."""
```

```
    # Get user profile
```

```
    try:
```

```
        user_profile = UserProfile.objects.select_related('program', 'position',  
'department').get(user=request.user)
```

```
    except UserProfile.DoesNotExist:
```

```
        user_profile = None
```

```
    context = {
```

```
        'student_id': student_id,
```

```
        'user': request.user,
```

```
        'user_profile': user_profile,
```

```
        'active_page': 'enrollment',
```

```
    }
```

```
    return render(request, 'admin_app/enrollment.html', context)
```

```
@admin_required
```

```
def enrollment_header_data(request):
```

```
    """
```

```
    API endpoint for enrollment header data
```

```
    Returns: school year, user fullname, role, and photo/initials
```

```
    """
```

```
    # Get active school year
```

```
    active_school_year = SchoolYear.get_active_school_year()
```

```
    # Get user profile
```

```
    try:
```

```
        user_profile = UserProfile.objects.select_related('program', 'position',  
'department').get(user=request.user)
```

```
    except UserProfile.DoesNotExist:
```

```
        user_profile = None
```

```
    # Get user's full name
```

```
    user = request.user
```

```
    full_name = f"{user.first_name} {user.last_name}" if user.first_name and user.last_name else  
user.username
```

```
    # Get initials
```

```
    if user.first_name and user.last_name:
```

```
        initials = f"{user.first_name[0]}{user.last_name[0]}.upper()
```

```
    elif user.first_name:
```

```
        initials = user.first_name[0].upper()
```

```
    elif user.last_name:
```

```

        initials = user.last_name[0].upper()
    else:
        initials = user.username[0].upper() if user.username else "U"

    # Get role
    role = user_profile.get_user_type_display() if user_profile else "Admin"

    # Get photo URL
    photo_url = None
    if user_profile and user_profile.photo:
        photo_url = user_profile.photo.url

    data = {
        'school_year': active_school_year.year_label if active_school_year else 'No Active Year',
        'full_name': full_name,
        'role': role,
        'initials': initials,
        'photo_url': photo_url,
        'program': user_profile.get_program_name() if user_profile else 'N/A',
    }

    return JsonResponse(data)

@admin_required
def enrollment_summary(request):
    """Return per-grade-level counts for the 4 grade cards."""
    school_year = _get_school_year_from_request(request)

    pending_statuses = ['submitted', 'under_review']

    # Grade level codes and their display names
    grade_levels = [
        ('G7', 'Grade 7'),
        ('G8', 'Grade 8'),
        ('G9', 'Grade 9'),
        ('G10', 'Grade 10'),
    ]

    grades_data = []
    for code, name in grade_levels:
        qs = StudentEnrollment.objects.filter(
            school_year=school_year,
            grade_level__code=code
        ).exclude(enrollment_status='draft')
        total = qs.count()
        pending = qs.filter(enrollment_status__in=pending_statuses).count()
        approved = qs.filter(enrollment_status='approved').count()

```

```

        grades_data.append({
            'code': code,
            'name': name,
            'total': total,
            'pending': pending,
            'approved': approved,
        })

    return JsonResponse({
        'school_year': school_year.year_label if school_year else None,
        'grades': grades_data,
    })

@admin_required
def enrollment_requests(request):
    """Return list of enrollment requests with filters including grade level."""
    school_year = _get_school_year_from_request(request)
    program_filter = request.GET.get('program')
    status_filter = request.GET.get('status')
    grade_filter = request.GET.get('grade') # NEW

    qs = StudentEnrollment.objects.filter(
        school_year=school_year
    ).exclude(enrollment_status='draft').select_related(
        'student__student_data',
        'student__program_selection',
        'student__survey_data',
        'grade_level',
        'student'
    )

    # DB filters applied BEFORE slicing
    if grade_filter and grade_filter.lower() != 'all':
        qs = qs.filter(grade_level__code=grade_filter)

    if status_filter and status_filter.lower() != 'all':
        if status_filter == 'pending':
            qs = qs.filter(enrollment_status__in=['submitted', 'under_review'])
        else:
            qs = qs.filter(enrollment_status=status_filter)

    # Slice AFTER all DB filters are applied
    qs = qs[:500]

    # Python filter for program (avoids join error)
    filtered_qs = []

```

```

for enrollment in qs:
    student = enrollment.student
    program_selection = getattr(student, 'program_selection', None)
    program_code = program_selection.selected_program_code if program_selection else
None

    if program_filter and program_filter.lower() != 'all' and (
        not program_code or program_code.lower() != program_filter.lower()
    ):
        continue
    filtered_qs.append(enrollment)

results = []
for enrollment in filtered_qs:
    student = enrollment.student
    student_data = getattr(student, 'student_data', None)
    program_selection = getattr(student, 'program_selection', None)
    survey_data = getattr(student, 'survey_data', None)

    full_name = student_data.full_name if student_data else 'N/A'
    program_code = program_selection.selected_program_code if program_selection else
'N/A'
    grade_level = enrollment.grade_level.name if enrollment.grade_level else (
        getattr(survey_data, 'current_grade_section', None) or 'N/A'
    )

    results.append({
        'ltn': student.ltn,
        'student_name': full_name,
        'program': program_code,
        'grade': grade_level,
        'submitted_at': student.created_at.strftime('%b %d, %Y'),
        'status': enrollment.enrollment_status, # Use enrollment status!
        'detail_url': reverse('admin_app:student_edit', args=[student.ltn]),
    })

return JsonResponse({
    'results': results,
    'total': len(filtered_qs)
})

```

masterlist_views.py – Master list generation and management (Section master list and Publishing and archival)

```

from django.shortcuts import render, get_object_or_404
from django.contrib.auth.decorators import login_required
from admin_app.decorators import admin_required

```

```
from admin_app.models import Section, SchoolYear
from enrollment_app.models import ProgramSelection, Student, StudentData, AcademicData
```

```
@admin_required
def masterlist(request):
    """
    General masterlist landing — redirects to sections.
    """
    return render(request, 'admin_app/masterlist.html', {})
```

```
@admin_required
def masterlist_by_section(request, section_id):
    """
    Masterlist for a specific section.
    Fetches all approved students assigned to this section.
    """
    section = get_object_or_404(
        Section.objects.select_related('program', 'adviser', 'grade_level', 'school_year'),
        pk=section_id
    )

    # Get all approved program selections for this section
    program_selections = ProgramSelection.objects.filter(
        assigned_section=section,
        admin_approved=True,
    ).select_related('student')

    # Build enriched student list
    students = []
    for ps in program_selections:
        student = ps.student
        try:
            student_data = student.student_data
        except Exception:
            student_data = None

        try:
            academic_data = student.academic_data
        except Exception:
            academic_data = None

        students.append({
            'ln': student.ln,
            'student_data': student_data,
            'program_selection': ps,
            'academic_data': academic_data,
```



```

        'get_enrollee_type_display': student.get_enrollee_type_display() if student.enrollee_type
    else '—',
    })

```

```

# Sort alphabetically by last name

```

```

students.sort(key=lambda x: x['student_data'].last_name.lower() if x['student_data'] else "")

```

```

active_school_year = (
    SchoolYear.objects.filter(is_active=True).first()
    or SchoolYear.objects.order_by('-start_date').first()
)

```

```

context = {
    'section': section,
    'students': students,
    'active_page': 'sections',
    'active_school_year': active_school_year, # ← ADD
}
return render(request, 'admin_app/masterlist.html', context)

```

sections_views.py – Section management (Create, edit, delete sections, and Teacher assignments)

```

from django.shortcuts import render
from django.contrib.auth.decorators import login_required
from django.http import JsonResponse
from django.views.decorators.http import require_http_methods
from django.db import transaction
import json
from admin_app.decorators import admin_required
from admin_app.models import Program, SchoolYear, UserProfile
from admin_app.models import Program, Teacher, Subject, Section, Building, Room,
ActivityLog, GradeLevel

```

```

def log_activity(user, action, description, request=None):

```

```

    """
    Helper function to log activities to the database
    """

```

```

    try:

```

```

        ip_address = None
        user_agent = None

```

```

    if request:

```

```

        # Get IP address
        x_forwarded_for = request.META.get('HTTP_X_FORWARDED_FOR')
        if x_forwarded_for:
            ip_address = x_forwarded_for.split(',')[0].strip()

```

```

        else:
            ip_address = request.META.get('REMOTE_ADDR')

        # Get user agent
        user_agent = request.META.get('HTTP_USER_AGENT', "")[:500]

        ActivityLog.objects.create(
            user=user,
            action=action,
            description=description,
            ip_address=ip_address,
            user_agent=user_agent
        )
    except Exception as e:
        # Log silently to avoid breaking the main functionality
        print(f"Error logging activity: {str(e)}")

@login_required
def sections_list(request):
    return render(request, 'admin_app/sections.html', {'active_page': 'sections'})

@login_required
def sections_by_program(request, program):
    return render(request, 'admin_app/sections.html', {'program': program})

@login_required
def section_detail(request, program, section_id):
    return render(request, 'admin_app/sections.html', {
        'program': program,
        'section_id': section_id
    })

# ===== HELPER FUNCTIONS =====

def _get_program_by_code(code):
    """Get program by code (case-insensitive)"""
    try:
        return Program.objects.get(code__iexact=code)
    except Program.DoesNotExist:
        return None

def _normalize_program_code(code):
    """Normalize TOP5/HETERO to REGULAR program with track info"""
    code_upper = (code or "").upper().strip()
    if code_upper in ['TOP5', 'TOP 5', 'HETERO']:
        regular_track = 'TOP5' if 'TOP' in code_upper else 'HETERO'
        return 'REGULAR', regular_track

```

```
return code_upper, None
```

```
# ===== API: PROGRAMS =====
```

```
@login_required
@require_http_methods(["GET"])
def get_programs(request):
    """Get all active programs"""
    programs = Program.objects.filter(is_active=True).order_by('code')
    data = [
        {
            'id': p.id,
            'code': p.code,
            'name': p.name,
            'description': p.description or ""
        }
        for p in programs
    ]
    return JsonResponse({'programs': data}, status=200)
```

```
@login_required
@require_http_methods(["GET"])
def get_all_programs(request):
    """Get all programs including inactive ones"""
    programs = Program.objects.all().order_by('code')
    data = [
        {
            'id': p.id,
            'code': p.code,
            'name': p.name,
            'description': p.description or "",
            'is_active': p.is_active,
            'created_at': p.created_at.strftime('%Y-%m-%d'),
            'sections_count': p.sections.count()
        }
        for p in programs
    ]
    return JsonResponse({'programs': data}, status=200)
```

```
@login_required
@require_http_methods(["POST"])
@transaction.atomic
def add_program(request):
    """Add a new program"""
    try:
        data = json.loads(request.body)
        code = (data.get('code') or "").strip().upper()
```

```

name = (data.get('name') or "").strip()
description = (data.get('description') or "").strip()
is_active = data.get('is_active', True)

if not code:
    return JsonResponse({'error': 'Program code is required'}, status=400)

if not name:
    return JsonResponse({'error': 'Program name is required'}, status=400)

# Check if program already exists
if Program.objects.filter(code__iexact=code).exists():
    return JsonResponse({'error': f'Program "{code}" already exists'}, status=400)

# Create program
program = Program.objects.create(
    code=code,
    name=name,
    description=description,
    is_active=is_active
)

# Log activity
log_activity(
    user=request.user,
    action='program_added',
    description=f'Added program: {program.code}',
    request=request
)

return JsonResponse({
    'message': f'Program "{code}" added successfully',
    'program': {
        'id': program.id,
        'code': program.code,
        'name': program.name,
        'description': program.description or "",
        'is_active': program.is_active
    }
}, status=201)

except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

@login_required

```

@require_http_methods(["POST"])
@transaction.atomic
def update_program(request, program_id):
    """Update an existing program"""
    try:
        program = Program.objects.get(pk=program_id)
        data = json.loads(request.body)

        code = (data.get('code') or "").strip().upper()
        name = (data.get('name') or "").strip()
        description = (data.get('description') or "").strip()
        is_active = data.get('is_active', program.is_active)

        if not code:
            return JsonResponse({'error': 'Program code is required'}, status=400)

        if not name:
            return JsonResponse({'error': 'Program name is required'}, status=400)

        # Check if code is changing and if new code already exists
        if code != program.code and Program.objects.filter(code__iexact=code).exists():
            return JsonResponse({'error': f'Program "{code}" already exists'}, status=400)

        # Update program
        program.code = code
        program.name = name
        program.description = description
        program.is_active = is_active
        program.save()

        # Log activity
        log_activity(
            user=request.user,
            action='program_updated',
            description=f'Updated program: {program.code}',
            request=request
        )

    return JsonResponse({
        'message': f'Program "{code}" updated successfully',
        'program': {
            'id': program.id,
            'code': program.code,
            'name': program.name,
            'description': program.description or "",
            'is_active': program.is_active
        }
    })

```

```

    }, status=200)

except Program.DoesNotExist:
    return JsonResponse({'error': 'Program not found'}, status=404)
except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

@login_required
@require_http_methods(["POST"])
@transaction.atomic
def delete_program(request, program_id):
    """Delete a program"""
    try:
        program = Program.objects.get(pk=program_id)

        # Check if program has sections
        sections_count = program.sections.count()
        if sections_count > 0:
            return JsonResponse({
                'error': f'Cannot delete program "{program.code}". It has {sections_count} section(s)
assigned to it.'
            }, status=400)

        program_code = program.code
        program.delete()

        # Log activity
        log_activity(
            user=request.user,
            action='program_deleted',
            description=f'Deleted program: {program_code}',
            request=request
        )

        return JsonResponse({
            'message': f'Program "{program_code}" deleted successfully'
        }, status=200)

except Program.DoesNotExist:
    return JsonResponse({'error': 'Program not found'}, status=404)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

@login_required
@require_http_methods(["POST"])

```

```

@transaction.atomic
def toggle_program_status(request, program_id):
    """Toggle program active status"""
    try:
        program = Program.objects.get(pk=program_id)
        program.is_active = not program.is_active
        program.save()

        status_text = 'activated' if program.is_active else 'deactivated'

        # Log activity
        log_activity(
            user=request.user,
            action='program_updated',
            description=f'{status_text.capitalize()} program: {program.code}',
            request=request
        )

        return JsonResponse({
            'message': f'Program "{program.code}" {status_text} successfully',
            'is_active': program.is_active
        }, status=200)

    except Program.DoesNotExist:
        return JsonResponse({'error': 'Program not found'}, status=404)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

# ===== API: TEACHERS =====

@login_required
@require_http_methods(["GET"])
def get_teachers(request):
    """Get all teachers with adviser status scoped to the active school year"""
    # Get active school year
    active_school_year = SchoolYear.objects.filter(is_active=True).first()

    # Get teacher IDs that are advisers in the CURRENT school year only
    if active_school_year:
        active_adviser_ids = set(
            Section.objects.filter(
                school_year=active_school_year,
                adviser__isnull=False
            ).values_list('adviser_id', flat=True)
        )
    else:
        active_adviser_ids = set()

```

```

teachers = Teacher.objects.all().order_by('last_name', 'first_name')
data = []
for t in teachers:
    is_adviser_this_year = t.id in active_adviser_ids
    # Find their current section in this school year if any
    advisory_section_id = None
    if is_adviser_this_year and active_school_year:
        section = Section.objects.filter(
            school_year=active_school_year,
            adviser=t
        ).first()
        advisory_section_id = section.id if section else None

    data.append({
        'id': t.id,
        'name': t.get_full_name(),
        'is_adviser': is_adviser_this_year,
        'advisory_section_id': advisory_section_id,
        'position': t.position.name if t.position else "",
        'department': t.department.name if t.department else ""
    })
return JsonResponse({'teachers': data}, status=200)

```

===== API: BUILDINGS & ROOMS =====

```

@login_required
@require_http_methods(["GET"])
def get_buildings(request):
    """Get all buildings with their rooms"""
    buildings = Building.objects.prefetch_related('rooms').all().order_by('name')
    data = []
    for building in buildings:
        rooms = [
            {
                'id': room.id,
                'room_number': room.room_number
            }
            for room in building.rooms.all().order_by('room_number')
        ]
        data.append({
            'id': building.id,
            'name': building.name,
            'rooms': rooms
        })
    return JsonResponse({'buildings': data}, status=200)

```



```

@login_required
@require_http_methods(["GET"])
def get_rooms(request):
    """Get rooms for a specific building"""
    building_id = request.GET.get('building')
    if not building_id:
        return JsonResponse({'error': 'Building ID is required'}, status=400)

    try:
        building = Building.objects.get(id=building_id)
        rooms = building.rooms.all().order_by('room_number')
        data = [
            {
                'id': room.id,
                'room_number': room.room_number,
                'building_id': room.building.id,
                'building_name': room.building.name
            }
            for room in rooms
        ]
        return JsonResponse({'rooms': data}, status=200)
    except Building.DoesNotExist:
        return JsonResponse({'error': 'Building not found'}, status=404)

```

===== API: SUBJECTS =====

```

@login_required
@require_http_methods(["GET"])
def get_subjects(request):
    """Get subjects, optionally filtered by program"""
    program_code = request.GET.get('program')
    qs = Subject.objects.filter(is_active=True)
    if program_code:
        program = _get_program_by_code(program_code)
        if not program:
            return JsonResponse({'error': 'Program not found'}, status=404)
        qs = qs.filter(program=program)
    qs = qs.select_related('program').order_by('program__code', 'name')
    subjects = [
        {
            'id': s.id,
            'name': s.name,
            'code': s.code,
            'description': s.description or "",
            'program_code': s.program.code,
            'program_name': s.program.name,
        }
    ]

```

```

        for s in qs
    ]
    return JsonResponse({'subjects': subjects}, status=200)

@login_required
@require_http_methods(["POST"])
def add_subject(request):
    """Add a new subject"""
    try:
        data = json.loads(request.body)
        name = (data.get('name') or "").strip()
        code = (data.get('code') or "").strip().upper()
        description = data.get('description') or ""
        program_code = data.get('program')

        if not program_code:
            return JsonResponse({'error': 'Program is required'}, status=400)

        program = _get_program_by_code(program_code)
        if not program:
            return JsonResponse({'error': 'Program not found'}, status=404)

        if not name:
            return JsonResponse({'error': 'Subject name is required'}, status=400)
        if not code:
            return JsonResponse({'error': 'Subject code is required'}, status=400)

        # Check if subject code already exists for this program
        if Subject.objects.filter(program=program, code=code).exists():
            return JsonResponse({'error': f'Subject code {code} already exists for this program'},
                                status=400)

        subject = Subject.objects.create(
            program=program,
            name=name,
            code=code,
            description=description
        )

        # Log activity
        log_activity(
            user=request.user,
            action='section_management',
            description=f'Added subject: {code} - {name} for {program.code}',
            request=request
        )

```

```

return JsonResponse({
    'message': 'Subject added successfully',
    'subject': {
        'id': subject.id,
        'name': subject.name,
        'code': subject.code,
        'description': subject.description or "",
        'program_code': subject.program.code,
        'program_name': subject.program.name,
    }
}, status=201)
except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

@login_required
@require_http_methods(["PUT"])
def update_subject(request, subject_id):
    """Update an existing subject"""
    try:
        subject = Subject.objects.get(pk=subject_id)
        data = json.loads(request.body)
        name = (data.get('name') or "").strip()
        code = (data.get('code') or "").strip().upper()
        description = data.get('description') or ""

        if not name:
            return JsonResponse({'error': 'Subject name is required'}, status=400)
        if not code:
            return JsonResponse({'error': 'Subject code is required'}, status=400)

        # Check if new code conflicts with another subject in same program
        if Subject.objects.filter(program=subject.program,
            code=code).exclude(pk=subject_id).exists():
            return JsonResponse({'error': f'Subject code {code} already exists for this program'},
                status=400)

        old_code = subject.code
        old_name = subject.name
        subject.name = name
        subject.code = code
        subject.description = description
        subject.save()

        # Log activity
        log_activity(

```

```

        user=request.user,
        action='section_management',
        description=f'Updated subject: {old_code} -> {code} ({old_name} -> {name}) for
{subject.program.code}',
        request=request
    )

    return JsonResponse({
        'message': 'Subject updated successfully',
        'subject': {
            'id': subject.id,
            'name': subject.name,
            'code': subject.code,
            'description': subject.description or "",
            'program_code': subject.program.code,
            'program_name': subject.program.name,
        }
    }, status=200)
except Subject.DoesNotExist:
    return JsonResponse({'error': 'Subject not found'}, status=404)
except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

@login_required
@require_http_methods(["DELETE"])
def delete_subject(request, subject_id):
    """Delete a subject"""
    try:
        subject = Subject.objects.get(pk=subject_id)
        subject_name = subject.name
        subject_code = subject.code
        program_code = subject.program.code
        subject.delete()

        # Log activity
        log_activity(
            user=request.user,
            action='section_management',
            description=f'Deleted subject: {subject_code} - {subject_name} from {program_code}',
            request=request
        )

        return JsonResponse({'message': f'Subject "{subject_name}" deleted successfully'},
            status=200)
    except Subject.DoesNotExist:

```

```

        return JsonResponse({'error': 'Subject not found'}, status=404)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

===== API: SECTIONS =====

```

@login_required
@require_http_methods(["GET"])
def get_sections(request):
    """Get sections, optionally filtered by program, for active school year"""
    program_code = request.GET.get('program')

    # Get active school year
    active_school_year = SchoolYear.objects.filter(is_active=True).first()
    if not active_school_year:
        return JsonResponse({'sections': [], 'warning': 'No active school year found'}, status=200)

    # Filter sections by active school year
    qs = Section.objects.select_related('program', 'adviser', 'school_year', 'grade_level').filter(
        school_year=active_school_year
    ).order_by('program__code', 'grade_level__code', 'name')

    # Filter by grade level if provided
    grade_level_id = request.GET.get('grade_level_id')
    if grade_level_id:
        qs = qs.filter(grade_level_id=grade_level_id)

    if program_code:
        # Normalize program code (TOP5/HETERO -> REGULAR with track filter)
        normalized_code, regular_track = _normalize_program_code(program_code)
        program = _get_program_by_code(normalized_code)
        if not program:
            return JsonResponse({'error': 'Program not found'}, status=404)
        qs = qs.filter(program=program)

    # If filtering by regular track, add track filter
    if regular_track:
        qs = qs.filter(regular_track=regular_track)

    sections = []
    for s in qs:
        sections.append({
            'id': s.id,
            'name': s.name,
            'program_code': s.program.code,
            'program_name': s.program.name,
            'regular_track': s.regular_track or "",

```

```

'school_year': s.school_year.year_label if s.school_year else 'N/A',
'adviser_id': s.adviser.id if s.adviser else None,
'adviser_name': s.adviser.get_full_name() if s.adviser else "",
'building': s.building or "",
'room': s.room or "",
'max_students': s.max_students,
'current_students': s.current_students,
'grade_level_id': s.grade_level.id if s.grade_level else None,
'grade_level_name': s.grade_level.name if s.grade_level else "",
'location': f"Bldg {s.building} Room {s.room}".strip() if s.building or s.room else "",
})
return JsonResponse({'sections': sections}, status=200)

```

```

@login_required
@require_http_methods(["POST"])
@transaction.atomic
def add_section(request):
    """Add a new section"""
    try:
        data = json.loads(request.body)
        name = (data.get('name') or "").strip()
        program_code = data.get('program')
        adviser_id = data.get('adviser')
        building_id = data.get('building')
        room_id = data.get('room')
        max_students = int(data.get('max_students') or 40)
        grade_level_id = data.get('grade_level_id')

        # Validate required fields
        if not name:
            return JsonResponse({'error': 'Section name is required'}, status=400)

        if not program_code:
            return JsonResponse({'error': 'Program is required'}, status=400)

        # Normalize program code (TOP5/HETERO -> REGULAR)
        normalized_code, regular_track = _normalize_program_code(program_code)

        # Get program
        program = _get_program_by_code(normalized_code)
        if not program:
            return JsonResponse({'error': 'Program not found'}, status=404)

        # Get active school year
        active_school_year = SchoolYear.objects.filter(is_active=True).first()
        if not active_school_year:

```

```
        return JsonResponse({'error': 'No active school year found. Please activate a school year first.'}, status=400)
```

```
    # Get grade level (optional)
```

```
    grade_level = None
```

```
    if grade_level_id:
```

```
        try:
```

```
            grade_level = GradeLevel.objects.get(pk=grade_level_id)
```

```
        except GradeLevel.DoesNotExist:
```

```
            return JsonResponse({'error': 'Selected grade level not found'}, status=404)
```

```
    # Check if section name already exists for this program and school year
```

```
    if Section.objects.filter(program=program, name=name,  
school_year=active_school_year).exists():
```

```
        return JsonResponse({'error': f'Section "{name}" already exists in {program.code}  
program for school year {active_school_year.year_label}'}, status=400)
```

```
    # Handle adviser
```

```
    adviser = None
```

```
    if adviser_id:
```

```
        try:
```

```
            adviser = Teacher.objects.get(pk=adviser_id)
```

```
            # Check if teacher is already an adviser in the CURRENT school year
```

```
            if active_school_year and Section.objects.filter(  
                school_year=active_school_year,  
                adviser=adviser
```

```
            ).exists():
```

```
                return JsonResponse({'error': f'{adviser.get_full_name()} is already assigned as an  
adviser in the current school year'}, status=400)
```

```
            except Teacher.DoesNotExist:
```

```
                return JsonResponse({'error': 'Selected adviser not found'}, status=404)
```

```
    # Handle building and room
```

```
    building_name = ""
```

```
    room_number = ""
```

```
    if building_id and room_id:
```

```
        try:
```

```
            building = Building.objects.get(pk=building_id)
```

```
            room = Room.objects.get(pk=room_id, building=building)
```

```
            building_name = building.name
```

```
            room_number = room.room_number
```

```
        except Building.DoesNotExist:
```

```
            return JsonResponse({'error': 'Selected building not found'}, status=404)
```

```
        except Room.DoesNotExist:
```

```
            return JsonResponse({'error': 'Selected room not found or does not belong to the  
building'}, status=404)
```

```

# Create section with active school year
section = Section.objects.create(
    program=program,
    name=name,
    regular_track=regular_track,
    school_year=active_school_year,
    adviser=adviser,
    building=building_name,
    room=room_number,
    max_students=max_students,
    grade_level=grade_level,
)

# Update adviser flag
if adviser:
    adviser.is_adviser = True
    adviser.save(update_fields=['is_adviser'])

# Log activity
location = f"Bldg {building_name} Room {room_number}" if building_name and
room_number else "No location"
adviser_info = f" with adviser {adviser.get_full_name()}" if adviser else ""
track_info = f" ({regular_track} track)" if regular_track else ""
log_activity(
    user=request.user,
    action='section_management',
    description=f"Created section: {program.code}{track_info} - {name} at
{location}{adviser_info}",
    request=request
)

return JsonResponse({
    'message': f"Section \"{name}\" created successfully for {active_school_year.year_label}",
    'section': {
        'id': section.id,
        'name': section.name,
        'program_code': section.program.code,
        'program_name': section.program.name,
        'regular_track': section.regular_track or "",
        'school_year': active_school_year.year_label,
        'adviser_id': adviser.id if adviser else None,
        'adviser_name': adviser.get_full_name() if adviser else "",
        'building': section.building or "",
        'room': section.room or "",
        'max_students': section.max_students,
        'current_students': section.current_students,
        'grade_level_id': section.grade_level.id if section.grade_level else None,
    }
})

```



```

        'grade_level_name': section.grade_level.name if section.grade_level else ",
    }
}, status=201)
except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)
except ValueError as e:
    return JsonResponse({'error': f'Invalid data format: {str(e)}'}, status=400)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

@login_required
@require_http_methods(["PUT"])
@transaction.atomic
def update_section(request, section_id):
    """Update an existing section"""
    try:
        section = Section.objects.select_related('adviser', 'program').get(pk=section_id)
        data = json.loads(request.body)

        name = (data.get('name') or "").strip()
        adviser_id = data.get('adviser')
        building_id = data.get('building')
        room_id = data.get('room')
        max_students = int(data.get('max_students') or section.max_students)
        grade_level_id = data.get('grade_level_id')

        # Validate required fields
        if not name:
            return JsonResponse({'error': 'Section name is required'}, status=400)

        # Check if new name conflicts with another section in same program
        if Section.objects.filter(program=section.program,
name=name).exclude(pk=section_id).exists():
            return JsonResponse({'error': f'Section "{name}" already exists in {section.program.code}
program'}, status=400)

        # Get grade level (optional)
        grade_level = section.grade_level # keep existing by default
        if 'grade_level_id' in data:
            if grade_level_id:
                try:
                    grade_level = GradeLevel.objects.get(pk=grade_level_id)
                except GradeLevel.DoesNotExist:
                    return JsonResponse({'error': 'Selected grade level not found'}, status=404)
            else:
                grade_level = None

```

```

# Handle adviser change
old_adviser = section.adviser
new_adviser = None

if adviser_id:
    try:
        new_adviser = Teacher.objects.get(pk=adviser_id)
        # Check if new adviser is already assigned in the current school year (excluding
current section)
        active_school_year = SchoolYear.objects.filter(is_active=True).first()
        if new_adviser != old_adviser and active_school_year and Section.objects.filter(
            school_year=active_school_year,
            adviser=new_adviser
        ).exclude(pk=section_id).exists():
            return JsonResponse({'error': f'{new_adviser.get_full_name()} is already assigned
as an adviser in the current school year'}, status=400)
        except Teacher.DoesNotExist:
            return JsonResponse({'error': 'Selected adviser not found'}, status=404)

# Handle building and room
building_name = ""
room_number = ""
if building_id and room_id:
    try:
        building = Building.objects.get(pk=building_id)
        room = Room.objects.get(pk=room_id, building=building)
        building_name = building.name
        room_number = room.room_number
    except Building.DoesNotExist:
        return JsonResponse({'error': 'Selected building not found'}, status=404)
    except Room.DoesNotExist:
        return JsonResponse({'error': 'Selected room not found or does not belong to the
building'}, status=404)

# Update section
section.name = name
section.adviser = new_adviser
section.building = building_name
section.room = room_number
section.max_students = max_students
section.grade_level = grade_level
section.save()

# Update adviser flags
if old_adviser and old_adviser != new_adviser:
    old_adviser.is_adviser = False
    old_adviser.save(update_fields=['is_adviser'])

```

```

if new_adviser and new_adviser != old_adviser:
    new_adviser.is_adviser = True
    new_adviser.save(update_fields=['is_adviser'])

# Log activity
changes = []
if section.name != name:
    changes.append(f"name: {section.name} -> {name}")
if old_adviser != new_adviser:
    old_adv = old_adviser.get_full_name() if old_adviser else "None"
    new_adv = new_adviser.get_full_name() if new_adviser else "None"
    changes.append(f"adviser: {old_adv} -> {new_adv}")
location = f"Bldg {building_name} Room {room_number}" if building_name and
room_number else "No location"
change_desc = ", ".join(changes) if changes else "details"
log_activity(
    user=request.user,
    action='section_management',
    description=f'Updated section: {section.program.code} - {name} ({change_desc}) at
{location}',
    request=request
)

return JsonResponse({
    'message': f'Section "{name}" updated successfully',
    'section': {
        'id': section.id,
        'name': section.name,
        'program_code': section.program.code,
        'program_name': section.program.name,
        'adviser_id': section.adviser.id if section.adviser else None,
        'adviser_name': section.adviser.get_full_name() if section.adviser else "",
        'building': section.building or "",
        'room': section.room or "",
        'max_students': section.max_students,
        'current_students': section.current_students,
        'grade_level_id': section.grade_level.id if section.grade_level else None,
        'grade_level_name': section.grade_level.name if section.grade_level else "",
    }
}, status=200)
except Section.DoesNotExist:
    return JsonResponse({'error': 'Section not found'}, status=404)
except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)
except ValueError as e:
    return JsonResponse({'error': f'Invalid data format: {str(e)}'}, status=400)

```

```

except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

@login_required
@require_http_methods(["DELETE"])
@transaction.atomic
def delete_section(request, section_id):
    """Delete a section"""
    try:
        section = Section.objects.select_related('adviser', 'program').get(pk=section_id)
        section_name = section.name
        program_code = section.program.code
        adviser = section.adviser
        adviser_name = adviser.get_full_name() if adviser else "No adviser"

        # Delete the section
        section.delete()

        # Update adviser flag if exists
        if adviser:
            adviser.is_adviser = False
            adviser.save(update_fields=['is_adviser'])

        # Log activity
        log_activity(
            user=request.user,
            action='section_management',
            description=f'Deleted section: {program_code} - {section_name} (Adviser:
{adviser_name})',
            request=request
        )

        return JsonResponse({'message': f'Section "{section_name}" deleted successfully'},
status=200)
    except Section.DoesNotExist:
        return JsonResponse({'error': 'Section not found'}, status=404)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

@admin_required
def sections_header_data(request):
    """API endpoint for [module] header data"""
    active_school_year = SchoolYear.get_active_school_year()

    try:
        user_profile = UserProfile.objects.select_related('program', 'position',
'department').get(user=request.user)

```

```

except UserProfile.DoesNotExist:
    user_profile = None

user = request.user
full_name = f"{user.first_name} {user.last_name}" if user.first_name and user.last_name else
user.username

if user.first_name and user.last_name:
    initials = f"{user.first_name[0]}{user.last_name[0]}.upper()"
elif user.first_name:
    initials = user.first_name[0].upper()
elif user.last_name:
    initials = user.last_name[0].upper()
else:
    initials = user.username[0].upper() if user.username else "U"

role = user_profile.get_user_type_display() if user_profile else "Admin"
photo_url = user_profile.photo.url if user_profile and user_profile.photo else None

return JsonResponse({
    'school_year': active_school_year.year_label if active_school_year else 'No Active Year',
    'full_name': full_name,
    'role': role,
    'initials': initials,
    'photo_url': photo_url,
})

```

studentdetails_views.py – Individual student view (Student profile and history)

```

from django.shortcuts import render, get_object_or_404
from django.db.models import Case, When, IntegerField
from admin_app.decorators import admin_required
from django.contrib.auth.decorators import login_required
from django.views.decorators.http import require_http_methods
from django.http import JsonResponse
from enrollment_app.models import (
    Student, StudentEnrollment, StudentAcademicYearStatus
)
from coordinator_app.models import AcademicPerformance, ProbationRecord
from admin_app.models import GradeLevel
from decimal import Decimal
from datetime import date
import json

def _age(dob):
    if not dob:
        return None

```

```
today = date.today()
return today.year - dob.year - ((today.month, today.day) < (dob.month, dob.day))
```

```
def _build_grade_data(student):
    performances = (
        AcademicPerformance.objects
        .filter(student=student)
        .select_related('subject', 'grade_level', 'school_year', 'program')
        .order_by('grade_level__code', 'subject__name', 'quarter')
    )
```

```
grade_data = {}
q_field = {1: 'q1', 2: 'q2', 3: 'q3', 4: 'q4', 5: 'final'}
```

```
for perf in performances:
    gl_id = perf.grade_level_id
    if gl_id not in grade_data:
        grade_data[gl_id] = {
            'grade_level': perf.grade_level,
            'school_year': perf.school_year,
            'program_code': perf.program.code if perf.program else "",
            'subjects': {},
        }
    subj_id = perf.subject_id
    if subj_id not in grade_data[gl_id]['subjects']:
        grade_data[gl_id]['subjects'][subj_id] = {
            'subject': perf.subject,
            'q1': None, 'q2': None, 'q3': None, 'q4': None, 'final': None,
        }
    field = q_field.get(perf.quarter)
    if field:
        grade_data[gl_id]['subjects'][subj_id][field] = perf.grade
```

```
result = []
for gl_id, data in sorted(grade_data.items(), key=lambda x: x[1]['grade_level'].code):
    subjects_list = []
    final_ratings = []
```

```
    for subj_data in sorted(data['subjects'].values(), key=lambda x: x['subject'].name):
        q_grades = [subj_data[f'q{i}'] for i in range(1, 5) if subj_data[f'q{i}'] is not None]
```

```
        if subj_data['final'] is not None:
            final_rating = subj_data['final']
        elif q_grades:
            final_rating = round(sum(q_grades) / len(q_grades), 2)
        else:
            final_rating = None
```

```

# STE probation threshold logic
subject_obj = subj_data['subject']
section_program = data.get('program_code', "")
is_ste_threshold = (
    getattr(subject_obj, 'is_threshold_subject', False)
    and section_program == 'STE'
)

if final_rating is not None:
    if is_ste_threshold:
        if final_rating >= 83:
            status = 'Passed'
        elif final_rating >= 75:
            status = 'Probation Risk'
        else:
            status = 'Failed'
    else:
        status = 'Passed' if final_rating >= 75 else 'Failed'
    final_ratings.append(final_rating)
else:
    status = None

# Per-quarter status
quarter_statuses = {}
for q_num in range(1, 5):
    q_grade = subj_data.get(f'q{q_num}')
    if q_grade is not None:
        if is_ste_threshold:
            if q_grade >= 83:
                quarter_statuses[f'q{q_num}_status'] = 'Passed'
            elif q_grade >= 75:
                quarter_statuses[f'q{q_num}_status'] = 'Probation Risk'
            else:
                quarter_statuses[f'q{q_num}_status'] = 'Failed'
        else:
            quarter_statuses[f'q{q_num}_status'] = 'Passed' if q_grade >= 75 else 'Failed'
    else:
        quarter_statuses[f'q{q_num}_status'] = None

subjects_list.append({
    **subj_data,
    'final_rating': final_rating,
    'status': status,
    'is_ste_threshold': is_ste_threshold,
    **quarter_statuses,
})

```

```

if final_ratings:
    general_average = round(sum(final_ratings) / len(final_ratings), 2)
    has_incomplete = any(s['final_rating'] is None for s in subjects_list)
    all_passed = all(
        s['final_rating'] >= 75
        for s in subjects_list if s['final_rating'] is not None
    )
    if has_incomplete:
        promotion_status = 'Incomplete'
    elif all_passed and general_average >= 75:
        promotion_status = 'Promoted'
    else:
        promotion_status = 'Failed'
else:
    general_average = None
    promotion_status = None

def q_avg(q_key):
    grades = [s[q_key] for s in subjects_list if s[q_key] is not None]
    if len(grades) == len(subjects_list) and len(grades) > 0:
        return round(sum(grades) / len(grades), 2)
    return None

q1_avg = q_avg('q1')
q2_avg = q_avg('q2')
q3_avg = q_avg('q3')
q4_avg = q_avg('q4')

result.append({
    'grade_level':    data['grade_level'],
    'school_year':    data['school_year'],
    'subjects_list':  subjects_list,
    'general_average': general_average,
    'promotion_status': promotion_status,
    'q1_available':   q1_avg is not None,
    'q2_available':   q2_avg is not None,
    'q3_available':   q3_avg is not None,
    'q4_available':   q4_avg is not None,
    'final_available': general_average is not None,
    'q1_average':     q1_avg,
    'q2_average':     q2_avg,
    'q3_average':     q3_avg,
    'q4_average':     q4_avg,
})

return result

```



```

def _build_progression_data(student):
    """
    Builds G7 -> G8 -> G9 -> G10 progression timeline using:
    - StudentEnrollment (per-year status, grade level, enrollee type)
    - StudentAcademicYearStatus (final outcome, overall grade, section, adviser)
    - ProgramSelection (current section, program code, track)
    """
    grade_order = Case(
        When(code='G7', then=1),
        When(code='G8', then=2),
        When(code='G9', then=3),
        When(code='G10', then=4),
        default=5,
        output_field=IntegerField()
    )
    all_grade_levels = GradeLevel.objects.filter(
        code__in=['G7', 'G8', 'G9', 'G10']
    ).annotate(sort_order=grade_order).order_by('sort_order')

    enrollments = (
        StudentEnrollment.objects
        .filter(student=student)
        .select_related('school_year', 'grade_level')
        .order_by('school_year__year_label')
    )
    enrollment_map = {e.grade_level_id: e for e in enrollments if e.grade_level_id}

    statuses = (
        StudentAcademicYearStatus.objects
        .filter(student=student)
        .select_related('school_year', 'grade_level', 'section', 'section__adviser', 'recorded_by')
    )
    status_map = {s.grade_level_id: s for s in statuses if s.grade_level_id}

    try:
        ps = student.program_selection
    except Exception:
        ps = None

    latest_enrollment = enrollments.last()
    current_gl_id = latest_enrollment.grade_level_id if latest_enrollment else None

    progression = []
    for gl in all_grade_levels:
        enrollment = enrollment_map.get(gl.id)
        acad_status = status_map.get(gl.id)

```

```
is_current = (gl.id == current_gl_id)
```

```
section_name = None  
adviser_name = None  
program_code = None  
regular_track = None
```

```
# Pull from academic year status (historical)  
if acad_status and acad_status.section:  
    section_name = acad_status.section.name  
    if acad_status.section.adviser:  
        adviser_name = acad_status.section.adviser.get_full_name()
```

```
# For the current grade, also pull from ProgramSelection  
if is_current and ps:  
    if ps.assigned_section:  
        section_name = section_name or ps.assigned_section.name  
        if not adviser_name and ps.assigned_section.adviser:  
            adviser_name = ps.assigned_section.adviser.get_full_name()  
        program_code = ps.selected_program_code  
        regular_track = ps.regular_track
```

```
progression.append({  
    'grade_level':    gl,  
    'school_year':    enrollment.school_year if enrollment else None,  
    'enrollment_status': enrollment.enrollment_status if enrollment else None,  
    'enrollee_type':  enrollment.enrollee_type if enrollment else None,  
    'section_name':   section_name,  
    'program_code':   program_code,  
    'regular_track':  regular_track,  
    'adviser_name':   adviser_name,  
    'overall_grade':  acad_status.overall_grade if acad_status else None,  
    'final_status':   acad_status.final_status if acad_status else None,  
    'remarks':        acad_status.remarks if acad_status else None,  
    'has_data':       enrollment is not None,  
    'is_current':     is_current,  
})
```

```
return progression
```

```
@admin_required
```

```
def student_details(request, lrn):  
    student = get_object_or_404(  
        Student.objects.select_related(  
            'student_data',  
  
            'family_data',
```

```

        'family_data__father',
        'family_data__mother',
        'family_data__other_guardian',
        'program_selection',
        'program_selection__assigned_section',
        'program_selection__assigned_section__program',
        'program_selection__assigned_section__grade_level',
        'program_selection__assigned_section__adviser',
    ),
    lrn=lrn
)

sd = getattr(student, 'student_data', None)
fd = getattr(student, 'family_data', None)
ps = getattr(student, 'program_selection', None)

student_age = _age(sd.date_of_birth) if sd and getattr(sd, 'date_of_birth', None) else None

enroll_list = []
if sd and getattr(sd, 'enrolling_as', None):
    raw = sd.enrolling_as
    if isinstance(raw, list):
        enroll_list = [e.replace('_', ' ').title() for e in raw]
    elif isinstance(raw, str):
        enroll_list = [raw.replace('_', ' ').title()]

# Get latest enrollment for school year / status display
latest_enrollment = (
    StudentEnrollment.objects
    .filter(student=student)
    .select_related('school_year', 'grade_level')
    .order_by('-school_year__year_label')
    .first()
)

back_url = request.GET.get('back', "")

# Get all probation records for this student
probation_records = ProbationRecord.objects.filter(
    student=student
).select_related('school_year', 'grade_level', 'reinstated_by').order_by('-flagged_at')

probation_map = {}
for rec in probation_records:
    if rec.grade_level_id:
        probation_map[rec.grade_level_id] = rec

```

```

context = {
    'student':      student,
    'sd':           sd,
    'fd':           fd,
    'ps':           ps,
    'student_age':  student_age,
    'enroll_str':   ', '.join(enroll_list) if enroll_list else 'N/A',
    'sy_label':     latest_enrollment.school_year.year_label if latest_enrollment and
latest_enrollment.school_year else 'N/A',
    'enroll_status': latest_enrollment.enrollment_status if latest_enrollment else 'N/A',
    'enrollee_type': latest_enrollment.get_enrollee_type_display() if latest_enrollment else
'N/A',
    'latest_enrollment': latest_enrollment,
    'back_url':     back_url,
    'grade_levels_data': _build_grade_data(student),
    'progression':   _build_progression_data(student),
    'probation_records': probation_records,
    'probation_map': probation_map,
}

```

```

if request.GET.get('partial') == '1':
    return render(request, 'admin_app/studentDetails_partial.html', context)

```

```

return render(request, 'admin_app/studentDetails.html', context)

```

```

@login_required
@require_http_methods(["POST"])
def lift_probation(request, lrn):
    """Lift an active probation record for a student."""
    try:
        data = json.loads(request.body)
        reason = (data.get('reason') or "").strip()

        if not reason:
            return JsonResponse({'success': False, 'error': 'Reason is required.'}, status=400)

        student = get_object_or_404(Student, lrn=lrn)

        probation = ProbationRecord.objects.filter(
            student=student,
            is_active=True,
        ).order_by('-flagged_at').first()

        if not probation:
            return JsonResponse({
                'success': False,

```

```
        'error': 'No active probation record found for this student.'
    }, status=404)
```

```
probation.reinstate(reinstated_by_user=request.user, reason=reason)
```

```
lifted_by = request.user.get_full_name() or request.user.username
```

```
return JsonResponse({
    'success': True,
    'message': f'Probation successfully lifted by {lifted_by}.',
    'lifted_by': lifted_by,
    'lifted_at': probation.reinstated_at.strftime('%B %d, %Y at %I:%M %p'),
    'reason': reason,
})
```

```
except json.JSONDecodeError:
    return JsonResponse({'success': False, 'error': 'Invalid JSON.'}, status=400)
except Exception as e:
    return JsonResponse({'success': False, 'error': str(e)}, status=500)
```

studentedit_views.py – Student information editing (Academic and personal data updates)

```
from django.shortcuts import render, get_object_or_404, redirect
from django.contrib.auth.decorators import login_required
from django.http import JsonResponse
from django.views.decorators.http import require_http_methods
from django.db import transaction
from django.core.files.storage import default_storage
from decimal import Decimal
import json
```

```
from enrollment_app.models import (
    Student, StudentData, FamilyData, Parent, Guardian,
    SurveyData, AcademicData, ProgramSelection
)
from admin_app.models import Program, SchoolYear
from admin_app.models import Section
```

```
@login_required
def student_edit(request, student_id):
    """Main view for student edit page"""
    student = get_object_or_404(Student, ln=student_id)

    # Get all school years for the filter
    school_years = SchoolYear.objects.all().order_by('-year_label')
    active_school_year = SchoolYear.objects.filter(is_active=True).first()

    # Get all programs for selection
```

```

programs = Program.objects.all()

# Admin portal is always read-only
is_readonly = True

context = {
    'student': student,
    'student_id': student_id,
    'school_years': school_years,
    'active_school_year': active_school_year,
    'programs': programs,
    'is_readonly': is_readonly,
}

return render(request, 'admin_app/studentEdit.html', context)

@login_required
@require_http_methods(["GET"])
def get_student_details(request, student_id):
    """API endpoint to fetch all student details"""
    try:
        student = get_object_or_404(Student, lrn=student_id)

    def fmt_date(d):
        """Safely format a date or return empty string."""
        try:
            return d.strftime('%Y-%m-%d') if d else ""
        except Exception:
            return ""

    def fmt_date(d):
        try:
            return d.strftime('%Y-%m-%d') if d else ""
        except Exception:
            return ""

    data = {
        'student': {
            'lrn': student.lrn,
            'email': getattr(student, 'email', "") or "",
            'enrollment_status': getattr(student, 'enrollment_status', ""),
            'school_year': student.school_year.year_label if getattr(student, 'school_year', None)
        }
    }
    else None,
        'is_lis_verified': getattr(student, 'is_lis_verified', False),
        'created_at': fmt_date(getattr(student, 'created_at', None)),
    }
}

```

— Student Data —

try:

```
sd = student.student_data
data['student_data'] = {
    'last_name':      sd.last_name or "",
    'first_name':     sd.first_name or "",
    'middle_name':    sd.middle_name or "",
    'gender':         sd.gender or "",
    'date_of_birth':  fmt_date(sd.date_of_birth),
    'place_of_birth': sd.place_of_birth or "",
    'religion':       sd.religion or "",
    'dialect_spoken': sd.dialect_spoken or "",
    'ethnic_tribe':   sd.ethnic_tribe or "",
    'address':        sd.address or "",
    'enrolling_as':   sd.enrolling_as or "",
    'is_sped':        bool(sd.is_sped),
    'sped_details':   sd.sped_details or "",
    'is_working_student': bool(sd.is_working_student),
    'working_details': sd.working_details or "",
    'last_school_attended': sd.last_school_attended or "",
    'previous_grade_section': sd.previous_grade_section or "",
    'last_school_year': sd.last_school_year or "",
    'student_photo':  sd.student_photo.url if sd.student_photo else None,
    'age':            sd.age if hasattr(sd, 'age') else None,
    'full_name':      getattr(sd, 'full_name', ""),
}
```

except Exception:

```
data['student_data'] = None
```

— Family Data —

try:

```
fd = student.family_data
```

```
def fmt_parent(p):
```

```
    if not p:
```

```
        return None
```

```
    return {
```

```
        'id':      p.id,
```

```
        'family_name': p.family_name or "",
```

```
        'first_name':  p.first_name or "",
```

```
        'middle_name': p.middle_name or "",
```

```
        'date_of_birth': fmt_date(p.date_of_birth),
```

```
        'occupation':   p.occupation or "",
```

```
        'address':      p.address or "",
```

```
        'contact_number': p.contact_number or "",
```

```
        'email':        p.email or "",
```

```

        'age':          p.age if hasattr(p, 'age') else None,
        'full_name':    getattr(p, 'full_name', ""),
    }

    data['father'] = fmt_parent(fd.father)
    data['mother'] = fmt_parent(fd.mother)

    data['guardian'] = {
        'official_guardian_type': fd.official_guardian_type or "",
        'parent_photo': fd.parent_photo.url if fd.parent_photo else None,
        'other_guardian': None,
    }

    if fd.other_guardian:
        g = fd.other_guardian
        data['guardian']['other_guardian'] = {
            'id':          g.id,
            'family_name': g.family_name or "",
            'first_name':  g.first_name or "",
            'middle_name': g.middle_name or "",
            'date_of_birth': fmt_date(g.date_of_birth),
            'occupation':   g.occupation or "",
            'address':      g.address or "",
            'contact_number': g.contact_number or "",
            'email':        g.email or "",
            'relationship_to_student': g.relationship_to_student or "",
            'age':          g.age if hasattr(g, 'age') else None,
            'full_name':    getattr(g, 'full_name', ""),
        }
    except Exception:
        data['father'] = None
        data['mother'] = None
        data['guardian'] = None

```

— Survey Data —

```

try:
    survey = student.survey_data
    data['survey_data'] = {
        'student_name':    survey.student_name or "",
        'age':             survey.age,
        'current_grade_section': survey.current_grade_section or "",
        'residence_barangay': survey.residence_barangay or "",
        'gender':          survey.gender or "",
        'learning_style':   survey.learning_style or "",
        'study_hours':      survey.study_hours or "",
        'study_environment': survey.study_environment or "",
        'schoolwork_support': survey.schoolwork_support or "",
    }

```



```

        'enjoyed_subjects':    survey.enjoyed_subjects or "",
        'interested_program': survey.interested_program or "",
        'program_motivation': survey.program_motivation or "",
        'enjoyed_activities': survey.enjoyed_activities or "",
        'enjoyed_activities_other': survey.enjoyed_activities_other or "",
        'assignments_on_time': survey.assignments_on_time or "",
        'handle_difficult_lessons': survey.handle_difficult_lessons or "",
        'device_availability': survey.device_availability or "",
        'internet_access':    survey.internet_access or "",
        'absences':           survey.absences or "",
        'absence_reason':     survey.absence_reason or "",
        'participation':      survey.participation or "",
        'difficulty_areas':   survey.difficulty_areas or "",
        'extra_support':      survey.extra_support or "",
        'quiet_place':        survey.quiet_place or "",
        'distance_from_school': survey.distance_from_school or "",
        'travel_difficulty':   survey.travel_difficulty or "",
    }
except Exception:
    data['survey_data'] = None

# — Academic Data —
try:
    acad = student.academic_data

    def safe_float(v):
        try:
            return float(v) if v is not None else None
        except Exception:
            return None

    data['academic_data'] = {
        'dost_exam_result':    acad.dost_exam_result or "",
        'mathematics':          safe_float(acad.mathematics),
        'araling_panlipunan':    safe_float(acad.araling_panlipunan),
        'english':              safe_float(acad.english),
        'edukasyon_sa_pagpapakatao': safe_float(acad.edukasyon_sa_pagpapakatao),
        'science':              safe_float(acad.science),
        'edukasyon_pangkabuhayan': safe_float(acad.edukasyon_pangkabuhayan),
        'filipino':             safe_float(acad.filipino),
        'mapeh':                safe_float(acad.mapeh),
        'report_card':          acad.report_card.url if acad.report_card else None,
        'is_working_student':    bool(acad.is_working_student),
        'working_type':          acad.working_type or "",
        'is_pwd':               bool(acad.is_pwd),
        'disability_type':       acad.disability_type or "",
        'overall_average':       safe_float(acad.overall_average) or 0,
    }

```

```

    }
except Exception:
    data['academic_data'] = None

# — Program Selection —
try:
    prog = student.program_selection

    # assigned_section can be a FK (Section object) or a plain string
    # depending on which app's model is in use — handle both safely
    assigned_section_name = ""
    try:
        sec = prog.assigned_section
        if sec is None:
            assigned_section_name = ""
        elif hasattr(sec, 'name'):
            # It's a Section FK object
            assigned_section_name = sec.name
        else:
            # It's a plain string
            assigned_section_name = str(sec)
    except Exception:
        assigned_section_name = ""

    data['program_selection'] = {
        'selected_program_code': prog.selected_program_code or "",
        'program_description': getattr(prog, 'program_description', "") or "",
        'selection_reason': prog.selection_reason or "",
        'admin_approved': bool(prog.admin_approved),
        'admin_notes': prog.admin_notes or "",
        'approved_by': prog.approved_by or "",
        'assigned_section': assigned_section_name,
        'enrollee_type': getattr(prog, 'enrollee_type', "")
                        or getattr(student, 'enrollee_type', "") or "",
    }
except Exception:
    data['program_selection'] = None

return JsonResponse({'success': True, 'data': data})

except Exception as e:
    import traceback
    traceback.print_exc()
    return JsonResponse({'success': False, 'error': str(e)}, status=500)

@login_required
@require_http_methods(["POST"])

```

```

def update_student_data(request, student_id):
    """API endpoint to update student information"""
    try:
        with transaction.atomic():
            student = get_object_or_404(Student, lrn=student_id)
            data = json.loads(request.body)

            # Update Student basic info
            if 'email' in data:
                student.email = data['email']
                student.save()

            # Update StudentData
            if 'student_data' in data:
                sd_data = data['student_data']
                student_data, created = StudentData.objects.get_or_create(student=student)

                # Update fields
                for field in ['last_name', 'first_name', 'middle_name', 'gender',
                              'date_of_birth', 'place_of_birth', 'religion',
                              'dialect_spoken', 'ethnic_tribe', 'address',
                              'last_school_attended', 'previous_grade_section',
                              'last_school_year', 'sped_details', 'working_details']:
                    if field in sd_data:
                        setattr(student_data, field, sd_data[field])

                # Boolean fields
                if 'is_sped' in sd_data:
                    student_data.is_sped = sd_data['is_sped']
                if 'is_working_student' in sd_data:
                    student_data.is_working_student = sd_data['is_working_student']

                # JSON fields
                if 'enrolling_as' in sd_data:
                    student_data.enrolling_as = sd_data['enrolling_as']

                student_data.save()

            return JsonResponse({'success': True, 'message': 'Student data updated successfully'})

    except Exception as e:
        return JsonResponse({'success': False, 'error': str(e)}, status=500)

@login_required
@require_http_methods(["POST"])
def update_family_data(request, student_id):
    """API endpoint to update family/guardian information"""

```

```

try:
    with transaction.atomic():
        student = get_object_or_404(Student, lrn=student_id)
        data = json.loads(request.body)

        family_data, created = FamilyData.objects.get_or_create(student=student)

        # Update Father
        if 'father' in data:
            father_data = data['father']
            if family_data.father:
                father = family_data.father
            else:
                father = Parent(parent_type='father')

            for field in ['family_name', 'first_name', 'middle_name',
                          'date_of_birth', 'occupation', 'address',
                          'contact_number', 'email']:
                if field in father_data:
                    setattr(father, field, father_data[field])

            father.save()
            family_data.father = father

        # Update Mother
        if 'mother' in data:
            mother_data = data['mother']
            if family_data.mother:
                mother = family_data.mother
            else:
                mother = Parent(parent_type='mother')

            for field in ['family_name', 'first_name', 'middle_name',
                          'date_of_birth', 'occupation', 'address',
                          'contact_number', 'email']:
                if field in mother_data:
                    setattr(mother, field, mother_data[field])

            mother.save()
            family_data.mother = mother

        # Update Guardian info
        if 'official_guardian_type' in data:
            family_data.official_guardian_type = data['official_guardian_type']

        if 'other_guardian' in data and data['other_guardian']:
            guardian_data = data['other_guardian']

```

```

        if family_data.other_guardian:
            guardian = family_data.other_guardian
        else:
            guardian = Guardian()

        for field in ['family_name', 'first_name', 'middle_name',
                     'date_of_birth', 'occupation', 'address',
                     'contact_number', 'email', 'relationship_to_student']:
            if field in guardian_data:
                setattr(guardian, field, guardian_data[field])

        guardian.save()
        family_data.other_guardian = guardian

    family_data.save()

    return JsonResponse({'success': True, 'message': 'Family data updated successfully'})

except Exception as e:
    return JsonResponse({'success': False, 'error': str(e)}, status=500)

@login_required
@require_http_methods(["POST"])
def update_survey_data(request, student_id):
    """API endpoint to update survey/non-academic profile"""
    try:
        student = get_object_or_404(Student, lrn=student_id)
        data = json.loads(request.body)

        survey_data, created = SurveyData.objects.get_or_create(student=student)

        # Update all survey fields
        for field in ['student_name', 'age', 'current_grade_section',
                     'residence_barangay', 'gender', 'learning_style',
                     'study_hours', 'study_environment', 'schoolwork_support',
                     'interested_program', 'program_motivation',
                     'enjoyed_activities_other', 'assignments_on_time',
                     'handle_difficult_lessons', 'device_availability',
                     'internet_access', 'absences', 'absence_reason',
                     'participation', 'extra_support', 'quiet_place',
                     'distance_from_school', 'travel_difficulty']:
            if field in data:
                setattr(survey_data, field, data[field])

        # JSON array fields
        for field in ['enjoyed_subjects', 'enjoyed_activities', 'difficulty_areas']:
            if field in data:

```

```

        setattr(survey_data, field, data[field])

survey_data.save()

return JsonResponse({'success': True, 'message': 'Survey data updated successfully'})

except Exception as e:
    return JsonResponse({'success': False, 'error': str(e)}, status=500)

@login_required
@require_http_methods(["POST"])
def update_academic_data(request, student_id):
    """API endpoint to update academic information and grades"""
    try:
        student = get_object_or_404(Student, ln=student_id)
        data = json.loads(request.body)

        academic_data, created = AcademicData.objects.get_or_create(student=student)

        # Update DOST result
        if 'dost_exam_result' in data:
            academic_data.dost_exam_result = data['dost_exam_result']

        # Update grades
        grade_fields = ['mathematics', 'araling_panlipunan', 'english',
                        'edukasyon_sa_pagpapakatao', 'science',
                        'edukasyon_pangkabuhayan', 'filipino', 'mapeh']

        for field in grade_fields:
            if field in data and data[field]:
                setattr(academic_data, field, Decimal(str(data[field])))

        # Update special cases
        if 'is_working_student' in data:
            academic_data.is_working_student = data['is_working_student']
        if 'working_type' in data:
            academic_data.working_type = data['working_type']
        if 'is_pwd' in data:
            academic_data.is_pwd = data['is_pwd']
        if 'disability_type' in data:
            academic_data.disability_type = data['disability_type']

        academic_data.save()

    return JsonResponse({
        'success': True,
        'message': 'Academic data updated successfully',
    })

```

```

        'overall_average': float(academic_data.overall_average)
    })

except Exception as e:
    return JsonResponse({'success': False, 'error': str(e)}, status=500)

@login_required
@require_http_methods(["POST"])
def update_program_selection(request, student_id):
    """API endpoint to update program selection and admin approval"""
    try:
        student = get_object_or_404(Student, lrn=student_id)
        data = json.loads(request.body)

        program_selection, created = ProgramSelection.objects.get_or_create(student=student)

        # Update program selection fields
        if 'selected_program_code' in data:
            program_selection.selected_program_code = data['selected_program_code']
        if 'program_description' in data:
            program_selection.program_description = data['program_description']
        if 'selection_reason' in data:
            program_selection.selection_reason = data['selection_reason']

        # Admin fields
        if 'admin_approved' in data:
            program_selection.admin_approved = data['admin_approved']
        if 'admin_notes' in data:
            program_selection.admin_notes = data['admin_notes']
        if 'approved_by' in data:
            program_selection.approved_by = data['approved_by']
        if 'assigned_section' in data:
            program_selection.assigned_section = data['assigned_section']

        program_selection.save()

        return JsonResponse({'success': True, 'message': 'Program selection updated successfully'})

    except Exception as e:
        return JsonResponse({'success': False, 'error': str(e)}, status=500)

@login_required
@require_http_methods(["POST"])
def update_enrollment_status(request, student_id):
    """API endpoint to update student enrollment status"""
    try:

```

```

student = get_object_or_404(Student, ln=student_id)
data = json.loads(request.body)

if 'enrollment_status' in data:
    old_status = student.enrollment_status
    new_status = data['enrollment_status']

    student.enrollment_status = new_status
    student.save()

    # Log the status change
    from enrollment_app.models import EnrollmentStatusLog
    EnrollmentStatusLog.objects.create(
        student=student,
        old_status=old_status,
        new_status=new_status,
        changed_by=request.user.username if hasattr(request.user, 'username') else 'admin',
        change_reason=data.get('reason', "")
    )

    return JsonResponse({'success': True, 'message': 'Enrollment status updated successfully'})
else:
    return JsonResponse({'success': False, 'error': 'Status not provided'}, status=400)

except Exception as e:
    return JsonResponse({'success': False, 'error': str(e)}, status=500)

@login_required
@require_http_methods(["POST"])
def upload_student_file(request, student_id):
    """API endpoint to handle file uploads (photos, documents)"""
    try:
        student = get_object_or_404(Student, ln=student_id)
        file_type = request.POST.get('file_type')

        if 'file' not in request.FILES:
            return JsonResponse({'success': False, 'error': 'No file provided'}, status=400)

        uploaded_file = request.FILES['file']

        if file_type == 'student_photo':
            student_data = StudentData.objects.get(student=student)
            student_data.student_photo = uploaded_file
            student_data.save()
            file_url = student_data.student_photo.url

```



```

elif file_type == 'parent_photo':
    family_data = FamilyData.objects.get(student=student)
    family_data.parent_photo = uploaded_file
    family_data.save()
    file_url = family_data.parent_photo.url

elif file_type == 'report_card':
    academic_data = AcademicData.objects.get(student=student)
    academic_data.report_card = uploaded_file
    academic_data.save()
    file_url = academic_data.report_card.url
else:
    return JsonResponse({'success': False, 'error': 'Invalid file type'}, status=400)

return JsonResponse({'success': True, 'file_url': file_url, 'message': 'File uploaded
successfully'})

except Exception as e:
    return JsonResponse({'success': False, 'error': str(e)}, status=500)

@login_required
@require_http_methods(["GET"])
def get_sections_by_program(request):
    """
    GET /admin-portal/api/sections/?program=STE
    Returns all sections for a given program code.
    """
    try:
        program_code = request.GET.get('program', "").strip()
        if not program_code:
            return JsonResponse({'error': 'program parameter required'}, status=400)

        sections = Section.objects.filter(
            program__code=program_code
        ).select_related('adviser', 'program').order_by('name')

        data = []
        for s in sections:
            data.append({
                'id': s.id,
                'name': s.name,
                'adviser_name': s.adviser.get_full_name() if s.adviser else None,
                'current_students': s.students.count(),
                'max_students': s.max_students,
            })

```

```

        return JsonResponse({'sections': data})
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```
@login_required
```

```
@require_http_methods(["POST"])
```

```
def admin_move_student(request):
```

```
    """
```

```
    POST /admin-portal/api/admin-move/
```

```
    Admin directly moves a student to a different program and section.
```

```
    Body: { student_lrn, to_program_code, to_section_id, reason }
```

```
    """
```

```
    try:
```

```
        data = json.loads(request.body)
```

```
        student_lrn = data.get('student_lrn')
```

```
        to_program_code = data.get('to_program_code')
```

```
        to_section_id = data.get('to_section_id')
```

```
        reason = data.get('reason', "")
```

```
        if not all([student_lrn, to_program_code, to_section_id]):
```

```
            return JsonResponse({'error': 'student_lrn, to_program_code, and to_section_id are required'}, status=400)
```

```
    from enrollment_app.models import Student, ProgramSelection
```

```
    from admin_app.models import Section, Program
```

```
    student = get_object_or_404(Student, lrn=student_lrn)
```

```
    section = get_object_or_404(Section, pk=to_section_id)
```

```
    program = get_object_or_404(Program, code=to_program_code)
```

```
    with transaction.atomic():
```

```
        prog_sel, _ = ProgramSelection.objects.get_or_create(student=student)
```

```
        old_section_name = prog_sel.assigned_section or 'None'
```

```
        old_program_code = prog_sel.selected_program_code or 'None'
```

```
        # Remove student from old section if assigned
```

```
        if prog_sel.assigned_section:
```

```
            try:
```

```
                old_sec = Section.objects.get(name=prog_sel.assigned_section,
                                                program__code=old_program_code)
```

```
                old_sec.students.remove(student)
```

```
            except Section.DoesNotExist:
```

```
                pass
```

```
        # Assign to new section
```

```
        section.students.add(student)
```

```

# Update program selection
prog_sel.selected_program_code = to_program_code
prog_sel.assigned_section = section.name
prog_sel.admin_approved = True
prog_sel.admin_notes = (prog_sel.admin_notes or "") + \
    f'\n[Admin Move] {old_program_code}/{old_section_name} →
{to_program_code}/{section.name}. Reason: {reason}'
prog_sel.save()

# Update enrollment status
student.enrollment_status = 'approved'
student.save()

# Log activity
from admin_app.views import log_activity
log_activity(
    user=request.user,
    action='student_moved',
    description=f'Admin moved {student.ln} from
{old_program_code}/{old_section_name} to {to_program_code}/{section.name}. Reason:
{reason}',
    request=request
)

return JsonResponse({
    'success': True,
    'message': 'Student moved successfully',
    'program_name': f'{program.code} — {program.name}',
    'section_name': section.name,
})

except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

settings_views.py – System configuration (Grade levels, subjects, documents)

```

from django.shortcuts import render, get_object_or_404
from django.contrib.auth.decorators import login_required
from admin_app.decorators import admin_required
from django.http import JsonResponse
from django.views.decorators.http import require_http_methods
from django.contrib.auth.models import User
from django.db import transaction
from django.core.exceptions import ValidationError
from admin_app.models import (
    UserProfile, Position, Department, Program, SystemSettings,

```

```

    StaffMember, ActivityLog, Building, Room, Section, SchoolYear,
    DocumentRequirement, Teacher, GradeLevel, Subject
)
from django.core.files.storage import default_storage
from django.core.files.base import ContentFile
from datetime import datetime
import json
import base64
from django.views.decorators.csrf import csrf_exempt
from django.http import HttpResponseNotAllowed, HttpResponseBadRequest,
HttpResponseNotFound
from decimal import Decimal
from enrollment_app.models import StudentAcademicYearStatus, StudentEnrollment,
ProgramSelection
from coordinator_app.models import AcademicPerformance, ProbationRecord
from django.http import JsonResponse
from django.contrib.auth.models import User

```

```
@login_required
```

```
def get_user_profile(request, user_id):
```

```
    """
```

```
    GET /api/users/<user_id>/
```

```
    Returns user profile data including position_id, department_id, program_id
    so the edit modal can pre-select the correct dropdown options.
```

```
    """
```

```
    if request.method != 'GET':
```

```
        return HttpResponseNotAllowed(['GET'])
```

```
    try:
```

```
        user = User.objects.get(pk=user_id)
```

```
        profile = None
```

```
        try:
```

```
            profile = user.profile
```

```
        except Exception:
```

```
            pass
```

```
    data = {
```

```
        'id': user.id,
```

```
        'username': user.username,
```

```
        'email': user.email,
```

```
        'first_name': user.first_name,
```

```
        'last_name': user.last_name,
```

```
        'is_active': user.is_active,
```

```
        'date_joined': user.date_joined.strftime('%b %d, %Y'),
```

```
    }
```

```
    if profile:
```

```

        data.update({
            'employee_id': getattr(profile, 'employee_id', ""),
            'user_type': getattr(profile, 'user_type', ""),
            'position': profile.get_position_name() if hasattr(profile, 'get_position_name') else "",
            'department': profile.get_department_name() if hasattr(profile,
            'get_department_name') else "",
            'position_id': profile.position_id if profile.position_id else None,
            'department_id': profile.department_id if profile.department_id else None,
            'program_id': profile.program_id if profile.program_id else None,
        })

    return JsonResponse({'user': data})
except User.DoesNotExist:
    return HttpResponseNotFound('User not found')

```

```

@csrf_exempt
@login_required
def update_user_profile(request, user_id):
    """
    PUT /api/users/<user_id>/update/
    Updates user and profile fields.
    Accepts position_id, department_id, program_id as integers.
    """
    if request.method != 'PUT':
        return HttpResponseNotAllowed(['PUT'])
    try:
        user = User.objects.get(pk=user_id)
        try:
            data = json.loads(request.body.decode('utf-8'))
        except Exception:
            return HttpResponseBadRequest('Invalid JSON')

        user.first_name = data.get('first_name', user.first_name)
        user.last_name = data.get('last_name', user.last_name)
        user.email = data.get('email', user.email)

        if User.objects.filter(email=user.email).exclude(pk=user_id).exists():
            return JsonResponse({'error': 'Email already in use by another account'}, status=400)

        user.save()

    try:
        profile = user.profile
    except Exception:
        return JsonResponse({'success': True, 'message': 'User updated (no profile found)'})

    if 'employee_id' in data:

```

```

new_emp_id = data['employee_id']
if UserProfile.objects.filter(employee_id=new_emp_id).exclude(pk=profile.pk).exists():
    return JsonResponse({'error': 'Employee ID already in use'}, status=400)
profile.employee_id = new_emp_id

if 'user_type' in data:
    profile.user_type = data['user_type']
    user.is_staff = (data['user_type'] == 'admin')
    user.save(update_fields=['is_staff'])

if 'position_id' in data:
    pos_id = data['position_id']
    if pos_id:
        try:
            profile.position = Position.objects.get(pk=pos_id)
        except Position.DoesNotExist:
            return JsonResponse({'error': 'Invalid position ID'}, status=400)
    else:
        profile.position = None

if 'department_id' in data:
    dept_id = data['department_id']
    if dept_id:
        try:
            profile.department = Department.objects.get(pk=dept_id)
        except Department.DoesNotExist:
            return JsonResponse({'error': 'Invalid department ID'}, status=400)
    else:
        profile.department = None

if 'program_id' in data:
    prog_id = data['program_id']
    if prog_id:
        try:
            profile.program = Program.objects.get(pk=prog_id)
        except Program.DoesNotExist:
            return JsonResponse({'error': 'Invalid program ID'}, status=400)
    else:
        profile.program = None

profile.save()

log_activity(
    user=request.user,
    action='user_updated',
    description=f'Updated user profile: {user.get_full_name() or user.username}',
    request=request

```

```

    )

    return JsonResponse({
        'success': True,
        'message': 'User updated successfully',
        'user': {
            'id': user.id,
            'full_name': f'{user.first_name} {user.last_name}'.strip() or user.username,
            'email': user.email,
        }
    })

except User.DoesNotExist:
    return HttpResponseNotFound('User not found')
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

@csrf_exempt
@login_required
def delete_user_profile(request, user_id):
    if request.method != 'DELETE':
        return HttpResponseNotAllowed(['DELETE'])
    try:
        user = User.objects.get(pk=user_id)
        log_activity(
            user=request.user,
            action='user_deleted',
            description=f'Deleted user: {user.get_full_name() or user.username}',
            request=request
        )
        user.delete()
        return JsonResponse({'success': True, 'message': 'User deleted'})
    except User.DoesNotExist:
        return HttpResponseNotFound('User not found')

def log_activity(user, action, description, request=None):
    """Helper function to log activities to the database"""
    try:
        ip_address = None
        user_agent = None
        if request:
            x_forwarded_for = request.META.get('HTTP_X_FORWARDED_FOR')
            if x_forwarded_for:
                ip_address = x_forwarded_for.split(',')[0].strip()
            else:
                ip_address = request.META.get('REMOTE_ADDR')
            user_agent = request.META.get('HTTP_USER_AGENT', '')[:500]

```

```

        ActivityLog.objects.create(
            user=user,
            action=action,
            description=description,
            ip_address=ip_address,
            user_agent=user_agent
        )
    except Exception as e:
        print(f"Error logging activity: {str(e)}")

```

— NEW: Document requirement carry-over helper

```

def _carry_over_document_requirements(from_sy, to_sy):
    """
    Auto-copy active document requirements from a previous school year to a
    new school year. Called when a new school year is activated.

    Args:
        from_sy: SchoolYear instance being deactivated (the previous SY).
        to_sy: SchoolYear instance being activated (the new SY).
    """
    if from_sy is None or to_sy is None:
        return

    old_reqs = DocumentRequirement.objects.filter(school_year=from_sy, is_active=True)
    copied = 0
    for req in old_reqs:
        # Only create if a requirement with the same name doesn't already exist
        if not DocumentRequirement.objects.filter(school_year=to_sy, name=req.name).exists():
            DocumentRequirement.objects.create(
                school_year=to_sy,
                name=req.name,
                description=req.description,
                requirement_type=req.requirement_type,
                file_format=req.file_format,
                max_file_size_mb=req.max_file_size_mb,
                is_active=req.is_active,
                order=req.order,
            )
            copied += 1
    print(f"[SY ACTIVATION] Carried over {copied} document requirements to {to_sy.year_label}")

```


#

```
@login_required
def settings(request):
    return render(request, 'admin_app/settings.html', {'active_page': 'settings'})

@login_required
@require_http_methods(["GET"])
def settings_header_data(request):
    """API endpoint for settings header data"""
    from admin_app.models import SchoolYear, UserProfile
    active_school_year = SchoolYear.get_active_school_year()
    try:
        user_profile = UserProfile.objects.select_related(
            'program', 'position', 'department'
        ).get(user=request.user)
    except UserProfile.DoesNotExist:
        user_profile = None

    user = request.user
    full_name = f"{user.first_name} {user.last_name}" if user.first_name and user.last_name else
    user.username
    if user.first_name and user.last_name:
        initials = f"{user.first_name[0]}{user.last_name[0]}.upper()"
    elif user.first_name:
        initials = user.first_name[0].upper()
    elif user.last_name:
        initials = user.last_name[0].upper()
    else:
        initials = user.username[0].upper() if user.username else "U"

    role = user_profile.get_user_type_display() if user_profile else "Admin"
    photo_url = user_profile.photo.url if user_profile and user_profile.photo else None

    return JsonResponse({
        'school_year': active_school_year.year_label if active_school_year else 'No Active Year',
        'full_name': full_name,
        'role': role,
        'initials': initials,
        'photo_url': photo_url,
    })

@login_required
def manage_users(request):
    return render(request, 'admin_app/settings.html', {'tab': 'users', 'active_page': 'settings'})
```

```

@login_required
def manage_content(request):
    return render(request, 'admin_app/settings.html', {'tab': 'content', 'active_page': 'settings'})

```

===== API ENDPOINTS =====

```

@login_required
@require_http_methods(["GET"])
def get_users(request):
    """Get all users with their profiles"""
    try:
        users_data = []
        users = User.objects.select_related('profile').all().order_by('-date_joined')
        for user in users:
            try:
                profile = user.profile
                users_data.append({
                    'id': user.id,
                    'username': user.username,
                    'first_name': user.first_name,
                    'last_name': user.last_name,
                    'full_name': f"{user.first_name} {user.last_name}".strip() or user.username,
                    'email': user.email,
                    'employee_id': profile.employee_id if hasattr(profile, 'employee_id') else 'N/A',
                    'position': profile.get_position_name(),
                    'department': profile.get_department_name(),
                    'user_type': profile.user_type,
                    'access_badges': profile.get_access_badges(),
                    'last_login': profile.get_last_login_formatted(),
                    'date_joined': profile.get_date_joined_formatted(),
                    'is_active': user.is_active,
                })
            except UserProfile.DoesNotExist:
                users_data.append({
                    'id': user.id,
                    'username': user.username,
                    'first_name': user.first_name,
                    'last_name': user.last_name,
                    'full_name': f"{user.first_name} {user.last_name}".strip() or user.username,
                    'email': user.email,
                    'employee_id': 'N/A',
                    'position': 'N/A',
                    'department': 'N/A',
                    'user_type': 'N/A',
                    'access_badges': [],
                })
    
```

```

        'last_login': 'Never',
        'date_joined': user.date_joined.strftime('%b %d, %Y'),
        'is_active': user.is_active,
    })
    return JsonResponse({'users': users_data}, status=200)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["POST"])
def add_user(request):
    """Add a new user with profile"""
    try:
        data = json.loads(request.body)
        required_fields = ['username', 'email', 'first_name', 'last_name', 'employee_id', 'password']
        for field in required_fields:
            if not data.get(field):
                return JsonResponse({'error': f'{field.replace("_", " ").title()} is required'}, status=400)
        if User.objects.filter(username=data['username']).exists():
            return JsonResponse({'error': 'Username already exists'}, status=400)
        if User.objects.filter(email=data['email']).exists():
            return JsonResponse({'error': 'Email already exists'}, status=400)
        if UserProfile.objects.filter(employee_id=data['employee_id']).exists():
            return JsonResponse({'error': 'Employee ID already exists'}, status=400)
        user_type = 'admin' if data.get('admin_access') else 'coordinator'
        if user_type == 'coordinator' and not data.get('program'):
            return JsonResponse({'error': 'Program is required for coordinators'}, status=400)
        with transaction.atomic():
            user = User.objects.create_user(
                username=data['username'],
                email=data['email'],
                first_name=data['first_name'],
                last_name=data['last_name'],
                password=data['password']
            )
            position = None
            if data.get('position'):
                try:
                    position = Position.objects.get(id=data['position'])
                except Position.DoesNotExist:
                    pass
            department = None
            if data.get('department'):
                try:
                    department = Department.objects.get(id=data['department'])
                except Department.DoesNotExist:

```

```

        pass
    program = None
    if data.get('program'):
        try:
            program = Program.objects.get(id=data['program'])
        except Program.DoesNotExist:
            pass
    profile = UserProfile.objects.create(
        user=user,
        user_type=user_type,
        employee_id=data['employee_id'],
        position=position,
        department=department,
        program=program
    )
    if user_type == 'admin':
        user.is_staff = True
        user.save()
    return JsonResponse({
        'message': 'User added successfully',
        'user': {'id': user.id, 'username': user.username,
                  'full_name': f"{user.first_name} {user.last_name}", 'email': user.email}
    }, status=201)
except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["GET"])
def get_positions(request):
    try:
        positions_qs = Position.objects.filter(is_active=True).values('id', 'name', 'description')
        positions = list(positions_qs)
        return JsonResponse({'positions': positions, 'data': positions}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["GET"])
def get_departments(request):
    try:
        departments_qs = Department.objects.filter(is_active=True).values('id', 'name',
        'description')
        departments = list(departments_qs)
    
```

```

        return JsonResponse({'departments': departments, 'data': departments}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["POST"])
def add_position(request):
    try:
        data = json.loads(request.body)
        name = (data.get('name') or "").strip()
        description = data.get('description') or ""
        if not name:
            return JsonResponse({'error': 'Position name is required'}, status=400)
        if Position.objects.filter(name__iexact=name).exists():
            return JsonResponse({'error': 'Position name already exists'}, status=400)
        position = Position.objects.create(name=name, description=description)
        log_activity(user=request.user, action='position_added',
                    description=f'Added position: {name}', request=request)
        return JsonResponse({'message': 'Position added successfully',
                            'position': {'id': position.id, 'name': position.name,
                                         'description': position.description}}, status=201)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON data'}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["PUT"])
def update_position(request, position_id):
    try:
        position = Position.objects.get(pk=position_id)
        data = json.loads(request.body)
        name = (data.get('name') or "").strip()
        description = data.get('description') or ""
        if not name:
            return JsonResponse({'error': 'Position name is required'}, status=400)
        if Position.objects.filter(name__iexact=name).exclude(pk=position_id).exists():
            return JsonResponse({'error': 'Another position with this name already exists'},
                                status=400)
        old_name = position.name
        position.name = name
        position.description = description
        position.save()
        log_activity(user=request.user, action='position_updated',
                    description=f'Updated position: {old_name} -> {name}', request=request)

```

```

        return JsonResponse({'message': 'Position updated successfully',
                              'position': {'id': position.id, 'name': position.name,
                                             'description': position.description}}, status=200)
    except Position.DoesNotExist:
        return JsonResponse({'error': 'Position not found'}, status=404)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON data'}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["DELETE"])
def delete_position(request, position_id):
    try:
        position = Position.objects.get(pk=position_id)
        position_name = position.name
        if not position.can_delete():
            return JsonResponse({'error': 'Cannot delete position with assigned users'}, status=400)
        position.delete()
        log_activity(user=request.user, action='position_deleted',
                     description=f'Deleted position: {position_name}', request=request)
        return JsonResponse({'message': 'Position deleted successfully'}, status=200)
    except Position.DoesNotExist:
        return JsonResponse({'error': 'Position not found'}, status=404)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["POST"])
def add_department(request):
    try:
        data = json.loads(request.body)
        name = (data.get('name') or "").strip()
        description = data.get('description') or ""
        if not name:
            return JsonResponse({'error': 'Department name is required'}, status=400)
        if Department.objects.filter(name__iexact=name).exists():
            return JsonResponse({'error': 'Department name already exists'}, status=400)
        department = Department.objects.create(name=name, description=description)
        log_activity(user=request.user, action='department_added',
                     description=f'Added department: {name}', request=request)
        return JsonResponse({'message': 'Department added successfully',
                              'department': {'id': department.id, 'name': department.name,
                                             'description': department.description}}, status=201)
    except json.JSONDecodeError:

```

```

        return JsonResponse({'error': 'Invalid JSON data'}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["PUT"])
def update_department(request, department_id):
    try:
        department = Department.objects.get(pk=department_id)
        data = json.loads(request.body)
        name = (data.get('name') or "").strip()
        description = data.get('description') or ""
        if not name:
            return JsonResponse({'error': 'Department name is required'}, status=400)
        if Department.objects.filter(name__iexact=name).exclude(pk=department_id).exists():
            return JsonResponse({'error': 'Another department with this name already exists'},
                                status=400)
        old_name = department.name
        department.name = name
        department.description = description
        department.save()
        log_activity(user=request.user, action='department_updated',
                     description=f'Updated department: {old_name} -> {name}', request=request)
        return JsonResponse({'message': 'Department updated successfully',
                             'department': {'id': department.id, 'name': department.name,
                                             'description': department.description}}, status=200)
    except Department.DoesNotExist:
        return JsonResponse({'error': 'Department not found'}, status=404)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON data'}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["DELETE"])
def delete_department(request, department_id):
    try:
        department = Department.objects.get(pk=department_id)
        department_name = department.name
        if not department.can_delete():
            return JsonResponse({'error': 'Cannot delete department with assigned users'},
                                status=400)
        department.delete()
        log_activity(user=request.user, action='department_deleted',
                     description=f'Deleted department: {department_name}', request=request)

```

```

        return JsonResponse({'message': 'Department deleted successfully'}, status=200)
    except Department.DoesNotExist:
        return JsonResponse({'error': 'Department not found'}, status=404)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["GET"])
def get_programs(request):
    try:
        programs = Program.objects.filter(is_active=True).values('id', 'code', 'name')
        programs_list = [{ 'id': p['id'], 'name': f"{p['code']} - {p['name']}" } for p in programs]
        return JsonResponse({'programs': programs_list}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

===== CONTENT MANAGEMENT ENDPOINTS =====

```

@login_required
@require_http_methods(["GET"])
def get_content_settings(request):
    try:
        settings = SystemSettings.objects.all()
        settings_dict = {}
        for setting in settings:
            settings_dict[setting.setting_type] = {
                'value': setting.setting_value,
                'image_url': setting.image.url if setting.image else None,
                'updated_at': setting.get_formatted_date(),
                'updated_by': setting.updated_by.get_full_name() if setting.updated_by else 'System'
            }
        return JsonResponse({'settings': settings_dict}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["POST"])
def save_content_setting(request):
    try:
        data = json.loads(request.body)
        setting_type = data.get('setting_type')
        setting_value = data.get('setting_value', "")
        if not setting_type:
            return JsonResponse({'error': 'Setting type is required'}, status=400)

```



```

        setting, created = SystemSettings.objects.get_or_create(
            setting_type=setting_type,
            defaults={'setting_value': setting_value, 'updated_by': request.user}
        )
        if not created:
            setting.setting_value = setting_value
            setting.updated_by = request.user
            setting.save()
        log_activity(user=request.user, action='content_updated',
            description=f'Updated content setting: {setting_type}', request=request)
        return JsonResponse({'message': 'Setting saved successfully',
            'setting': {'type': setting.setting_type, 'value': setting.setting_value,
                'updated_at': setting.get_formatted_date()}}, status=200)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON data'}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["POST"])
def upload_content_image(request):
    try:
        if 'image' not in request.FILES:
            return JsonResponse({'error': 'No image file provided'}, status=400)
        setting_type = request.POST.get('setting_type')
        if not setting_type:
            return JsonResponse({'error': 'Setting type is required'}, status=400)
        image_file = request.FILES['image']
        setting, created = SystemSettings.objects.get_or_create(
            setting_type=setting_type,
            defaults={'updated_by': request.user}
        )
        setting.image = image_file
        setting.updated_by = request.user
        setting.save()
        return JsonResponse({'message': 'Image uploaded successfully', 'image_url':
            setting.image.url,
            'setting': {'type': setting.setting_type,
                'updated_at': setting.get_formatted_date()}}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["GET"])
def get_staff_members(request):

```

```

try:
    staff = StaffMember.objects.filter(is_active=True).order_by('display_order', 'name')
    staff_list = [{ 'id': m.id, 'name': m.name, 'position': m.position,
                    'photo_url': m.photo.url if m.photo else None,
                    'display_order': m.display_order } for m in staff]
    return JsonResponse({'staff': staff_list}, status=200)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["POST"])
def add_staff_member(request):
    try:
        name = request.POST.get('name', "").strip()
        position = request.POST.get('position', "").strip()
        display_order = int(request.POST.get('display_order', 0))
        if not name or not position:
            return JsonResponse({'error': 'Name and position are required'}, status=400)
        staff = StaffMember.objects.create(name=name, position=position,
        display_order=display_order)
        if 'photo' in request.FILES:
            staff.photo = request.FILES['photo']
            staff.save()
        return JsonResponse({'message': 'Staff member added successfully',
                            'staff': { 'id': staff.id, 'name': staff.name, 'position': staff.position,
                                        'photo_url': staff.photo.url if staff.photo else None }}, status=201)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["PUT"])
def update_staff_member(request, staff_id):
    try:
        staff = StaffMember.objects.get(pk=staff_id)
        data = json.loads(request.body)
        name = data.get('name', "").strip()
        position = data.get('position', "").strip()
        if not name or not position:
            return JsonResponse({'error': 'Name and position are required'}, status=400)
        staff.name = name
        staff.position = position
        staff.display_order = int(data.get('display_order', staff.display_order))
        staff.save()
        return JsonResponse({'message': 'Staff member updated successfully',
                            'staff': { 'id': staff.id, 'name': staff.name, 'position': staff.position }}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

except StaffMember.DoesNotExist:
    return JsonResponse({'error': 'Staff member not found'}, status=404)
except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["DELETE"])
def delete_staff_member(request, staff_id):
    try:
        staff = StaffMember.objects.get(pk=staff_id)
        staff.delete()
        return JsonResponse({'message': 'Staff member deleted successfully'}, status=200)
    except StaffMember.DoesNotExist:
        return JsonResponse({'error': 'Staff member not found'}, status=404)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["GET"])
def get_activity_logs(request):
    try:
        action_filter = request.GET.get('action', "")
        user_filter = request.GET.get('user', "")
        search = request.GET.get('search', "")
        limit = int(request.GET.get('limit', 100))
        logs = ActivityLog.objects.all().order_by('-created_at')
        if action_filter:
            logs = logs.filter(action=action_filter)
        if user_filter:
            logs = logs.filter(user__username=user_filter)
        if search:
            logs = logs.filter(description__icontains=search)
        logs = logs[:limit]
        logs_data = []
        for log in logs:
            logs_data.append({
                'id': log.id,
                'user': log.user.get_full_name() or log.user.username if log.user else 'System',
                'action': log.get_action_display(),
                'action_code': log.action,
                'description': log.description,
                'ip_address': log.ip_address or 'N/A',
                'date': log.created_at.strftime('%Y-%m-%d'),
            })
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

        'time': log.created_at.strftime('%H:%M:%S'),
        'timestamp': log.created_at.isoformat(),
    })
    return JsonResponse({'logs': logs_data, 'count': len(logs_data)}, status=200)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

===== BUILDINGS MANAGEMENT =====

```

@login_required
@require_http_methods(["GET"])
def get_buildings_with_rooms(request):
    try:
        buildings = Building.objects.prefetch_related('rooms').all().order_by('name')
        buildings_list = [{'id': b.id, 'name': b.name, 'room_count': b.rooms.count()} for b in buildings]
        return JsonResponse({'buildings': buildings_list}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["POST"])
def add_building(request):
    try:
        data = json.loads(request.body)
        name = (data.get('name') or "").strip()
        if not name:
            return JsonResponse({'error': 'Building name is required'}, status=400)
        if Building.objects.filter(name__iexact=name).exists():
            return JsonResponse({'error': 'Building name already exists'}, status=400)
        building = Building.objects.create(name=name)
        log_activity(user=request.user, action='settings_changed',
                    description=f'Added building: {name}', request=request)
        return JsonResponse({'message': 'Building added successfully',
                            'building': {'id': building.id, 'name': building.name, 'room_count': 0}},
                            status=201)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON data'}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["PUT"])
def update_building(request, building_id):
    try:

```

```

    building = Building.objects.get(pk=building_id)
    data = json.loads(request.body)
    name = (data.get('name') or "").strip()
    if not name:
        return JsonResponse({'error': 'Building name is required'}, status=400)
    if Building.objects.filter(name__iexact=name).exclude(pk=building_id).exists():
        return JsonResponse({'error': 'Another building with this name already exists'},
status=400)
    old_name = building.name
    building.name = name
    building.save()
    log_activity(user=request.user, action='settings_changed',
        description=f'Updated building: {old_name} -> {name}', request=request)
    return JsonResponse({'message': 'Building updated successfully',
        'building': {'id': building.id, 'name': building.name,
            'room_count': building.rooms.count()}}, status=200)
except Building.DoesNotExist:
    return JsonResponse({'error': 'Building not found'}, status=404)
except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

@login_required
@require_http_methods(["DELETE"])
@transaction.atomic
def delete_building(request, building_id):
    try:
        building = Building.objects.get(pk=building_id)
        building_name = building.name
        room_count = building.rooms.count()
        rooms_in_use = []
        for room in building.rooms.all():
            if Section.objects.filter(building=building_name, room=room.room_number).exists():
                rooms_in_use.append(room.room_number)
        if rooms_in_use:
            return JsonResponse({
                'error': f'Cannot delete building. The following rooms are assigned to sections: {"',
".join(rooms_in_use)}'
            }, status=400)
        building.delete()
        log_activity(user=request.user, action='settings_changed',
            description=f'Deleted building: {building_name} (with {room_count} rooms)',
request=request)
        return JsonResponse({'message': 'Building and all its rooms deleted successfully'},
status=200)

```

```

except Building.DoesNotExist:
    return JsonResponse({'error': 'Building not found'}, status=404)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

===== ROOMS MANAGEMENT =====

```

@login_required
@require_http_methods(["GET"])
def get_rooms_by_building(request):
    try:
        building_id = request.GET.get('building_id')
        if not building_id:
            return JsonResponse({'error': 'Building ID is required'}, status=400)
        building = Building.objects.get(pk=building_id)
        rooms = building.rooms.all().order_by('room_number')
        rooms_list = [{'id': r.id, 'room_number': r.room_number,
                        'building_id': building.id, 'building_name': building.name} for r in rooms]
        return JsonResponse({'rooms': rooms_list}, status=200)
    except Building.DoesNotExist:
        return JsonResponse({'error': 'Building not found'}, status=404)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["POST"])
def add_room(request):
    try:
        data = json.loads(request.body)
        building_id = data.get('building_id')
        room_number = (data.get('room_number') or "").strip()
        if not building_id:
            return JsonResponse({'error': 'Building ID is required'}, status=400)
        if not room_number:
            return JsonResponse({'error': 'Room number is required'}, status=400)
        building = Building.objects.get(pk=building_id)
        if Room.objects.filter(building=building, room_number__iexact=room_number).exists():
            return JsonResponse({'error': 'Room number already exists in this building'}, status=400)
        room = Room.objects.create(building=building, room_number=room_number)
        log_activity(user=request.user, action='settings_changed',
                    description=f'Added room {room_number} to {building.name}', request=request)
        return JsonResponse({'message': 'Room added successfully',
                            'room': {'id': room.id, 'room_number': room.room_number,
                                     'building_id': building.id, 'building_name': building.name}}, status=201)
    except Building.DoesNotExist:

```

```

        return JsonResponse({'error': 'Building not found'}, status=404)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON data'}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["PUT"])
def update_room(request, room_id):
    try:
        room = Room.objects.select_related('building').get(pk=room_id)
        data = json.loads(request.body)
        room_number = (data.get('room_number') or "").strip()
        if not room_number:
            return JsonResponse({'error': 'Room number is required'}, status=400)
        if Room.objects.filter(building=room.building,
room_number__iexact=room_number).exclude(pk=room_id).exists():
            return JsonResponse({'error': 'Room number already exists in this building'}, status=400)
        old_room_number = room.room_number
        room.room_number = room_number
        room.save()
        Section.objects.filter(building=room.building.name,
room=old_room_number).update(room=room_number)
        log_activity(user=request.user, action='settings_changed',
description=f'Updated room in {room.building.name}: {old_room_number} ->
{room_number}',
request=request)
        return JsonResponse({'message': 'Room updated successfully',
'room': {'id': room.id, 'room_number': room.room_number,
'building_id': room.building.id, 'building_name': room.building.name}},
status=200)
    except Room.DoesNotExist:
        return JsonResponse({'error': 'Room not found'}, status=404)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON data'}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["DELETE"])
def delete_room(request, room_id):
    try:
        room = Room.objects.select_related('building').get(pk=room_id)
        room_number = room.room_number
        building_name = room.building.name

```

```

        sections_using_room = Section.objects.filter(building=building_name,
room=room_number)
        if sections_using_room.exists():
            section_names = ', '.join([s.name for s in sections_using_room[:3]])
            if sections_using_room.count() > 3:
                section_names += f' and {sections_using_room.count() - 3} more'
            return JsonResponse({
                'error': f'Cannot delete room. It is assigned to the following sections: {section_names}'
            }, status=400)
        room.delete()
        log_activity(user=request.user, action='settings_changed',
            description=f'Deleted room {room_number} from {building_name}',
request=request)
        return JsonResponse({'message': 'Room deleted successfully'}, status=200)
    except Room.DoesNotExist:
        return JsonResponse({'error': 'Room not found'}, status=404)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

===== SCHOOL YEAR MANAGEMENT =====

```

def _parse_date(date_str, field_name):
    try:
        return datetime.strptime(date_str, '%Y-%m-%d').date()
    except (TypeError, ValueError):
        raise ValidationError({'field_name': 'Invalid date format. Use YYYY-MM-DD.'})

```

```

def _school_year_to_dict(school_year):
    return {
        'id': school_year.id,
        'year_label': school_year.year_label,
        'start_date': school_year.start_date.isoformat(),
        'end_date': school_year.end_date.isoformat(),
        'is_active': school_year.is_active,
        'enrollment_open': school_year.enrollment_open,
        'created_at': school_year.created_at.isoformat(),
        'updated_at': school_year.updated_at.isoformat(),
        'sections_count': school_year.sections.count(),
        'students_count': school_year.get_total_students(),
    }

```

```

@login_required
@require_http_methods(["GET"])
def get_school_years(request):

```



```

try:
    school_years = SchoolYear.objects.all().order_by('-year_label')
    data = [_school_year_to_dict(sy) for sy in school_years]
    active_year = next((sy for sy in data if sy['is_active']), None)
    return JsonResponse({'school_years': data, 'active_year': active_year}, status=200)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["POST"])
def add_school_year(request):
    try:
        payload = json.loads(request.body)
        year_label = (payload.get('year_label') or "").strip()
        start_date_raw = payload.get('start_date')
        end_date_raw = payload.get('end_date')
        is_active = bool(payload.get('is_active'))
        enrollment_open = bool(payload.get('enrollment_open'))

        if not year_label or not start_date_raw or not end_date_raw:
            return JsonResponse({'error': 'Year label, start date, and end date are required'},
                                status=400)

        start_date = _parse_date(start_date_raw, 'start_date')
        end_date = _parse_date(end_date_raw, 'end_date')

        school_year = SchoolYear(
            year_label=year_label,
            start_date=start_date,
            end_date=end_date,
            is_active=is_active,
            enrollment_open=enrollment_open
        )
        school_year.save()

```

```

# — Carry over document requirements if this year is being created as active —
if is_active:
    source_sy = (
        SchoolYear.objects.exclude(pk=school_year.pk)
        .filter(document_requirements__isnull=False)
        .distinct()
        .order_by('-year_label')
        .first()
    )
    if source_sy:
        _carry_over_document_requirements(

```

```

        from_sy=source_sy,
        to_sy=school_year,
    )

    log_activity(user=request.user, action='settings_changed',
                description=f'Added school year: {year_label}', request=request)

    return JsonResponse({'message': 'School year added successfully',
                        'school_year': _school_year_to_dict(school_year)}, status=201)
except ValidationError as ve:
    error_messages = (
        '; '.join([f'{field}: {' '.join(msgs)}' for field, msgs in ve.message_dict.items()])
        if hasattr(ve, 'message_dict') else str(ve)
    )
    return JsonResponse({'error': error_messages}, status=400)
except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

@login_required

@require_http_methods(["PUT"])

def update_school_year(request, school_year_id):

"""

Update an existing school year.

When is_active is flipped to True, the previously active SY is deactivated
and its document requirements are automatically carried over to the new SY.

"""

try:

```

    school_year = SchoolYear.objects.get(pk=school_year_id)
    payload = json.loads(request.body)

```

```

    year_label = (payload.get('year_label') or "").strip()
    start_date_raw = payload.get('start_date')
    end_date_raw = payload.get('end_date')
    new_is_active = bool(payload.get('is_active'))
    enrollment_open = bool(payload.get('enrollment_open'))

```

if not year_label or not start_date_raw or not end_date_raw:

```

    return JsonResponse({'error': 'Year label, start date, and end date are required'},
                        status=400)

```

— Detect activation transition

```

was_inactive = not school_year.is_active
being_activated = new_is_active and was_inactive

```

```

# Capture the previously active SY BEFORE saving (SchoolYear.save())
# may deactivate it automatically depending on your model logic).
previous_active_sy = None
if being_activated:
    previous_active_sy = SchoolYear.objects.filter(
        is_active=True
    ).exclude(pk=school_year_id).first()

# — Persist changes


---


school_year.year_label = year_label
school_year.start_date = _parse_date(start_date_raw, 'start_date')
school_year.end_date = _parse_date(end_date_raw, 'end_date')
school_year.is_active = new_is_active
school_year.enrollment_open = enrollment_open
school_year.save()

# — Carry over document requirements when activating 

---


if being_activated:
    # Find the most recent school year (other than this one) that has requirements
    source_sy = None
    if previous_active_sy:
        source_sy = previous_active_sy
    # Fallback: find any other year with requirements if previous active has none
    if not source_sy or not
DocumentRequirement.objects.filter(school_year=source_sy).exists():
    source_sy = (
        SchoolYear.objects.exclude(pk=school_year.pk)
        .filter(document_requirements__isnull=False)
        .distinct()
        .order_by('-year_label')
        .first()
    )
    if source_sy:
        _carry_over_document_requirements(
            from_sy=source_sy,
            to_sy=school_year,
        )

log_activity(user=request.user, action='settings_changed',
            description=f'Updated school year: {year_label}', request=request)

return JsonResponse({'message': 'School year updated successfully',
                    'school_year': _school_year_to_dict(school_year)}, status=200)

except SchoolYear.DoesNotExist:

```

```

        return JsonResponse({'error': 'School year not found'}, status=404)
    except ValidationError as ve:
        error_messages = (
            '; '.join([f'{field}: {'; '.join(msgs)}' for field, msgs in ve.message_dict.items()])
            if hasattr(ve, 'message_dict') else str(ve)
        )
        return JsonResponse({'error': error_messages}, status=400)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON data'}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["DELETE"])
def delete_school_year(request, school_year_id):
    try:
        school_year = SchoolYear.objects.get(pk=school_year_id)
        if school_year.is_active:
            return JsonResponse({'error': 'Cannot delete the active school year'}, status=400)
        if school_year.sections.exists() or school_year.get_total_students() > 0:
            sections_count = school_year.sections.count()
            students_count = school_year.get_total_students()
            return JsonResponse({
                'error': (
                    'Cannot delete school year with existing data '
                    f'(sections: {sections_count}, students: {students_count}).'
                )
            }, status=400)
        year_label = school_year.year_label
        school_year.delete()
        log_activity(user=request.user, action='settings_changed',
                    description=f'Deleted school year: {year_label}', request=request)
        return JsonResponse({'message': 'School year deleted successfully'}, status=200)
    except SchoolYear.DoesNotExist:
        return JsonResponse({'error': 'School year not found'}, status=404)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

===== DOCUMENT REQUIREMENTS MANAGEMENT =====

```

def _document_requirement_to_dict(req: DocumentRequirement):
    return {
        'id': req.id,
        'school_year_id': req.school_year_id,
        'school_year': req.school_year.year_label if req.school_year else None,
    }

```

```

'name': req.name,
'description': req.description,
'requirement_type': req.requirement_type,
'file_format': req.file_format,
'allowed_extensions': req.get_allowed_extensions(),
'max_file_size_mb': float(req.max_file_size_mb),
'is_active': req.is_active,
'order': req.order,
'created_at': req.created_at.isoformat(),
'updated_at': req.updated_at.isoformat(),
}

```

```

@login_required
@require_http_methods(["GET"])
def get_document_requirements(request):
    try:
        school_year_id = request.GET.get('school_year_id')
        search = (request.GET.get('search') or "").strip()
        qs = DocumentRequirement.objects.select_related('school_year').all().order_by(
            'school_year__year_label', 'order', 'name')
        if school_year_id:
            qs = qs.filter(school_year_id=school_year_id)
        if search:
            qs = qs.filter(name__icontains=search)
        data = [_document_requirement_to_dict(r) for r in qs]
        return JsonResponse({'requirements': data, 'count': len(data)}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["POST"])
def add_document_requirement(request):
    try:
        payload = json.loads(request.body)
        school_year_id = payload.get('school_year_id')
        name = (payload.get('name') or "").strip()
        description = payload.get('description') or ""
        requirement_type = (payload.get('requirement_type') or 'mandatory').strip()
        file_format = (payload.get('file_format') or 'pdf,jpg,jpeg,png').strip()
        max_file_size_mb = payload.get('max_file_size_mb', 5.0)
        is_active = bool(payload.get('is_active', True))
        order = int(payload.get('order', 0))

        if not school_year_id:
            return JsonResponse({'error': 'school_year_id is required'}, status=400)

```

```

if not name:
    return JsonResponse({'error': 'name is required'}, status=400)

try:
    school_year = SchoolYear.objects.get(pk=school_year_id)
except SchoolYear.DoesNotExist:
    return JsonResponse({'error': 'School year not found'}, status=404)

if DocumentRequirement.objects.filter(school_year=school_year,
name__iexact=name).exists():
    return JsonResponse(
        {'error': 'A requirement with this name already exists for the selected school year'},
        status=400)

req = DocumentRequirement(
    school_year=school_year, name=name, description=description,
    requirement_type=requirement_type, file_format=file_format,
    max_file_size_mb=max_file_size_mb, is_active=is_active, order=order,
    created_by=request.user,
)
req.save()

log_activity(user=request.user, action='settings_changed',
    description=f'Added document requirement "{name}" for {school_year.year_label}',
    request=request)

return JsonResponse({'message': 'Requirement added successfully',
    'requirement': _document_requirement_to_dict(req)}, status=201)
except ValidationError as ve:
    error_messages = (
        '; '.join([f'{field}: {', '.join(msgs)}' for field, msgs in ve.message_dict.items()])
        if hasattr(ve, 'message_dict') else str(ve)
    )
    return JsonResponse({'error': error_messages}, status=400)
except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["PUT"])
def update_document_requirement(request, requirement_id: int):
    try:
        try:
            req =
DocumentRequirement.objects.select_related('school_year').get(pk=requirement_id)

```

```

except DocumentRequirement.DoesNotExist:
    return JsonResponse({'error': 'Requirement not found'}, status=404)

payload = json.loads(request.body)
name = (payload.get('name') or req.name).strip()
description = payload.get('description', req.description)
requirement_type = (payload.get('requirement_type') or req.requirement_type).strip()
file_format = (payload.get('file_format') or req.file_format).strip()
max_file_size_mb = payload.get('max_file_size_mb', float(req.max_file_size_mb))
is_active = bool(payload.get('is_active', req.is_active))
order = int(payload.get('order', req.order))

if not name:
    return JsonResponse({'error': 'name is required'}, status=400)

if DocumentRequirement.objects.filter(
    school_year=req.school_year, name__iexact=name
).exclude(pk=req.id).exists():
    return JsonResponse(
        {'error': 'Another requirement with this name already exists for this school year'},
status=400)

old_name = req.name
req.name = name
req.description = description
req.requirement_type = requirement_type
req.file_format = file_format
req.max_file_size_mb = max_file_size_mb
req.is_active = is_active
req.order = order
req.save()

log_activity(user=request.user, action='settings_changed',
    description=f'Updated document requirement: {old_name} -> {req.name}'
    ({req.school_year.year_label}),
    request=request)

return JsonResponse({'message': 'Requirement updated successfully',
    'requirement': _document_requirement_to_dict(req)}, status=200)
except ValidationError as ve:
    error_messages = (
        '; '.join([f'{field}: {'; '.join(msgs)}' for field, msgs in ve.message_dict.items()])
        if hasattr(ve, 'message_dict') else str(ve)
    )
    return JsonResponse({'error': error_messages}, status=400)
except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)

```

```
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)
```

```
@login_required
@require_http_methods(["DELETE"])
def delete_document_requirement(request, requirement_id: int):
    try:
        try:
            req =
DocumentRequirement.objects.select_related('school_year').get(pk=requirement_id)
        except DocumentRequirement.DoesNotExist:
            return JsonResponse({'error': 'Requirement not found'}, status=404)

        name = req.name
        year_label = req.school_year.year_label if req.school_year else 'Unknown'
        req.delete()

        log_activity(user=request.user, action='settings_changed',
                    description=f'Deleted document requirement "{name}" ({year_label})',
request=request)

        return JsonResponse({'message': 'Requirement deleted successfully'}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)
```

```
# ===== GRADE LEVEL MANAGEMENT =====
```

```
@login_required
@require_http_methods(["GET"])
def get_grade_levels(request):
    try:
        from admin_app.models import GradeLevel
        qs = GradeLevel.objects.all().order_by('code')
        data = [{'id': gl.id, 'code': gl.code, 'name': gl.name,
                'description': gl.description or "", 'is_active': gl.is_active} for gl in qs]
        return JsonResponse({'grade_levels': data, 'count': len(data)}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)
```

```
@login_required
@require_http_methods(["POST"])
def add_grade_level(request):
    try:
        from admin_app.models import GradeLevel
```



```

payload = json.loads(request.body)
code = (payload.get('code') or "").strip().upper()
name = (payload.get('name') or "").strip()
description = payload.get('description') or ""
is_active = bool(payload.get('is_active', True))
if not code:
    return JsonResponse({'error': 'Code is required'}, status=400)
if not name:
    return JsonResponse({'error': 'Name is required'}, status=400)
if GradeLevel.objects.filter(code__iexact=code).exists():
    return JsonResponse({'error': f'A grade level with code "{code}" already exists'},
status=400)
if GradeLevel.objects.filter(name__iexact=name).exists():
    return JsonResponse({'error': f'A grade level named "{name}" already exists'},
status=400)
gl = GradeLevel.objects.create(code=code, name=name, description=description,
is_active=is_active)
log_activity(user=request.user, action='settings_changed',
description=f'Added grade level "{name}" ({code})', request=request)
return JsonResponse({'message': 'Grade level added successfully',
'grade_level': {'id': gl.id, 'code': gl.code, 'name': gl.name,
'description': gl.description or "", 'is_active': gl.is_active}},
status=201)
except json.JSONDecodeError:
    return JsonResponse({'error': 'Invalid JSON data'}, status=400)
except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

@login_required

@require_http_methods(["PUT"])

def update_grade_level(request, grade_level_id):

```

try:
    from admin_app.models import GradeLevel
    try:
        gl = GradeLevel.objects.get(pk=grade_level_id)
    except GradeLevel.DoesNotExist:
        return JsonResponse({'error': 'Grade level not found'}, status=404)
    payload = json.loads(request.body)
    code = (payload.get('code') or gl.code).strip().upper()
    name = (payload.get('name') or gl.name).strip()
    description = payload.get('description', gl.description) or ""
    is_active = bool(payload.get('is_active', gl.is_active))
    if not code:
        return JsonResponse({'error': 'Code is required'}, status=400)
    if not name:
        return JsonResponse({'error': 'Name is required'}, status=400)

```

```

        if GradeLevel.objects.filter(code__iexact=code).exclude(pk=grade_level_id).exists():
            return JsonResponse({'error': f'Another grade level with code "{code}" already exists'},
                                status=400)
        if GradeLevel.objects.filter(name__iexact=name).exclude(pk=grade_level_id).exists():
            return JsonResponse({'error': f'Another grade level named "{name}" already exists'},
                                status=400)
        old = f'{gl.name} ({gl.code})'
        gl.code, gl.name, gl.description, gl.is_active = code, name, description, is_active
        gl.save()
        log_activity(user=request.user, action='settings_changed',
                    description=f'Updated grade level: {old} → {gl.name} ({gl.code})', request=request)
        return JsonResponse({'message': 'Grade level updated successfully',
                            'grade_level': {'id': gl.id, 'code': gl.code, 'name': gl.name,
                                            'description': gl.description or "", 'is_active': gl.is_active}},
                            status=200)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON data'}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

@login_required

@require_http_methods(["DELETE"])

def delete_grade_level(request, grade_level_id):

```

    try:
        from admin_app.models import GradeLevel
        try:
            gl = GradeLevel.objects.get(pk=grade_level_id)
        except GradeLevel.DoesNotExist:
            return JsonResponse({'error': 'Grade level not found'}, status=404)
        from enrollment_app.models import Student
        student_count = Student.objects.filter(grade_level=gl).count()
        section_count = gl.sections.count() if hasattr(gl, 'sections') else 0
        if student_count or section_count:
            return JsonResponse({
                'error': f'Cannot delete: {student_count} student(s) and {section_count} section(s) still
reference this grade level.'
            }, status=400)
        label = f'{gl.name} ({gl.code})'
        gl.delete()
        log_activity(user=request.user, action='settings_changed',
                    description=f'Deleted grade level "{label}"', request=request)
        return JsonResponse({'message': 'Grade level deleted successfully'}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

# ===== TEACHER MANAGEMENT =====

@login_required
@require_http_methods(["GET"])
def get_teachers_for_settings(request):
    try:
        # Get the active school year
        active_school_year = SchoolYear.objects.filter(is_active=True).first()

        # Get teacher IDs that are advisers of sections in the CURRENT school year only
        if active_school_year:
            active_adviser_ids = set(
                Section.objects.filter(
                    school_year=active_school_year,
                    adviser__isnull=False
                ).values_list('adviser_id', flat=True)
            )
        else:
            active_adviser_ids = set()

        teachers = Teacher.objects.select_related(
            'position', 'department').all().order_by('last_name', 'first_name')
        data = []
        for t in teachers:
            # is_adviser is True only if assigned to a section in the ACTIVE school year
            is_adviser_this_year = t.id in active_adviser_ids
            data.append({
                'id': t.id,
                'first_name': t.first_name,
                'middle_name': t.middle_name or "",
                'last_name': t.last_name,
                'full_name': t.get_full_name(),
                'email': t.email,
                'position_id': t.position_id,
                'position_name': t.position.name if t.position else "",
                'department_id': t.department_id,
                'department_name': t.department.name if t.department else "",
                'address': t.address or "",
                'is_adviser': is_adviser_this_year,
                'created_at': t.created_at.strftime('%b %d, %Y') if t.created_at else "",
            })
        return JsonResponse({'teachers': data, 'data': data}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required

```

```

@require_http_methods(["POST"])
def add_teacher(request):
    try:
        data = json.loads(request.body)
        first_name = (data.get('first_name') or "").strip()
        middle_name = (data.get('middle_name') or "").strip() or None
        last_name = (data.get('last_name') or "").strip()
        email = (data.get('email') or "").strip().lower()
        address = (data.get('address') or "").strip() or None
        position_id = data.get('position_id')
        department_id = data.get('department_id')
        if not first_name:
            return JsonResponse({'error': 'First name is required'}, status=400)
        if not last_name:
            return JsonResponse({'error': 'Last name is required'}, status=400)
        if not email:
            return JsonResponse({'error': 'Email is required'}, status=400)
        if Teacher.objects.filter(email__iexact=email).exists():
            return JsonResponse({'error': 'A teacher with this email already exists'}, status=400)
        position = None
        department = None
        if position_id:
            try:
                position = Position.objects.get(pk=position_id)
            except Position.DoesNotExist:
                return JsonResponse({'error': 'Invalid position'}, status=400)
        if department_id:
            try:
                department = Department.objects.get(pk=department_id)
            except Department.DoesNotExist:
                return JsonResponse({'error': 'Invalid department'}, status=400)
        teacher = Teacher.objects.create(
            first_name=first_name, middle_name=middle_name, last_name=last_name,
            email=email, address=address, position=position, department=department
        )
        log_activity(user=request.user, action='teacher_added',
            description=f'Added teacher: {teacher.get_full_name()}', request=request)
        return JsonResponse({'message': 'Teacher added successfully',
            'teacher': {'id': teacher.id, 'full_name': teacher.get_full_name(),
                'email': teacher.email}}, status=201)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON data'}, status=400)
    except ValidationError as ve:
        error_messages = (
            '; '.join([f'{field}: {', '.join(msgs)}' for field, msgs in ve.message_dict.items()])
            if hasattr(ve, 'message_dict') else str(ve)
        )

```

```

        return JsonResponse({'error': error_messages}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["PUT"])
def update_teacher(request, teacher_id):
    try:
        teacher = Teacher.objects.get(pk=teacher_id)
        data = json.loads(request.body)
        first_name = (data.get('first_name') or "").strip()
        middle_name = (data.get('middle_name') or "").strip() or None
        last_name = (data.get('last_name') or "").strip()
        email = (data.get('email') or "").strip().lower()
        address = (data.get('address') or "").strip() or None
        position_id = data.get('position_id')
        department_id = data.get('department_id')
        if not first_name:
            return JsonResponse({'error': 'First name is required'}, status=400)
        if not last_name:
            return JsonResponse({'error': 'Last name is required'}, status=400)
        if not email:
            return JsonResponse({'error': 'Email is required'}, status=400)
        if Teacher.objects.filter(email__iexact=email).exclude(pk=teacher_id).exists():
            return JsonResponse({'error': 'Another teacher with this email already exists'},
status=400)
        position = None
        department = None
        if position_id:
            try:
                position = Position.objects.get(pk=position_id)
            except Position.DoesNotExist:
                return JsonResponse({'error': 'Invalid position'}, status=400)
        if department_id:
            try:
                department = Department.objects.get(pk=department_id)
            except Department.DoesNotExist:
                return JsonResponse({'error': 'Invalid department'}, status=400)
        old_name = teacher.get_full_name()
        teacher.first_name = first_name
        teacher.middle_name = middle_name
        teacher.last_name = last_name
        teacher.email = email
        teacher.address = address
        teacher.position = position
        teacher.department = department

```

```

        teacher.save()
        log_activity(user=request.user, action='teacher_updated',
                    description=f'Updated teacher: {old_name} -> {teacher.get_full_name()}',
                    request=request)
        return JsonResponse({'message': 'Teacher updated successfully',
                            'teacher': {'id': teacher.id, 'full_name': teacher.get_full_name(),
                                        'email': teacher.email}}, status=200)
    except Teacher.DoesNotExist:
        return JsonResponse({'error': 'Teacher not found'}, status=404)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON data'}, status=400)
    except ValidationError as ve:
        error_messages = (
            '; '.join([f'{field}: {' '.join(msgs)}' for field, msgs in ve.message_dict.items()])
            if hasattr(ve, 'message_dict') else str(ve)
        )
        return JsonResponse({'error': error_messages}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

@login_required
@require_http_methods(["DELETE"])
def delete_teacher(request, teacher_id):
    try:
        teacher = Teacher.objects.get(pk=teacher_id)
        teacher_name = teacher.get_full_name()
        # Only block deletion if assigned as adviser in the ACTIVE school year
        active_school_year = SchoolYear.objects.filter(is_active=True).first()
        if active_school_year:
            is_active_adviser = Section.objects.filter(
                adviser=teacher,
                school_year=active_school_year
            ).exists()
        else:
            is_active_adviser = Section.objects.filter(adviser=teacher).exists()
        if is_active_adviser:
            return JsonResponse({'error': 'Cannot delete teacher who is assigned as a section adviser in the current school year'}, status=400)
        teacher.delete()
        log_activity(user=request.user, action='teacher_deleted',
                    description=f'Deleted teacher: {teacher_name}', request=request)
        return JsonResponse({'message': 'Teacher deleted successfully'}, status=200)
    except Teacher.DoesNotExist:
        return JsonResponse({'error': 'Teacher not found'}, status=404)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["POST"])
def delete_content_image(request):
    try:
        setting_type = request.POST.get('setting_type')
        if not setting_type:
            return JsonResponse({'error': 'Setting type is required'}, status=400)
        try:
            setting = SystemSettings.objects.get(setting_type=setting_type)
            if setting.image:
                storage = setting.image.storage
                path = setting.image.name
                setting.image = None
                setting.updated_by = request.user
                setting.save()
                if storage.exists(path):
                    storage.delete(path)
            return JsonResponse({'message': 'Image removed successfully'}, status=200)
        except SystemSettings.DoesNotExist:
            return JsonResponse({'message': 'Nothing to delete'}, status=200)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)

```

```

@login_required
@require_http_methods(["POST"])
def generate_promotion_statuses(request):
    """
    Admin endpoint to manually trigger StudentAcademicYearStatus generation
    for a specific school year.
    POST body: { school_year_id: <id>, overwrite: true/false }
    """
    try:
        data = json.loads(request.body)
        school_year_id = data.get('school_year_id')
        overwrite = data.get('overwrite', False)

        if school_year_id:
            school_year = get_object_or_404(SchoolYear, pk=school_year_id)
        else:
            school_year = SchoolYear.objects.filter(
                is_active=False
            ).order_by('-year_label').first()
            if not school_year:

```

```
        return JsonResponse({'success': False, 'error': 'No inactive school year found'},
status=400)
```

```
    from django.core.management import call_command
    from io import StringIO
    out = StringIO()
    call_command(
        'generate_academic_statuses',
        school_year=school_year.year_label,
        **({'overwrite': True}) if overwrite else {},
        stdout=out,
    )

    return JsonResponse({
        'success': True,
        'school_year': school_year.year_label,
        'message': f'Promotion statuses generated for {school_year.year_label}.',
        'details': out.getvalue(),
    })
except Exception as e:
    import traceback
    traceback.print_exc()
    return JsonResponse({'success': False, 'error': str(e)}, status=500)
```

```
@admin_required
```

```
@require_http_methods(["POST"])
```

```
def generate_promotion_statuses(request):
```

```
    """
```

Admin-only trigger to generate StudentAcademicYearStatus records
for all students in a given school year.

Logic per student per grade level:

- Skip if StudentAcademicYearStatus already exists
- Use quarter=5 Final grades if available
- Otherwise auto-calculate from Q1+Q2+Q3+Q4 (all 4 required)
- Skip if any quarter is missing for any subject

```
    """
```

```
try:
```

```
    data = json.loads(request.body)
```

```
    school_year_id = data.get('school_year_id')
```

```
except Exception:
```

```
    return JsonResponse({'success': False, 'error': 'Invalid JSON.'}, status=400)
```

```
if not school_year_id:
```

```
    return JsonResponse({'success': False, 'error': 'school_year_id is required.'}, status=400)
```



```
try:
    school_year = SchoolYear.objects.get(id=school_year_id)
except SchoolYear.DoesNotExist:
    return JsonResponse({'success': False, 'error': 'School year not found.'}, status=404)
```

```
# Get all students enrolled in this school year
enrollments = (
    StudentEnrollment.objects
    .filter(school_year=school_year)
    .select_related('student', 'grade_level', 'student__program_selection',
                    'student__program_selection__assigned_section',
                    'student__program_selection__assigned_section__adviser',
                    'student__program_selection__assigned_section__program')
)
```

```
promoted = 0
retained = 0
skipped = 0
already_exists = 0
errors = []
```

```
for enrollment in enrollments:
    student = enrollment.student
    grade_level = enrollment.grade_level
```

```
if not grade_level:
    skipped += 1
    continue
```

```
# Skip if record already exists
if StudentAcademicYearStatus.objects.filter(
    student=student,
    school_year=school_year,
).exists():
    already_exists += 1
    continue
```

```
# Get all AcademicPerformance for this student + school year + grade level
performances = AcademicPerformance.objects.filter(
    student=student,
    school_year=school_year,
    grade_level=grade_level,
).select_related('subject')
```

```
if not performances.exists():
    skipped += 1
```

```

        continue

# Group by subject
subject_data = {}
for perf in performances:
    subj_id = perf.subject_id
    if subj_id not in subject_data:
        subject_data[subj_id] = {
            'subject': perf.subject,
            'q1': None, 'q2': None,
            'q3': None, 'q4': None,
            'final': None,
        }
    q_map = {1: 'q1', 2: 'q2', 3: 'q3', 4: 'q4', 5: 'final'}
    field = q_map.get(perf.quarter)
    if field:
        subject_data[subj_id][field] = perf.grade

if not subject_data:
    skipped += 1
    continue

# Compute final rating per subject
final_ratings = []
skip_student = False

for subj_id, s in subject_data.items():
    if s['final'] is not None:
        # Use uploaded Final grade
        final_ratings.append(s['final'])
    else:
        # Require ALL 4 quarters
        q_grades = [s['q1'], s['q2'], s['q3'], s['q4']]
        if any(g is None for g in q_grades):
            # Missing at least one quarter — skip this student
            skip_student = True
            break
        auto_final = round(sum(q_grades) / 4, 2)
        final_ratings.append(auto_final)

if skip_student or not final_ratings:
    skipped += 1
    continue

# Compute overall grade
overall_grade = round(sum(final_ratings) / len(final_ratings), 2)

```

```

# Determine final status
has_failed = any(g < 75 for g in final_ratings)
final_status = 'retained' if has_failed else 'promoted'

# Get assigned section
assigned_section = None
try:
    ps = student.program_selection
    if ps and ps.assigned_section:
        assigned_section = ps.assigned_section
except Exception:
    pass

try:
    StudentAcademicYearStatus.objects.create(
        student=student,
        school_year=school_year,
        grade_level=grade_level,
        section=assigned_section,
        final_status=final_status,
        overall_grade=overall_grade,
        remarks='Auto-generated by admin trigger.',
    )

    if final_status == 'promoted':
        promoted += 1
    else:
        retained += 1

# Also run probation check for STE students
if (assigned_section and assigned_section.program
    and assigned_section.program.code == 'STE'):
    _check_probation_for_admin_trigger(
        student, assigned_section, school_year, grade_level, subject_data
    )

except Exception as e:
    errors.append(f'{student.lrn}: {str(e)}')
    skipped += 1

return JsonResponse({
    'success': True,
    'promoted': promoted,
    'retained': retained,
    'skipped': skipped,
    'already_exists': already_exists,
    'errors': errors[:10],
})

```

```

        'message': (
            f'{promoted} promoted, {retained} retained, '
            f'{already_exists} already had records, {skipped} skipped.'
        )
    })

```

```

def _check_probation_for_admin_trigger(student, section, school_year, grade_level,
subject_data):

```

```

    """

```

```

    Probation check used by the admin trigger.

```

```

    Uses pre-computed subject_data dict instead of re-querying.

```

```

    """

```

```

    from coordinator_app.models import ProbationRecord

```

```

    already_exists = ProbationRecord.objects.filter(
        student=student,
        school_year=school_year,
        grade_level=grade_level,
        is_active=True,
    ).exists()

```

```

    if already_exists:
        return

```

```

    breached = []
    for subj_id, s in subject_data.items():
        subject = s['subject']
        if not getattr(subject, 'is_threshold_subject', False):
            continue

```

```

        # Get final rating (uploaded or auto-calculated)

```

```

        if s['final'] is not None:

```

```

            final_rating = s['final']

```

```

        else:

```

```

            q_grades = [s['q1'], s['q2'], s['q3'], s['q4']]

```

```

            if any(g is None for g in q_grades):

```

```

                continue

```

```

            final_rating = round(sum(q_grades) / 4, 2)

```

```

        if final_rating < 83:

```

```

            breached.append(

```

```

                f'{subject.name} Final grade: {final_rating} '

```

```

                f'(below 83 retention threshold)'

```

```

            )

```

```

    if not breached:

```

```
return
```

```
reason = "STE retention threshold breached. " + "; ".join(breached) + "."
```

```
ProbationRecord.objects.create(  
    student=student,  
    school_year=school_year,  
    grade_level=grade_level,  
    previous_program=section.program.code,  
    moved_to_program='REGULAR',  
    reason=reason,  
    is_active=True,  
)
```

batch_upload_views.py – Bulk data import (Section data upload, and Student import)

```
from django.shortcuts import render, redirect  
from django.contrib.auth.decorators import login_required  
from django.contrib import messages  
from django.http import HttpResponseRedirect, JsonResponse  
from django.db import transaction  
from django.core.exceptions import ValidationError  
from decimal import Decimal, InvalidOperation  
from admin_app.models import Teacher, Position, Department  
import openpyxl  
from openpyxl.styles import Font, PatternFill, Alignment  
import csv  
import io  
import pandas as pd  
from datetime import datetime
```

```
@login_required
```

```
def download_teacher_template(request):  
    """Download Excel template for batch teacher upload"""
```

```
    try:  
        # Create Excel workbook  
        wb = openpyxl.Workbook()  
        ws = wb.active  
        ws.title = "Teacher Template"
```

```
    # Add headers
```

```
    headers = [  
        'first_name',  
        'middle_name',  
        'last_name',  
        'email',  
        'position_id',  
        'department_id',  
        'address'
```

```

]
ws.append(headers)

# Style headers
header_fill = PatternFill(
    start_color='991B1B', end_color='991B1B', fill_type='solid')
header_font = Font(bold=True, color='FFFFFF')

for col_num, header in enumerate(headers, 1):
    cell = ws.cell(row=1, column=col_num)
    cell.value = header
    cell.font = header_font
    cell.fill = header_fill
    cell.alignment = Alignment(horizontal='center')
    ws.column_dimensions[cell.column_letter].width = max(
        len(header) + 4, 15)

# Add sample data row
sample_row = [
    'John',
    'M',
    'Doe',
    'john.doe@example.com',
    '1', # position_id (you'll need to replace with actual IDs)
    '1', # department_id
    'Sample Address'
]
ws.append(sample_row)

# Add second sheet with available positions and departments
ws2 = wb.create_sheet("Reference Data")
ws2.append(["Available Positions", "Department IDs"])

# Add positions
positions = Position.objects.filter(
    is_active=True).values_list('id', 'name')
for pos_id, pos_name in positions:
    ws2.append([f'{pos_id}: {pos_name}', ""])

ws2.append([])
ws2.append(["Available Departments"])
departments = Department.objects.filter(
    is_active=True).values_list('id', 'name')
for dept_id, dept_name in departments:
    ws2.append([f'{dept_id}: {dept_name}', ""])

response = HttpResponse(

```

```

        content_type='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
    )
    response['Content-Disposition'] = f'attachment;
filename="teacher_template_{datetime.now().strftime("%Y%m%d")}.xlsx"'
    wb.save(response)

    return response

except Exception as e:
    messages.error(request, f'Error generating template: {str(e)}')
    return redirect('admin_app:settings')

@login_required
def batch_upload_teachers(request):
    """Handle batch upload of teachers via Excel/CSV"""
    if request.method != 'POST':
        return JsonResponse({'error': 'Method not allowed'}, status=405)

    try:
        # Check if file was uploaded
        if 'upload_file' not in request.FILES:
            return JsonResponse({'success': False, 'message': 'No file uploaded'})

        uploaded_file = request.FILES['upload_file']
        print(
            f'File received: {uploaded_file.name}, size: {uploaded_file.size}')

        # Parse file based on extension
        file_name = uploaded_file.name.lower()
        try:
            if file_name.endswith('.csv'):
                df = pd.read_csv(uploaded_file)
            elif file_name.endswith(('.xlsx', '.xls')):
                df = pd.read_excel(uploaded_file)
            else:
                return JsonResponse({'success': False, 'message': 'Invalid file format. Please upload
.xlsx, .xls, or .csv file'})
        except Exception as e:
            return JsonResponse({'success': False, 'message': f'Error reading file: {str(e)}'})

        print(f'DataFrame loaded: {len(df)} rows')
        print(f'Columns found: {list(df.columns)}')

        # Validate required columns
        required_columns = ['first_name', 'last_name', 'email']
        missing_columns = [
            col for col in required_columns if col not in df.columns]

```

```

if missing_columns:
    return JsonResponse({
        'success': False,
        'message': f'Missing required columns: {", ".join(missing_columns)}. Found columns: {", ".join(list(df.columns))}'
    })

```

```

# Process records

```

```

results = {
    'total': len(df),
    'success': 0,
    'failed': 0,
    'errors': []
}

```

```

for index, row in df.iterrows():

```

```

    try:
        # Extract data with defaults
        first_name = str(row.get('first_name', "")).strip(
        ) if pd.notna(row.get('first_name')) else ""
        middle_name = str(row.get('middle_name', "")).strip(
        ) if pd.notna(row.get('middle_name')) else None
        last_name = str(row.get('last_name', "")).strip(
        ) if pd.notna(row.get('last_name')) else ""
        email = str(row.get('email', "")).strip(
        ).lower() if pd.notna(row.get('email')) else ""
        address = str(row.get('address', "")).strip(
        ) if pd.notna(row.get('address')) else None

```

```

        # Handle position_id

```

```

        position_id = None
        if pd.notna(row.get('position_id')):
            try:
                pos_str = str(row.get('position_id')).strip()
                if ':' in pos_str:
                    position_id = int(pos_str.split(':')[0].strip())
                else:
                    position_id = int(float(pos_str))
            except (ValueError, TypeError):
                # Skip position if invalid
                pass

```

```

        # Handle department_id

```

```

        department_id = None
        if pd.notna(row.get('department_id')):
            try:

```



```

        dept_str = str(row.get('department_id')).strip()
        if ':' in dept_str:
            department_id = int(dept_str.split(':')[0].strip())
        else:
            department_id = int(float(dept_str))
    except (ValueError, TypeError):
        # Skip department if invalid
        pass

# Validate required fields
if not first_name:
    raise ValidationError("First name is required")
if not last_name:
    raise ValidationError("Last name is required")
if not email:
    raise ValidationError("Email is required")

# Check email uniqueness
if Teacher.objects.filter(email__iexact=email).exists():
    raise ValidationError(
        f"Teacher with email '{email}' already exists")

# Get position and department objects if IDs provided
position = None
if position_id:
    try:
        position = Position.objects.get(
            id=position_id, is_active=True)
    except Position.DoesNotExist:
        results['errors'].append(
            f"Row {index + 2}: Position ID {position_id} not found - continuing without
position")

department = None
if department_id:
    try:
        department = Department.objects.get(
            id=department_id, is_active=True)
    except Department.DoesNotExist:
        results['errors'].append(
            f"Row {index + 2}: Department ID {department_id} not found - continuing
without department")

# Create teacher
teacher = Teacher.objects.create(
    first_name=first_name,
    middle_name=middle_name,

```

```

        last_name=last_name,
        email=email,
        address=address,
        position=position,
        department=department
    )

    print(
        f"Created teacher: {teacher.id} - {teacher.get_full_name()}"
    )
    results['success'] += 1

except Exception as e:
    results['failed'] += 1
    error_msg = f"Row {index + 2}: {str(e)}"
    results['errors'].append(error_msg)
    print(f"Error: {error_msg}")

# Prepare response message
if results['failed'] == 0:
    message = f'Successfully uploaded {results["success"]} teacher(s)!'
    return JsonResponse({
        'success': True,
        'message': message,
        'data': results
    })
elif results['success'] > 0:
    message = f'Uploaded {results["success"]} teacher(s) with {results["failed"]} error(s).'
    return JsonResponse({
        'success': True,
        'message': message,
        'data': results
    })
else:
    message = f'Upload failed. {results["failed"]} error(s).'
    return JsonResponse({
        'success': False,
        'message': message,
        'data': results
    })

except Exception as e:
    print(f"Batch upload error: {str(e)}")
    import traceback
    traceback.print_exc()
    return JsonResponse({
        'success': False,
        'message': f'Error processing file: {str(e)}',

```

```

        'data': {'errors': [str(e)]}
    })

```

analytics_views.py – System analytics and reports

```

from django.shortcuts import render
from django.http import JsonResponse
from django.db.models import Count, Avg, Q, F
from django.db.models.functions import TruncMonth
from django.contrib.auth.decorators import login_required
from admin_app.decorators import admin_required
from admin_app.models import SchoolYear, UserProfile, Section, GradeLevel, Program
from enrollment_app.models import (
    Student, StudentEnrollment, StudentData, ProgramSelection,
    StudentDocumentSubmission, AcademicData
)
from coordinator_app.models import (
    AcademicPerformance, CoordinatorActivityLog,
    Qualified_for_ste, ProbationRecord
)
from datetime import timedelta
from django.utils import timezone

def _get_active_sy():
    return SchoolYear.objects.filter(is_active=True).first()

def _safe_pct(part, total):
    return round((part / total * 100), 1) if total > 0 else 0

#
=====
====
# ANALYTICS VIEW
#
=====
====

@admin_required
def analytics(request):
    active_sy = SchoolYear.objects.filter(is_active=True).first()
    try:
        user_profile = UserProfile.objects.select_related(
            'program', 'position', 'department'
        ).get(user=request.user)
    except UserProfile.DoesNotExist:
        user_profile = None

    context = {

```

```

        'user': request.user,
        'user_profile': user_profile,
        'active_page': 'analytics',
        'active_school_year': active_sy,
    }
    return render(request, 'admin_app/analytics.html', context)

#
=====
=====
# MAIN ANALYTICS DATA API
#
=====
=====

@admin_required
def analytics_data(request):
    """
    Returns all analytics data for the dashboard.
    Accepts:
    ?grade=G7|G8|G9|G10
    ?program=REGULAR|STE|SPTVE|SPFL
    ?enrollee=new|continuing|transferee|returnee
    """
    grade_filter = request.GET.get('grade', None)
    program_filter = request.GET.get('program', None)
    enrollee_filter = request.GET.get('enrollee', None)

    sy = _get_active_sy()

    # — Base enrollment queryset
    

---


    enrollments = StudentEnrollment.objects.filter(school_year=sy) if sy else
    StudentEnrollment.objects.none()
    if grade_filter:
        enrollments = enrollments.filter(grade_level__code=grade_filter)
    if enrollee_filter:
        enrollments = enrollments.filter(enrollee_type=enrollee_filter)

    # Program filter needs a join through ProgramSelection
    if program_filter:
        lrns_with_program = ProgramSelection.objects.filter(
            school_year=sy,
            selected_program_code=program_filter
        ).values_list('student__lrn', flat=True)
        enrollments = enrollments.filter(student__lrn__in=lrns_with_program)

```

```

total = enrollments.count()
approved = enrollments.filter(enrollment_status='approved').count()
pending = enrollments.filter(enrollment_status__in=['submitted','under_review']).count()
rejected = enrollments.filter(enrollment_status='rejected').count()
sections = Section.objects.filter(school_year=sy).count() if sy else 0

```

```

approval_rate = _safe_pct(approved, total)

```

— Funnel

```

funnel = {
    'labels': ['Submitted','Under Review','Approved','Rejected'],
    'data': [
        enrollments.filter(enrollment_status='submitted').count(),
        enrollments.filter(enrollment_status='under_review').count(),
        approved, rejected,
    ],
    'colors': ['#3b82f6','#59e0b','#10b981','#ef4444'],
}

```

— Enrollee types

```

enrollee_types = {
    'new': enrollments.filter(enrollee_type='new').count(),
    'continuing': enrollments.filter(enrollee_type='continuing').count(),
    'transferee': enrollments.filter(enrollee_type='transferee').count(),
    'returnee': enrollments.filter(enrollee_type='returnee').count(),
}

```

— Program distribution

```

student_lrn = enrollments.values_list('student__lrn', flat=True)
prog_qs = (
    ProgramSelection.objects
    .filter(school_year=sy, student__lrn__in=student_lrn)
    .values('selected_program_code')
    .annotate(count=Count('student'))
    .order_by('-count')
)
program_dist = {
    'labels': [p['selected_program_code'] or 'Unknown' for p in prog_qs],
    'data': [p['count'] for p in prog_qs],
    'colors': ['#991b1b','#10b981','#3b82f6','#59e0b','#8b5cf6','#ef4444','#6b7280','#14b8a6'],
}

```

— Grade level distribution (only when no grade filter) —————

```

grade_dist = None

```

```

if not grade_filter:
    grade_qs = (
        enrollments
        .values('grade_level__name', 'grade_level__code')
        .annotate(count=Count('student'))
        .order_by('grade_level__code')
    )
    grade_dist = {
        'labels': [g['grade_level__name'] or 'Unknown' for g in grade_qs],
        'data': [g['count'] for g in grade_qs],
        'colors': ['#991b1b', '#3b82f6', '#10b981', '#f59e0b'],
    }

```

— Gender

```

gender_qs = (
    StudentData.objects
    .filter(student__ltn__in=student_ltns)
    .values('gender')
    .annotate(count=Count('student'))
)
gmap = {g['gender']: g['count'] for g in gender_qs}
gender_dist = {
    'male': gmap.get('male', 0),
    'female': gmap.get('female', 0),
    'other': gmap.get('other', 0),
}

```

— Monthly trend

```

nine_months_ago = timezone.now() - timedelta(days=270)
monthly_qs = (
    StudentEnrollment.objects
    .filter(school_year=sy, created_at__gte=nine_months_ago)
    .annotate(month=TruncMonth('created_at'))
    .values('month')
    .annotate(
        total=Count('id'),
        approved_count=Count('id', filter=Q(enrollment_status='approved'))
    )
    .order_by('month')
)
if grade_filter:
    monthly_qs = monthly_qs.filter(grade_level__code=grade_filter)
if enrollee_filter:
    monthly_qs = monthly_qs.filter(enrollee_type=enrollee_filter)

```

```

trend = {
    'labels': [m['month'].strftime('%b') for m in monthly_qs],
    'total': [m['total'] for m in monthly_qs],
    'approved': [m['approved_count'] for m in monthly_qs],
}

```

— Section capacity

```

sec_qs = Section.objects.filter(school_year=sy).select_related('program','grade_level')
if grade_filter:
    sec_qs = sec_qs.filter(grade_level__code=grade_filter)
if program_filter:
    sec_qs = sec_qs.filter(program__code=program_filter)

section_capacity = []
for sec in sec_qs.order_by('grade_level__code','name')[1:12]:
    actual = sec.get_actual_count()
    pct = _safe_pct(actual, sec.max_students)
    section_capacity.append({
        'name': f'{sec.grade_level.name if sec.grade_level else ""} — {sec.name}',
        'current': actual,
        'max': sec.max_students,
        'pct': pct,
        'status': 'full' if pct >= 95 else 'almost' if pct >= 80 else 'ok',
        'track': sec.regular_track or "",
    })

```

— Document completion

```

docs_qs = StudentDocumentSubmission.objects.filter(student__lrn__in=student_lrns)
doc_stats = {
    'labels': ['Approved','Pending','Rejected'],
    'data': [
        docs_qs.filter(status='approved').values('student').distinct().count(),
        docs_qs.filter(status='pending').values('student').distinct().count(),
        docs_qs.filter(status='rejected').values('student').distinct().count(),
    ],
    'colors': ['#10b981','#f59e0b','#ef4444'],
}

```

— Academic performance

```

acad_qs = (
    AcademicPerformance.objects
        .filter(student__lrn__in=student_lrns, quarter=5)
        .values('grade_level__name','grade_level__code')
        .annotate(avg_grade=Avg('grade'))
)

```

```

        .order_by('grade_level__code')
    )
    acad_perf = {
        'labels': [a['grade_level__name'] or 'Unknown' for a in acad_qs],
        'data': [round(float(a['avg_grade']), 2) if a['avg_grade'] else 0 for a in acad_qs],
    }

# — Probation


---


    prob_qs = ProbationRecord.objects.filter(school_year=sy, is_active=True)
    if grade_filter:
        prob_qs = prob_qs.filter(grade_level__code=grade_filter)
    prob_by_prog = prob_qs.values('previous_program').annotate(count=Count('id')).order_by('-count')
    probation = {
        'total': prob_qs.count(),
        'labels': [p['previous_program'] for p in prob_by_prog],
        'data': [p['count'] for p in prob_by_prog],
    }

# — STE qualification


---


    ste_qs = Qualified_for_ste.objects.filter(school_year=sy)
    if grade_filter:
        ste_qs = ste_qs.filter(grade_level__code=grade_filter)
    ste = {
        'qualified': ste_qs.filter(status='qualified').count(),
        'not_qualified': ste_qs.filter(status='not_qualified').count(),
        'pending': ste_qs.filter(status='pending').count(),
        'waitlisted': ste_qs.filter(status='waitlisted').count(),
        'total': ste_qs.count(),
    }

# — Coordinator activity (last 30 days)


---


    thirty_ago = timezone.now() - timedelta(days=30)
    coord_qs = (
        CoordinatorActivityLog.objects
        .filter(created_at__gte=thirty_ago)
        .values('action')
        .annotate(count=Count('id'))
        .order_by('-count')[:6]
    )
    coord_activity = {
        'labels': [c['action'].replace('_', ' ').title() for c in coord_qs],
        'data': [c['count'] for c in coord_qs],
        'colors': ['#10b981', '#3b82f6', '#f59e0b', '#8b5cf6', '#ef4444', '#6b7280'],
    }

```


— AI acceptance rate

```
ai_total = ProgramSelection.objects.filter(school_year=sy,
requires_program_selection=True).count()
ai_approved = ProgramSelection.objects.filter(school_year=sy,
requires_program_selection=True, admin_approved=True).count()
ai_rate = _safe_pct(ai_approved, ai_total)
```

— Auto insights

```
insights = []
if approval_rate >= 75:
    insights.append({'icon': 'fa-arrow-trend-up', 'color': '#10b981', 'bg': '#f0fdf4',
                    'text': f'Approval rate is {approval_rate}% — above the 75% target.'})
else:
    insights.append({'icon': 'fa-exclamation-triangle', 'color': '#f59e0b', 'bg': '#fffbeb',
                    'text': f'Approval rate is {approval_rate}% — below the 75% target.'})
if pending > 0:
    insights.append({'icon': 'fa-hourglass-half', 'color': '#3b82f6', 'bg': '#eff6ff',
                    'text': f'{pending} student{"s" if pending>1 else ""} pending coordinator review.'})
if probation['total'] > 0:
    insights.append({'icon': 'fa-exclamation-circle', 'color': '#ef4444', 'bg': '#fef2f2',
                    'text': f'{probation["total"]} student{"s" if probation["total"]>1 else ""} on academic
probation.'})
full_secs = [s for s in section_capacity if s['status']=='full']
if full_secs:
    insights.append({'icon': 'fa-lock', 'color': '#8b5cf6', 'bg': '#f5f3ff',
                    'text': f'{len(full_secs)} section{"s are" if len(full_secs)>1 else " is"} at full
capacity.'})
if ste['total'] > 0:
    insights.append({'icon': 'fa-flask', 'color': '#14b8a6', 'bg': '#f0fdfa',
                    'text': f'{ste["qualified"]} of {ste["total"]} STE applicants qualified.'})
if gender_dist['female'] > gender_dist['male']:
    diff = gender_dist['female'] - gender_dist['male']
    insights.append({'icon': 'fa-venus', 'color': '#ec4899', 'bg': '#fdf2f8',
                    'text': f'Female students outnumber males by {diff}.'})
elif gender_dist['male'] > gender_dist['female']:
    diff = gender_dist['male'] - gender_dist['female']
    insights.append({'icon': 'fa-mars', 'color': '#3b82f6', 'bg': '#eff6ff',
                    'text': f'Male students outnumber females by {diff}.'})
if ai_rate > 0:
    insights.append({'icon': 'fa-robot', 'color': '#f59e0b', 'bg': '#fffbeb',
                    'text': f'AI-recommended placements have a {ai_rate}% approval rate.'})

return JsonResponse({
    'sy_label': sy.year_label if sy else 'N/A',
```

```

'kpis': {
    'total': total, 'approved': approved, 'pending': pending,
    'rejected': rejected, 'sections': sections,
    'approval_rate': approval_rate, 'ai_rate': ai_rate,
},
'funnel': funnel,
'enrollee_types': enrollee_types,
'program_dist': program_dist,
'grade_dist': grade_dist,
'gender_dist': gender_dist,
'trend': trend,
'section_capacity': section_capacity,
'doc_stats': doc_stats,
'acad_perf': acad_perf,
'probation': probation,
'ste': ste,
'coord_activity': coord_activity,
'insights': insights[:6],
})

```

@admin_required

```
def analytics_header_data(request):
```

```
    active_school_year = SchoolYear.get_active_school_year()
```

```
    try:
```

```
        user_profile = UserProfile.objects.select_related(
            'program','position','department'
        ).get(user=request.user)
```

```
    except UserProfile.DoesNotExist:
```

```
        user_profile = None
```

```
    user = request.user
```

```
    full_name = f"{user.first_name} {user.last_name}" if user.first_name and user.last_name else
    user.username
```

```
    if user.first_name and user.last_name:
```

```
        initials = f"{user.first_name[0]}{user.last_name[0]}".upper()
```

```
    elif user.first_name:
```

```
        initials = user.first_name[0].upper()
```

```
    elif user.last_name:
```

```
        initials = user.last_name[0].upper()
```

```
    else:
```

```
        initials = user.username[0].upper() if user.username else "U"
```

```
    return JsonResponse({
```

```
        'school_year': active_school_year.year_label if active_school_year else 'No Active Year',
```

```
        'full_name': full_name,
```

```
        'role': user_profile.get_user_type_display() if user_profile else "Admin",
```

```
        'initials': initials,
```

```

        'photo_url': user_profile.photo.url if user_profile and user_profile.photo else None,
    })

#
=====
=====
# REPORTS / SETTINGS (unchanged)
#
=====
=====

@admin_required
def reports(request):
    try:
        user_profile = UserProfile.objects.select_related(
            'program','position','department'
        ).get(user=request.user)
    except UserProfile.DoesNotExist:
        user_profile = None
    return render(request, 'admin_app/reports.html', {'user': request.user, 'user_profile':
user_profile})

@admin_required
def reports_header_data(request):
    active_school_year = SchoolYear.get_active_school_year()
    try:
        user_profile = UserProfile.objects.select_related(
            'program','position','department'
        ).get(user=request.user)
    except UserProfile.DoesNotExist:
        user_profile = None
    user = request.user
    full_name = f'{user.first_name} {user.last_name}' if user.first_name and user.last_name else
user.username
    if user.first_name and user.last_name:
        initials = f'{user.first_name[0]}{user.last_name[0]}.upper()'
    elif user.first_name:
        initials = user.first_name[0].upper()
    elif user.last_name:
        initials = user.last_name[0].upper()
    else:
        initials = user.username[0].upper() if user.username else "U"
    return JsonResponse({
        'school_year': active_school_year.year_label if active_school_year else 'No Active Year',
        'full_name': full_name,
        'role': user_profile.get_user_type_display() if user_profile else "Admin",
        'initials': initials,

```

```

        'photo_url': user_profile.photo.url if user_profile and user_profile.photo else None,
    })

@admin_required
def settings(request):
    try:
        user_profile = UserProfile.objects.select_related(
            'program','position','department'
        ).get(user=request.user)
    except UserProfile.DoesNotExist:
        user_profile = None
    return render(request, 'admin_app/settings.html', {'user': request.user, 'user_profile':
user_profile})

@admin_required
def settings_header_data(request):
    active_school_year = SchoolYear.get_active_school_year()
    try:
        user_profile = UserProfile.objects.select_related(
            'program','position','department'
        ).get(user=request.user)
    except UserProfile.DoesNotExist:
        user_profile = None
    user = request.user
    full_name = f"{user.first_name} {user.last_name}" if user.first_name and user.last_name else
user.username
    if user.first_name and user.last_name:
        initials = f"{user.first_name[0]}{user.last_name[0]}.upper()"
    elif user.first_name:
        initials = user.first_name[0].upper()
    elif user.last_name:
        initials = user.last_name[0].upper()
    else:
        initials = user.username[0].upper() if user.username else "U"
    return JsonResponse({
        'school_year': active_school_year.year_label if active_school_year else 'No Active Year',
        'full_name': full_name,
        'role': user_profile.get_user_type_display() if user_profile else "Admin",
        'initials': initials,
        'photo_url': user_profile.photo.url if user_profile and user_profile.photo else None,
    })

```

archive_views.py – Data archival

```

from django.shortcuts import render
from django.http import JsonResponse, HttpResponse
from admin_app.decorators import admin_required
from admin_app.models import SchoolYear, Section, Program, GradeLevel

```

```

from enrollment_app.models import (
    Student, StudentEnrollment, StudentAcademicYearStatus, ProgramSelection
)
from coordinator_app.models import AcademicPerformance, ProbationRecord
import openpyxl
from openpyxl.styles import Font, Alignment, PatternFill
from django.db.models import Count, Avg, Q

```

```

@admin_required
def archive(request):
    """Main archive page — admin only."""
    active_school_year = SchoolYear.objects.filter(is_active=True).first()
    context = {
        'active_page': 'archive',
        'active_school_year': active_school_year,
    }
    return render(request, 'admin_app/archive.html', context)

```

```

@admin_required
def archive_school_years(request):
    """Returns all INACTIVE school years for the archive dropdown."""
    school_years = SchoolYear.objects.filter(
        is_active=False
    ).order_by('-year_label').values('id', 'year_label', 'start_date', 'end_date')

    return JsonResponse({
        'success': True,
        'school_years': [
            {
                'id': sy['id'],
                'year_label': sy['year_label'],
                'start_date': sy['start_date'].strftime('%b %d, %Y') if sy['start_date'] else "",
                'end_date': sy['end_date'].strftime('%b %d, %Y') if sy['end_date'] else "",
            }
            for sy in school_years
        ]
    })

```

```

@admin_required
def archive_enrollment(request):
    """Tab 1 — Enrollment summary for a given school year."""
    sy_id = request.GET.get('sy')
    if not sy_id:
        return JsonResponse({'success': False, 'error': 'sy parameter required.'}, status=400)

    try:
        school_year = SchoolYear.objects.get(id=sy_id, is_active=False)

```

```
except SchoolYear.DoesNotExist:
    return JsonResponse({'success': False, 'error': 'School year not found or is currently
active.'}, status=404)
```

```
enrollments = StudentEnrollment.objects.filter(school_year=school_year)
```

```
# Overall counts
```

```
total = enrollments.count()
```

```
by_type = {
```

```
    'new': enrollments.filter(enrollee_type='new').count(),
    'continuing': enrollments.filter(enrollee_type='continuing').count(),
    'transferee': enrollments.filter(enrollee_type='transferee').count(),
    'returnee': enrollments.filter(enrollee_type='returnee').count(),
```

```
}
```

```
by_status = {
```

```
    'approved': enrollments.filter(enrollment_status='approved').count(),
    'rejected': enrollments.filter(enrollment_status='rejected').count(),
    'pending': enrollments.filter(enrollment_status='pending').count(),
    'submitted': enrollments.filter(enrollment_status='submitted').count(),
```

```
}
```

```
# Per program breakdown
```

```
program_selections = ProgramSelection.objects.filter(
```

```
    school_year=school_year
```

```
).values('selected_program_code').annotate(
```

```
    total=Count('student'),
```

```
    approved=Count('student', filter=Q(admin_approved=True)),
```

```
    rejected=Count('student', filter=Q(admin_rejected=True)),
```

```
).order_by('selected_program_code')
```

```
programs = []
```

```
for p in program_selections:
```

```
    if p['selected_program_code']:
```

```
        programs.append({
```

```
            'code': p['selected_program_code'],
```

```
            'total': p['total'],
```

```
            'approved': p['approved'],
```

```
            'rejected': p['rejected'],
```

```
            'pending': p['total'] - p['approved'] - p['rejected'],
```

```
        })
```

```
return JsonResponse({
```

```
    'success': True,
```

```
    'school_year': school_year.year_label,
```

```
    'total': total,
```

```
    'by_type': by_type,
```

```
    'by_status': by_status,
```

```
        'by_program': programs,
    })
```

```
@admin_required
```

```
def archive_sections(request):
```

```
    """Tab 2 — Sections and their student masterlists."""
```

```
    sy_id = request.GET.get('sy')
```

```
    section_id = request.GET.get('section_id')
```

```
    if not sy_id:
```

```
        return JsonResponse({'success': False, 'error': 'sy parameter required.'}, status=400)
```

```
    try:
```

```
        school_year = SchoolYear.objects.get(id=sy_id, is_active=False)
```

```
    except SchoolYear.DoesNotExist:
```

```
        return JsonResponse({'success': False, 'error': 'School year not found.'}, status=404)
```

```
# If section_id provided — return full grade table for that section
```

```
if section_id:
```

```
    try:
```

```
        section = Section.objects.select_related(
```

```
            'program', 'grade_level', 'adviser'
```

```
        ).get(id=section_id, school_year=school_year)
```

```
    except Section.DoesNotExist:
```

```
        return JsonResponse({'success': False, 'error': 'Section not found.'}, status=404)
```

```
    program_selections = ProgramSelection.objects.filter(
```

```
        assigned_section=section,
```

```
        admin_approved=True,
```

```
    ).select_related('student', 'student__student_data').order_by(
```

```
        'student__student_data__last_name'
```

```
    )
```

```
# Get subjects for this section
```

```
from admin_app.models import Subject
```

```
from coordinator_app.views.coor_masterlist_views import _resolve_subject_program
```

```
subject_program = _resolve_subject_program(section)
```

```
subjects = list(Subject.objects.filter(
```

```
    program=subject_program, is_active=True
```

```
).order_by('name'))
```

```
students = []
```

```
for ps in program_selections:
```

```
    student = ps.student
```

```
    sd = getattr(student, 'student_data', None)
```

```
    if not sd:
```

```
        continue
```

```

# Get grades for this student
performances = AcademicPerformance.objects.filter(
    student=student,
    school_year=school_year,
    grade_level=section.grade_level,
).select_related('subject')

grade_map = {}
for perf in performances:
    subj_id = perf.subject_id
    if subj_id not in grade_map:
        grade_map[subj_id] = {
            'q1': None, 'q2': None,
            'q3': None, 'q4': None, 'final': None
        }
    q_map = {1:'q1', 2:'q2', 3:'q3', 4:'q4', 5:'final'}
    field = q_map.get(perf.quarter)
    if field:
        grade_map[subj_id][field] = float(perf.grade)

# Build subject grades list
subject_grades = []
final_ratings = []
for subj in subjects:
    g = grade_map.get(subj.id, {})
    q1 = g.get('q1')
    q2 = g.get('q2')
    q3 = g.get('q3')
    q4 = g.get('q4')
    final = g.get('final')
    if final is None and all(x is not None for x in [q1, q2, q3, q4]):
        final = round((q1 + q2 + q3 + q4) / 4, 2)
    if final is not None:
        final_ratings.append(final)
    subject_grades.append({
        'subject': subj.name,
        'q1': q1, 'q2': q2, 'q3': q3, 'q4': q4,
        'final': final,
        'status': ('Passed' if final >= 75 else 'Failed') if final is not None else None,
    })

overall = round(sum(final_ratings) / len(final_ratings), 2) if final_ratings else None

# Get promotion status
try:
    acad_status = StudentAcademicYearStatus.objects.get(

```



```

        student=student, school_year=school_year
    )
    promotion_status = acad_status.final_status
except StudentAcademicYearStatus.DoesNotExist:
    promotion_status = None

students.append({
    'lrn': student.lrn,
    'name': f"{sd.last_name}, {sd.first_name}" + sd.middle_name if sd.middle_name else
    "}",
    'gender': sd.gender,
    'subject_grades': subject_grades,
    'overall_grade': overall,
    'promotion_status': promotion_status,
})

return JsonResponse({
    'success': True,
    'section': {
        'id': section.id,
        'name': section.name,
        'program': section.program.code if section.program else "",
        'grade_level': section.grade_level.name if section.grade_level else "",
        'adviser': section.adviser.get_full_name() if section.adviser else '—',
    },
    'subjects': [s.name for s in subjects],
    'students': students,
})

# Otherwise — return list of all sections for this school year
sections = Section.objects.filter(
    school_year=school_year
).select_related('program', 'grade_level', 'adviser').order_by(
    'program__code', 'grade_level__code', 'name'
)

section_list = []
for s in sections:
    student_count = ProgramSelection.objects.filter(
        assigned_section=s, admin_approved=True
    ).count()
    section_list.append({
        'id': s.id,
        'name': s.name,
        'program': s.program.code if s.program else "",
        'grade_level': s.grade_level.name if s.grade_level else "",
        'adviser': s.adviser.get_full_name() if s.adviser else '—',

```

```

        'student_count': student_count,
        'regular_track': s.regular_track or "",
    })

    return JsonResponse({
        'success': True,
        'school_year': school_year.year_label,
        'sections': section_list,
    })

@admin_required
def archive_academic(request):
    """Tab 3 — Academic performance summary."""
    sy_id = request.GET.get('sy')
    if not sy_id:
        return JsonResponse({'success': False, 'error': 'sy parameter required.'}, status=400)

    try:
        school_year = SchoolYear.objects.get(id=sy_id, is_active=False)
    except SchoolYear.DoesNotExist:
        return JsonResponse({'success': False, 'error': 'School year not found.'}, status=404)

    statuses = StudentAcademicYearStatus.objects.filter(
        school_year=school_year
    ).select_related('student', 'student__student_data', 'section', 'section__program', 'grade_level')

    # Aggregate counts
    total = statuses.count()
    promoted = statuses.filter(final_status='promoted').count()
    retained = statuses.filter(final_status='retained').count()
    incomplete = statuses.filter(final_status='pending').count()
    others = total - promoted - retained - incomplete

    # Per program breakdown
    program_stats = {}
    section_stats = {}

    for s in statuses:
        # Program grouping
        program_code = s.section.program.code if s.section and s.section.program else 'Unknown'
        if program_code not in program_stats:
            program_stats[program_code] = {
                'promoted': 0, 'retained': 0, 'pending': 0, 'other': 0,
                'grades': [], 'total': 0
            }
        program_stats[program_code]['total'] += 1
        if s.final_status == 'promoted':

```

```

        program_stats[program_code]['promoted'] += 1
    elif s.final_status == 'retained':
        program_stats[program_code]['retained'] += 1
    elif s.final_status == 'pending':
        program_stats[program_code]['pending'] += 1
    else:
        program_stats[program_code]['other'] += 1
    if s.overall_grade:
        program_stats[program_code]['grades'].append(float(s.overall_grade))

# Section grouping
section_name = s.section.name if s.section else 'Unknown'
section_key = f"{program_code} — {section_name}"
if section_key not in section_stats:
    section_stats[section_key] = {
        'section': section_name,
        'program': program_code,
        'grade_level': s.grade_level.name if s.grade_level else "",
        'promoted': 0, 'retained': 0, 'pending': 0,
        'grades': [], 'total': 0
    }
section_stats[section_key]['total'] += 1
if s.final_status == 'promoted':
    section_stats[section_key]['promoted'] += 1
elif s.final_status == 'retained':
    section_stats[section_key]['retained'] += 1
elif s.final_status == 'pending':
    section_stats[section_key]['pending'] += 1
if s.overall_grade:
    section_stats[section_key]['grades'].append(float(s.overall_grade))

# Individual student records
individual = []
for s in statuses:
    sd = getattr(s.student, 'student_data', None)
    individual.append({
        'lrn': s.student.lrn,
        'name': f"{sd.last_name}, {sd.first_name}" if sd else s.student.lrn,
        'program': s.section.program.code if s.section and s.section.program else '—',
        'section': s.section.name if s.section else '—',
        'grade_level': s.grade_level.name if s.grade_level else '—',
        'overall_grade': float(s.overall_grade) if s.overall_grade else None,
        'final_status': s.final_status,
    })

# Compute averages
for k in program_stats:

```

```

        g = program_stats[k]['grades']
        program_stats[k]['average'] = round(sum(g) / len(g), 2) if g else None
        del program_stats[k]['grades']

    for k in section_stats:
        g = section_stats[k]['grades']
        section_stats[k]['average'] = round(sum(g) / len(g), 2) if g else None
        del section_stats[k]['grades']

    return JsonResponse({
        'success': True,
        'school_year': school_year.year_label,
        'summary': {
            'total': total,
            'promoted': promoted,
            'retained': retained,
            'incomplete': incomplete,
            'others': others,
        },
        'by_program': [
            {'program': k, **v} for k, v in sorted(program_stats.items())
        ],
        'by_section': [
            {'key': k, **v} for k, v in sorted(section_stats.items())
        ],
        'individual': sorted(individual, key=lambda x: x['name']),
    })

@admin_required
def archive_probation(request):
    """Tab 4 — Probation records for a given school year."""
    sy_id = request.GET.get('sy')
    if not sy_id:
        return JsonResponse({'success': False, 'error': 'sy parameter required.'}, status=400)

    try:
        school_year = SchoolYear.objects.get(id=sy_id, is_active=False)
    except SchoolYear.DoesNotExist:
        return JsonResponse({'success': False, 'error': 'School year not found.'}, status=404)

    records = ProbationRecord.objects.filter(
        school_year=school_year
    ).select_related(
        'student', 'student__student_data',
        'grade_level', 'reinstated_by'
    ).order_by('-flagged_at')

```

```

data = []
for rec in records:
    sd = getattr(rec.student, 'student_data', None)
    data.append({
        'ltn': rec.student.ltn,
        'name': f"{sd.last_name}, {sd.first_name}" if sd else rec.student.ltn,
        'grade_level': rec.grade_level.name if rec.grade_level else '—',
        'previous_program': rec.previous_program,
        'moved_to_program': rec.moved_to_program,
        'reason': rec.reason,
        'is_active': rec.is_active,
        'flagged_at': rec.flagged_at.strftime('%b %d, %Y') if rec.flagged_at else '—',
        'lifted_by': (
            rec.reinstated_by.get_full_name() or rec.reinstated_by.username
        ) if rec.reinstated_by else None,
        'lifted_at': rec.reinstated_at.strftime('%b %d, %Y') if rec.reinstated_at else None,
        'lift_reason': rec.reinstatement_reason or None,
    })

return JsonResponse({
    'success': True,
    'school_year': school_year.year_label,
    'total': len(data),
    'active': sum(1 for d in data if d['is_active']),
    'lifted': sum(1 for d in data if not d['is_active']),
    'records': data,
})

```

#

—

EXPORTS

#

—

```

def _header_style():
    fill = PatternFill(start_color='991B1B', end_color='991B1B', fill_type='solid')
    font = Font(bold=True, color='FFFFFF')
    align = Alignment(horizontal='center', vertical='center')
    return fill, font, align

```

@admin_required

```

def export_archive_enrollment(request):
    """Export enrollment summary to Excel."""
    sy_id = request.GET.get('sy')
    try:

```

```

    school_year = SchoolYear.objects.get(id=sy_id, is_active=False)
except SchoolYear.DoesNotExist:
    return HttpResponse('School year not found.', status=404)

enrollments = StudentEnrollment.objects.filter(school_year=school_year)
program_selections = ProgramSelection.objects.filter(school_year=school_year)

wb = openpyxl.Workbook()
ws = wb.active
ws.title = 'Enrollment Summary'

fill, font, align = _header_style()

# Title
ws.merge_cells('A1:F1')
ws['A1'] = f'Enrollment Summary — {school_year.year_label}'
ws['A1'].font = Font(bold=True, size=14)
ws['A1'].alignment = Alignment(horizontal='center')
ws.row_dimensions[1].height = 24

# Overall summary
ws.append([])
ws.append(['Overall Summary'])
ws['A3'].font = Font(bold=True, size=11)
ws.append(['Total Enrolled', enrollments.count()])
ws.append(['New', enrollments.filter(enrollee_type='new').count()])
ws.append(['Continuing', enrollments.filter(enrollee_type='continuing').count()])
ws.append(['Transferee', enrollments.filter(enrollee_type='transferee').count()])
ws.append(['Approved', enrollments.filter(enrollment_status='approved').count()])
ws.append(['Rejected', enrollments.filter(enrollment_status='rejected').count()])

ws.append([])
ws.append(['Per Program Breakdown'])
ws.cell(row=ws.max_row, column=1).font = Font(bold=True, size=11)

headers = ['Program', 'Total', 'Approved', 'Rejected', 'Pending']
ws.append(headers)
header_row = ws.max_row
for col, h in enumerate(headers, 1):
    cell = ws.cell(row=header_row, column=col)
    cell.fill = fill
    cell.font = font
    cell.alignment = align

programs = program_selections.values('selected_program_code').annotate(
    total=Count('student'),
    approved=Count('student', filter=Q(admin_approved=True)),

```

```

        rejected=Count('student', filter=Q(admin_rejected=True)),
    ).order_by('selected_program_code')

    for p in programs:
        if p['selected_program_code']:
            pending = p['total'] - p['approved'] - p['rejected']
            ws.append([p['selected_program_code'], p['total'], p['approved'], p['rejected'], pending])

    for col in ['A', 'B', 'C', 'D', 'E', 'F']:
        ws.column_dimensions[col].width = 18

    response = HttpResponse(
        content_type='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
    )
    response['Content-Disposition'] = f'attachment;
filename="archive_enrollment_{school_year.year_label}.xlsx"'
    wb.save(response)
    return response

@admin_required
def export_archive_sections(request):
    """Export all sections and their student lists to Excel."""
    sy_id = request.GET.get('sy')
    try:
        school_year = SchoolYear.objects.get(id=sy_id, is_active=False)
    except SchoolYear.DoesNotExist:
        return HttpResponse('School year not found.', status=404)

    sections = Section.objects.filter(school_year=school_year).select_related(
        'program', 'grade_level', 'adviser'
    ).order_by('program__code', 'grade_level__code', 'name')

    wb = openpyxl.Workbook()
    wb.remove(wb.active)
    fill, font, align = _header_style()

    for section in sections:
        ws_name = f'{section.program.code if section.program else "UNK"}-{section.name}':31
        ws = wb.create_sheet(title=ws_name)

        ws.merge_cells('A1:F1')
        ws['A1'] = f'{section.name} — {school_year.year_label}'
        ws['A1'].font = Font(bold=True, size=13)
        ws['A1'].alignment = Alignment(horizontal='center')

    ws.append([
        'Program:', section.program.code if section.program else '—',

```

```

        'Grade Level:', section.grade_level.name if section.grade_level else '—',
        'Adviser:', section.adviser.get_full_name() if section.adviser else '—'
    ])
    ws.append([])

    headers = ['#', 'LRN', 'Name', 'Gender', 'Overall Grade', 'Promotion Status']
    ws.append(headers)
    h_row = ws.max_row
    for col, h in enumerate(headers, 1):
        c = ws.cell(row=h_row, column=col)
        c.fill = fill
        c.font = font
        c.alignment = align

    students = ProgramSelection.objects.filter(
        assigned_section=section, admin_approved=True
    ).select_related('student', 'student__student_data').order_by(
        'student__student_data__last_name'
    )

    for idx, ps in enumerate(students, 1):
        sd = getattr(ps.student, 'student_data', None)
        try:
            status = StudentAcademicYearStatus.objects.get(
                student=ps.student, school_year=school_year
            )
            overall = float(status.overall_grade) if status.overall_grade else '—'
            final_status = status.final_status.title()
        except StudentAcademicYearStatus.DoesNotExist:
            overall = '—'
            final_status = '—'

        ws.append([
            idx,
            ps.student.lrn,
            f"{sd.last_name}, {sd.first_name}" if sd else ps.student.lrn,
            sd.gender.capitalize() if sd else '—',
            overall,
            final_status,
        ])

    for col_letter, width in zip(['A', 'B', 'C', 'D', 'E', 'F'], [5, 15, 30, 10, 15, 18]):
        ws.column_dimensions[col_letter].width = width

    if not wb.sheetnames:
        wb.create_sheet('No Data')

```



```

response = HttpResponse(
    content_type='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
)
response['Content-Disposition'] = f'attachment;
filename="archive_sections_{school_year.year_label}.xlsx"'
wb.save(response)
return response

```

```

@admin_required
def export_archive_academic(request):
    """Export academic performance to Excel."""
    sy_id = request.GET.get('sy')
    try:
        school_year = SchoolYear.objects.get(id=sy_id, is_active=False)
    except SchoolYear.DoesNotExist:
        return HttpResponse('School year not found.', status=404)

```

```

statuses = StudentAcademicYearStatus.objects.filter(
    school_year=school_year
).select_related(
    'student', 'student__student_data',
    'section', 'section__program', 'grade_level'
).order_by('section__program__code', 'section__name',
'student__student_data__last_name')

```

```

wb = openpyxl.Workbook()
ws = wb.active
ws.title = 'Academic Performance'
fill, font, align = _header_style()

```

```

ws.merge_cells('A1:G1')
ws['A1'] = f'Academic Performance — {school_year.year_label}'
ws['A1'].font = Font(bold=True, size=14)
ws['A1'].alignment = Alignment(horizontal='center')
ws.row_dimensions[1].height = 24
ws.append([])

```

```

headers = ['#', 'LRN', 'Student Name', 'Program', 'Section', 'Overall Grade', 'Status']
ws.append(headers)
h_row = ws.max_row
for col, h in enumerate(headers, 1):
    c = ws.cell(row=h_row, column=col)
    c.fill = fill
    c.font = font
    c.alignment = align

```

```

for idx, s in enumerate(statuses, 1):

```

```

sd = getattr(s.student, 'student_data', None)
ws.append([
    idx,
    s.student.lrn,
    f'{sd.last_name}, {sd.first_name}' if sd else s.student.lrn,
    s.section.program.code if s.section and s.section.program else '—',
    s.section.name if s.section else '—',
    float(s.overall_grade) if s.overall_grade else '—',
    s.final_status.title(),
])

for col_letter, width in zip(['A','B','C','D','E','F','G'], [5, 15, 30, 10, 20, 15, 15]):
    ws.column_dimensions[col_letter].width = width

response = HttpResponse(
    content_type='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
)
response['Content-Disposition'] = f'attachment;
filename="archive_academic_{school_year.year_label}.xlsx"'
wb.save(response)
return response

```

```

@admin_required
def export_archive_probation(request):
    """Export probation records to Excel."""
    sy_id = request.GET.get('sy')
    try:
        school_year = SchoolYear.objects.get(id=sy_id, is_active=False)
    except SchoolYear.DoesNotExist:
        return HttpResponse('School year not found.', status=404)

```

```

records = ProbationRecord.objects.filter(
    school_year=school_year
).select_related(
    'student', 'student__student_data', 'grade_level', 'reinstated_by'
).order_by('-flagged_at')

```

```

wb = openpyxl.Workbook()
ws = wb.active
ws.title = 'Probation Records'
fill, font, align = _header_style()

```

```

ws.merge_cells('A1:H1')
ws['A1'] = f'STE Probation Records — {school_year.year_label}'
ws['A1'].font = Font(bold=True, size=14)
ws['A1'].alignment = Alignment(horizontal='center')
ws.row_dimensions[1].height = 24

```

```

ws.append([])

headers = ['#', 'LRN', 'Student Name', 'Grade Level', 'Reason', 'Status', 'Lifted By', 'Lifted At']
ws.append(headers)
h_row = ws.max_row
for col, h in enumerate(headers, 1):
    c = ws.cell(row=h_row, column=col)
    c.fill = fill
    c.font = font
    c.alignment = align

for idx, rec in enumerate(records, 1):
    sd = getattr(rec.student, 'student_data', None)
    lifted_by = ""
    if rec.reinstated_by:
        lifted_by = rec.reinstated_by.get_full_name() or rec.reinstated_by.username

    ws.append([
        idx,
        rec.student.lrn,
        f"{sd.last_name}, {sd.first_name}" if sd else rec.student.lrn,
        rec.grade_level.name if rec.grade_level else '—',
        rec.reason,
        'Active' if rec.is_active else 'Lifted',
        lifted_by,
        rec.reinstated_at.strftime('%b %d, %Y') if rec.reinstated_at else '—',
    ])

for col_letter, width in zip(['A','B','C','D','E','F','G','H'], [5,15,30,12,50,10,25,15]):
    ws.column_dimensions[col_letter].width = width

response = HttpResponse(
    content_type='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
)
response['Content-Disposition'] = f'attachment;
filename="archive_probation_{school_year.year_label}.xlsx"'
wb.save(response)
return response

```

move_request_views.py – Program movement handling

```

from django.http import JsonResponse
from django.views.decorators.http import require_http_methods
from django.contrib.auth.decorators import login_required
from django.db import transaction
from django.utils import timezone
import json

```

```

from admin_app.decorators import admin_required
from admin_app.models import ProgramMoveRequest, Program, SchoolYear, Section
from enrollment_app.models import Student, ProgramSelection, EnrollmentStatusLog

@admin_required
def get_pending_move_requests(request):
    """Returns pending move requests count + list for admin bell notification."""
    pending = ProgramMoveRequest.objects.filter(status='pending').select_related(
        'student__student_data', 'requested_by'
    )

    items = []
    for mr in pending:
        student_name = (
            mr.student.student_data.full_name
            if hasattr(mr.student, 'student_data') and mr.student.student_data
            else mr.student.lrn
        )
        items.append({
            'id': mr.id,
            'student_lrn': mr.student.lrn,
            'student_name': student_name,
            'from_program': mr.from_program_code,
            'to_program': mr.to_program_code,
            'reason': mr.reason,
            'requested_by': mr.requested_by.get_full_name() if mr.requested_by else 'Unknown',
            'created_at': mr.created_at.strftime("%b %d, %Y"),
        })

    return JsonResponse({'count': len(items), 'requests': items})

@admin_required
@require_http_methods(["POST"])
def review_move_request(request, move_request_id):
    """
    Admin approves or rejects a move request.
    On approval: moves student to new program, clears old section,
    and auto-places in new program section if available.
    Admin manually picks target section.
    """
    try:
        with transaction.atomic():
            data = json.loads(request.body)
            action = data.get('action') # 'approve' or 'reject'
            review_notes = data.get('review_notes', "")
            target_section_id = data.get('target_section_id') # required for approve

```

```

if action not in ('approve', 'reject'):
    return JsonResponse({'success': False, 'error': 'Invalid action.'}, status=400)

move_request = ProgramMoveRequest.objects.select_related(
    'student', 'from_section'
).get(id=move_request_id, status='pending')

if action == 'reject':
    move_request.status = 'rejected'
    move_request.reviewed_by = request.user
    move_request.review_notes = review_notes
    move_request.reviewed_at = timezone.now()
    move_request.save()
    return JsonResponse({'success': True, 'message': 'Move request rejected.'})

# --- APPROVE ---
if not target_section_id:
    return JsonResponse({
        'success': False,
        'error': 'Target section is required to approve a move request.'
    }, status=400)

target_section = Section.objects.select_related('program').get(id=target_section_id)

# Verify section belongs to the target program
if target_section.program.code.upper() != move_request.to_program_code.upper():
    return JsonResponse({
        'success': False,
        'error': f'Selected section belongs to {target_section.program.code}, '
                f'not {move_request.to_program_code}.'
    }, status=400)

# Check section capacity
if target_section.get_actual_count() >= target_section.max_students:
    return JsonResponse({
        'success': False,
        'error': f'Section {target_section.name} is full '
                f'({target_section.max_students}/{target_section.max_students}).'
    }, status=400)

student = move_request.student
program_selection = ProgramSelection.objects.get(student=student)

# Remove from old section
old_section = program_selection.assigned_section
old_program_code = program_selection.selected_program_code

```

```

# Update program selection
program_selection.selected_program_code = move_request.to_program_code
program_selection.assigned_section = target_section
program_selection.section_assigned_at = timezone.now()
program_selection.admin_approved = True
program_selection.admin_notes = (
    f'[PROGRAM MOVE] From {old_program_code} to {move_request.to_program_code}.

    f'Admin: {review_notes}'
)
program_selection.save()

# Update section counts
if old_section:
    old_section.update_current_students_count()
target_section.update_current_students_count()

# Log status change
EnrollmentStatusLog.objects.create(
    student=student,
    old_status=student.enrollment_status,
    new_status='approved',
    changed_by=request.user.get_full_name() or request.user.username,
    change_reason=(
        f'Program moved from {old_program_code} to {move_request.to_program_code} '
        f'by admin. Section: {target_section.name}'
    )
)

# Mark move request as approved
move_request.status = 'approved'
move_request.reviewed_by = request.user
move_request.review_notes = review_notes
move_request.reviewed_at = timezone.now()
move_request.save()

student_name = (
    student.student_data.full_name
    if hasattr(student, 'student_data') and student.student_data
    else student.lrn
)

return JsonResponse({
    'success': True,
    'message': (
        f'{student_name} has been moved from {old_program_code} '
        f'to {move_request.to_program_code} and placed in {target_section.name}.'
    )
})

```

```

    ),
    'student_name': student_name,
    'new_program': move_request.to_program_code,
    'new_section': target_section.name,
})

except ProgramMoveRequest.DoesNotExist:
    return JsonResponse({'success': False, 'error': 'Move request not found or already
reviewed.'}, status=404)
except Section.DoesNotExist:
    return JsonResponse({'success': False, 'error': 'Target section not found.'}, status=404)
except Exception as e:
    import traceback
    traceback.print_exc()
    return JsonResponse({'success': False, 'error': str(e)}, status=500)

@admin_required
def get_sections_for_program(request):
    """Returns sections available for a given program code (for admin move modal)."""
    program_code = request.GET.get('program_code', "").upper()
    if not program_code:
        return JsonResponse({'success': False, 'error': 'Program code required.'}, status=400)

    school_year = SchoolYear.objects.filter(is_active=True).first()
    sections = Section.objects.filter(
        program__code=program_code,
        school_year=school_year
    ).order_by('name')

    data = []
    for s in sections:
        actual = s.get_actual_count()
        data.append({
            'id': s.id,
            'name': s.name,
            'current': actual,
            'max': s.max_students,
            'available': s.max_students - actual,
            'is_full': actual >= s.max_students,
            'regular_track': getattr(s, 'regular_track', "") or "",
        })

    return JsonResponse({'success': True, 'sections': data})

```

reports_views.py – Report generation

"""

admin_app/views/reports_views.py

"""

```
import csv
from django.contrib.auth.decorators import login_required
from django.http import JsonResponse, HttpResponse
from django.shortcuts import render
from django.views.decorators.http import require_http_methods
from django.db.models import Count, Q

from admin_app.models import SchoolYear, UserProfile, Section, GradeLevel, Program
from enrollment_app.models import (
    Student, StudentEnrollment, ProgramSelection,
    StudentAcademicYearStatus, StudentData,
    StudentDocumentSubmission,
)
```

```
@login_required
def reports(request):
    active_sy = SchoolYear.objects.filter(is_active=True).first()
    all_sy = SchoolYear.objects.all().order_by('-year_label')
    programs = Program.objects.all().order_by('code')
    grades = GradeLevel.objects.all().order_by('code')
    return render(request, 'admin_app/reports.html', {
        'active_page': 'reports',
        'active_sy': active_sy,
        'active_school_year': active_sy, # ← base.html reads this
        'all_sy': all_sy,
        'programs': programs,
        'grades': grades,
    })
```

```
@login_required
@require_http_methods(["GET"])
def reports_header_data(request):
    active_sy = SchoolYear.objects.filter(is_active=True).first()
    try:
        user_profile = UserProfile.objects.select_related(
            'program', 'position', 'department'
        ).get(user=request.user)
    except UserProfile.DoesNotExist:
        user_profile = None

    user = request.user
    full_name = f"{user.first_name} {user.last_name}".strip() or user.username
    initials = ".join([n[0].upper() for n in full_name.split()[:2]]) or 'A'
    role = user_profile.get_user_type_display() if user_profile else 'Admin'
    photo_url = user_profile.photo.url if user_profile and user_profile.photo else None
```



```

return JsonResponse({
    'school_year': active_sy.year_label if active_sy else 'No Active Year',
    'full_name': full_name,
    'role': role,
    'initials': initials,
    'photo_url': photo_url,
})

```

```

@login_required
def generate_report(request):
    if request.method == 'POST':
        return JsonResponse({'status': 'success', 'message': 'Report generated'})
    return JsonResponse({'status': 'error', 'message': 'Invalid request'})

```

```

#

```

```

# SUMMARY STATS
#

```

```

@login_required
@require_http_methods(["GET"])
def reports_summary(request):
    sy_label = request.GET.get('school_year')
    sy = (
        SchoolYear.objects.filter(year_label=sy_label).first()
        if sy_label else
        SchoolYear.objects.filter(is_active=True).first()
    )
    if not sy:
        return JsonResponse({'error': 'No school year found'}, status=400)

```

```

total_enrolled = StudentEnrollment.objects.filter(
    school_year=sy, enrollment_status='approved',
).count()

```

```

total_sections = Section.objects.filter(school_year=sy).count()

```

```

program_dist = (
    ProgramSelection.objects
        .filter(school_year=sy, admin_approved=True)
        .values('selected_program_code')
        .annotate(count=Count('student'))
        .order_by('-count')
)

```

```

promoted = StudentAcademicYearStatus.objects.filter(
    school_year=sy, final_status='promoted',
).count()

retained = StudentAcademicYearStatus.objects.filter(
    school_year=sy, final_status='retained',
).count()

pending_enrollment = ProgramSelection.objects.filter(
    school_year=sy, admin_approved=False, admin_rejected=False,
).count()

return JsonResponse({
    'school_year':      sy.year_label,
    'total_enrolled':   total_enrolled,
    'total_sections':   total_sections,
    'promoted':         promoted,
    'retained':         retained,
    'pending_enrollment': pending_enrollment,
    'program_distribution': list(program_dist),
})

```

#

PREVIEW

#

```

@login_required
@require_http_methods(["GET"])
def get_report_preview(request):
    report_type = request.GET.get('type')
    sy_label = request.GET.get('school_year')
    program_code = request.GET.get('program', 'all')
    grade_code = request.GET.get('grade', 'all')

    sy = (
        SchoolYear.objects.filter(year_label=sy_label).first()
        if sy_label else
        SchoolYear.objects.filter(is_active=True).first()
    )
    if not sy:
        return JsonResponse({'error': 'No school year found'}, status=400)

```

```

try:
    if report_type == 'enrolled':
        qs = ProgramSelection.objects.filter(school_year=sy, admin_approved=True)
        if program_code != 'all':
            qs = qs.filter(selected_program_code=program_code)
        return JsonResponse({'count': qs.count(), 'label': 'enrolled students'})

    elif report_type == 'sections':
        qs = Section.objects.filter(school_year=sy)
        if program_code != 'all':
            qs = qs.filter(program__code=program_code)
        return JsonResponse({'count': qs.count(), 'label': 'sections'})

    elif report_type == 'promotion':
        qs = StudentAcademicYearStatus.objects.filter(school_year=sy)
        return JsonResponse({
            'count': qs.count(),
            'promoted': qs.filter(final_status='promoted').count(),
            'retained': qs.filter(final_status='retained').count(),
            'pending': qs.filter(final_status='pending').count(),
            'label': 'students with academic status',
        })

    elif report_type == 'pending':
        qs = ProgramSelection.objects.filter(
            school_year=sy, admin_approved=False, admin_rejected=False,
        )
        return JsonResponse({'count': qs.count(), 'label': 'pending enrollments'})

    elif report_type == 'masterlist':
        sections = Section.objects.filter(school_year=sy)
        if program_code != 'all':
            sections = sections.filter(program__code=program_code)
        students = ProgramSelection.objects.filter(
            school_year=sy, admin_approved=True,
            assigned_section__in=sections,
        ).count()
        return JsonResponse({
            'count': students,
            'sections': sections.count(),
            'label': 'students across sections',
        })

    elif report_type == 'transferees':
        qs = StudentEnrollment.objects.filter(
            school_year=sy, enrollee_type='transferee',
        )

```

```

        return JsonResponse({'count': qs.count(), 'label': 'transferee students'})

elif report_type == 'probation':
    try:
        from coordinator_app.models import ProbationRecord
        qs = ProbationRecord.objects.filter(school_year=sy, is_active=True)
        return JsonResponse({'count': qs.count(), 'label': 'students on probation'})
    except Exception:
        return JsonResponse({'count': 0, 'label': 'students on probation'})

elif report_type == 'ai_vs_manual':
    approved_qs = ProgramSelection.objects.filter(
        school_year=sy, admin_approved=True,
    )
    ai_count = approved_qs.filter(approved_by__icontains='AI').count()
    manual_count = approved_qs.exclude(approved_by__icontains='AI').count()
    return JsonResponse({
        'count': approved_qs.count(),
        'ai': ai_count,
        'manual': manual_count,
        'label': 'approved enrollments',
    })

elif report_type == 'no_section':
    qs = ProgramSelection.objects.filter(
        school_year=sy, admin_approved=True, assigned_section__isnull=True,
    )
    if program_code != 'all':
        qs = qs.filter(selected_program_code=program_code)
    return JsonResponse({'count': qs.count(), 'label': 'students without section'})

elif report_type == 'documents':
    missing = (
        StudentEnrollment.objects
        .filter(school_year=sy, enrollment_status='approved', documents_completed=False)
        .count()
    )
    complete = (
        StudentEnrollment.objects
        .filter(school_year=sy, enrollment_status='approved', documents_completed=True)
        .count()
    )
    return JsonResponse({
        'count': missing + complete,
        'complete': complete,
        'missing': missing,
        'label': 'students (document status)',
    })

```

```

    })

elif report_type == 'activity_log':
    try:
        from coordinator_app.models import CoordinatorActivityLog
        qs = CoordinatorActivityLog.objects.filter(
            created_at__year=sy.start_date.year
        ) if hasattr(sy, 'start_date') and sy.start_date else CoordinatorActivityLog.objects.all()
        return JsonResponse({'count': qs.count(), 'label': 'activity log entries'})
    except Exception:
        return JsonResponse({'count': 0, 'label': 'activity log entries'})

elif report_type == 'enrollee_type':
    breakdown = (
        StudentEnrollment.objects
        .filter(school_year=sy)
        .values('enrollee_type')
        .annotate(count=Count('id'))
        .order_by('enrollee_type')
    )
    return JsonResponse({
        'count': StudentEnrollment.objects.filter(school_year=sy).count(),
        'breakdown': list(breakdown),
        'label': 'total enrollments',
    })

elif report_type == 'move_requests':
    try:
        from admin_app.models import ProgramMoveRequest
        qs = ProgramMoveRequest.objects.filter(
            student__enrollments__school_year=sy
        ).distinct()
        return JsonResponse({
            'count': qs.count(),
            'approved': qs.filter(status='approved').count(),
            'rejected': qs.filter(status='rejected').count(),
            'pending': qs.filter(status='pending').count(),
            'label': 'program move requests',
        })
    except Exception:
        return JsonResponse({'count': 0, 'label': 'program move requests'})

return JsonResponse({'error': 'Unknown report type'}, status=400)

except Exception as e:
    return JsonResponse({'error': str(e)}, status=500)

```

```
#
```

```
# EXPORT HELPERS
```

```
#
```

```
def _get_sy(sy_label):
    return (
        SchoolYear.objects.filter(year_label=sy_label).first()
        if sy_label else
        SchoolYear.objects.filter(is_active=True).first()
    )

def _csv_response(filename):
    response = HttpResponse(content_type='text/csv')
    response['Content-Disposition'] = f'attachment; filename="{filename}"'
    return response
```

```
#
```

```
# EXPORT ENDPOINTS
```

```
#
```

```
@login_required
@require_http_methods(["GET"])
def export_enrolled_students(request):
    sy = _get_sy(request.GET.get('school_year'))
    if not sy:
        return HttpResponse('No school year found.', status=400)

    program_code = request.GET.get('program', 'all')
    grade_code = request.GET.get('grade', 'all')

    qs = ProgramSelection.objects.filter(
        school_year=sy, admin_approved=True,
    ).select_related(
        'student', 'student__student_data',
        'assigned_section', 'assigned_section__grade_level',
    ).order_by('selected_program_code', 'student__student_data__last_name')

    if program_code != 'all':
        qs = qs.filter(selected_program_code=program_code)
```

```

if grade_code != 'all':
    qs = qs.filter(assigned_section__grade_level__code=grade_code)

response = _csv_response(f"enrolled_students_{sy.year_label.replace('-', '_')}.csv")
writer = csv.writer(response)
writer.writerow([
    'LRN', 'Last Name', 'First Name', 'Middle Name',
    'Gender', 'Program', 'Track', 'Grade Level',
    'Section', 'Enrollment Type', 'Approved By', 'Approved At',
])

```

```

for ps in qs:
    sd = getattr(ps.student, 'student_data', None)
    sec = ps.assigned_section
    enrollment = StudentEnrollment.objects.filter(
        student=ps.student, school_year=sy,
    ).first()

    writer.writerow([
        ps.student.lrn,
        getattr(sd, 'last_name', "") or "",
        getattr(sd, 'first_name', "") or "",
        getattr(sd, 'middle_name', "") or "",
        getattr(sd, 'gender', "") or "",
        ps.selected_program_code or "",
        ps.regular_track or "",
        sec.grade_level.name if sec and sec.grade_level else "",
        sec.name if sec else 'Not Assigned',
        enrollment.enrollee_type if enrollment else "",
        ps.approved_by or "",
        ps.approved_at.strftime('%Y-%m-%d %H:%M') if ps.approved_at else "",
    ])

```

```

return response

```

```

@login_required
@require_http_methods(["GET"])
def export_section_list(request):
    sy = _get_sy(request.GET.get('school_year'))
    if not sy:
        return HttpResponse('No school year found.', status=400)

    program_code = request.GET.get('program', 'all')
    grade_code = request.GET.get('grade', 'all')

    qs = Section.objects.filter(school_year=sy).select_related(
        'program', 'grade_level', 'adviser'
    )

```

```

).order_by('program__code', 'grade_level__code', 'name')

if program_code != 'all':
    qs = qs.filter(program__code=program_code)
if grade_code != 'all':
    qs = qs.filter(grade_level__code=grade_code)

response = _csv_response(f"sections_{sy.year_label.replace('-', '_')}.csv")
writer = csv.writer(response)
writer.writerow([
    'Section Name', 'Program', 'Track', 'Grade Level',
    'Adviser', 'Max Students', 'Enrolled', 'Available Slots',
])

for section in qs:
    section.update_current_students_count()
    adviser_name = ""
    if section.adviser:
        adviser_name = getattr(section.adviser, 'full_name', str(section.adviser))
    writer.writerow([
        section.name,
        section.program.code if section.program else "",
        section.regular_track or "",
        section.grade_level.name if section.grade_level else "",
        adviser_name,
        section.max_students,
        section.current_students,
        section.max_students - section.current_students,
    ])

return response

@login_required
@require_http_methods(["GET"])
def export_promotion_status(request):
    sy_label = request.GET.get('school_year')
    sy = (
        SchoolYear.objects.filter(year_label=sy_label).first()
        if sy_label else
        SchoolYear.objects.filter(is_active=False).order_by('-year_label').first()
    )
    if not sy:
        return HttpResponse('No school year found.', status=400)

    program_code = request.GET.get('program', 'all')
    status_filter = request.GET.get('status', 'all')

```



```

qs = StudentAcademicYearStatus.objects.filter(
    school_year=sy,
).select_related(
    'student', 'student__student_data', 'grade_level', 'section',
).order_by('final_status', 'student__student_data__last_name')

if status_filter != 'all':
    qs = qs.filter(final_status=status_filter)
if program_code != 'all':
    qs = qs.filter(section__program__code=program_code)

response = _csv_response(f"promotion_status_{sy.year_label.replace('-', '_')}.csv")
writer = csv.writer(response)
writer.writerow([
    'LRN', 'Last Name', 'First Name',
    'Grade Level', 'Section', 'Program',
    'Overall Grade', 'Final Status', 'Remarks',
])

for status in qs:
    sd = getattr(status.student, 'student_data', None)
    sec = status.section
    writer.writerow([
        status.student.lrn,
        getattr(sd, 'last_name', "") or "",
        getattr(sd, 'first_name', "") or "",
        status.grade_level.name if status.grade_level else "",
        sec.name if sec else "",
        sec.program.code if sec and sec.program else "",
        status.overall_grade or "",
        status.final_status,
        status.remarks or "",
    ])

return response

@login_required
@require_http_methods(["GET"])
def export_pending_enrollments(request):
    sy = _get_sy(request.GET.get('school_year'))
    if not sy:
        return HttpResponse('No school year found.', status=400)

    qs = ProgramSelection.objects.filter(
        school_year=sy, admin_approved=False, admin_rejected=False,
    ).select_related(
        'student', 'student__student_data',

```

```

).order_by('selected_program_code', 'created_at')

response = _csv_response(f"pending_enrollments_{sy.year_label.replace('-', '_')}.csv")
writer = csv.writer(response)
writer.writerow([
    'LRN', 'Last Name', 'First Name',
    'Program Applied', 'Track', 'Enrollee Type',
    'Enrollment Status', 'Submitted At', 'Notes',
])

```

```

for ps in qs:
    sd = getattr(ps.student, 'student_data', None)
    enrollment = StudentEnrollment.objects.filter(
        student=ps.student, school_year=sy,
    ).first()

    writer.writerow([
        ps.student.lrn,
        getattr(sd, 'last_name', '') or '',
        getattr(sd, 'first_name', '') or '',
        ps.selected_program_code or '',
        ps.regular_track or '',
        enrollment.enrollee_type if enrollment else '',
        enrollment.enrollment_status if enrollment else '',
        ps.created_at.strftime('%Y-%m-%d %H:%M') if ps.created_at else '',
        ps.admin_notes or '',
    ])

```

```

return response

```

```

@login_required
@require_http_methods(["GET"])
def export_masterlist(request):
    sy = _get_sy(request.GET.get('school_year'))
    if not sy:
        return HttpResponse('No school year found.', status=400)

    program_code = request.GET.get('program', 'all')
    grade_code = request.GET.get('grade', 'all')

    sections_qs = Section.objects.filter(school_year=sy).select_related(
        'program', 'grade_level', 'adviser'
    ).order_by('program__code', 'grade_level__code', 'name')

    if program_code != 'all':
        sections_qs = sections_qs.filter(program__code=program_code)
    if grade_code != 'all':

```

```

sections_qs = sections_qs.filter(grade_level__code=grade_code)

response = _csv_response(f"masterlist_{sy.year_label.replace('-', '_')}.csv")
writer = csv.writer(response)

for section in sections_qs:
    adviser_name = ""
    if section.adviser:
        adviser_name = getattr(section.adviser, 'full_name', str(section.adviser))

    writer.writerow([])
    writer.writerow([
        f"SECTION: {section.name}",
        f"Program: {section.program.code if section.program else ""}",
        f"Grade: {section.grade_level.name if section.grade_level else ""}",
        f"Adviser: {adviser_name}",
        f"School Year: {sy.year_label}",
    ])
    writer.writerow([
        '#', 'LRN', 'Last Name', 'First Name', 'Middle Name',
        'Gender', 'Date of Birth', 'Address',
    ])

    students_in_section = ProgramSelection.objects.filter(
        school_year=sy, assigned_section=section, admin_approved=True,
    ).select_related('student', 'student__student_data').order_by(
        'student__student_data__last_name'
    )

    for i, ps in enumerate(students_in_section, 1):
        sd = getattr(ps.student, 'student_data', None)
        writer.writerow([
            i,
            ps.student.lrn,
            getattr(sd, 'last_name', "") or "",
            getattr(sd, 'first_name', "") or "",
            getattr(sd, 'middle_name', "") or "",
            getattr(sd, 'gender', "") or "",
            getattr(sd, 'date_of_birth', "") or "",
            getattr(sd, 'address', "") or "",
        ])

    writer.writerow([f"Total: {students_in_section.count()}"])

return response

@login_required

```

```

@require_http_methods(["GET"])
def export_transferees(request):
    sy = _get_sy(request.GET.get('school_year'))
    if not sy:
        return HttpResponse('No school year found.', status=400)

    qs = StudentEnrollment.objects.filter(
        school_year=sy, enrollee_type='transferee',
    ).select_related('student', 'student__student_data', 'grade_level').order_by(
        'student__student_data__last_name'
    )

    response = _csv_response(f"transferees_{sy.year_label.replace('-', '_')}.csv")
    writer = csv.writer(response)
    writer.writerow([
        'LRN', 'Last Name', 'First Name', 'Middle Name',
        'Gender', 'Grade Level', 'Program Applied',
        'Previous School', 'Enrollment Status',
    ])

    for e in qs:
        sd = getattr(e.student, 'student_data', None)
        ps = ProgramSelection.objects.filter(student=e.student).first()
        writer.writerow([
            e.student.lrn,
            getattr(sd, 'last_name', "") or "",
            getattr(sd, 'first_name', "") or "",
            getattr(sd, 'middle_name', "") or "",
            getattr(sd, 'gender', "") or "",
            e.grade_level.name if e.grade_level else "",
            ps.selected_program_code if ps else "",
            getattr(sd, 'last_school_attended', "") or "",
            e.enrollment_status,
        ])

    return response

@login_required
@require_http_methods(["GET"])
def export_probation_list(request):
    sy = _get_sy(request.GET.get('school_year'))
    if not sy:
        return HttpResponse('No school year found.', status=400)

    response = _csv_response(f"probation_list_{sy.year_label.replace('-', '_')}.csv")
    writer = csv.writer(response)

```

```

try:
    from coordinator_app.models import ProbationRecord

    qs = ProbationRecord.objects.filter(
        school_year=sy,
    ).select_related(
        'student', 'student__student_data', 'grade_level',
    ).order_by('student__student_data__last_name')

    writer.writerow([
        'LRN', 'Last Name', 'First Name',
        'Grade Level', 'Previous Program', 'Moved To',
        'Reason', 'Is Active',
    ])

    for p in qs:
        sd = getattr(p.student, 'student_data', None)
        writer.writerow([
            p.student.lrn,
            getattr(sd, 'last_name', "") or "",
            getattr(sd, 'first_name', "") or "",
            p.grade_level.name if p.grade_level else "",
            p.previous_program or "",
            p.moved_to_program or "",
            p.reason or "",
            'Yes' if p.is_active else 'No',
        ])

    except Exception as e:
        writer.writerow(['Error: {str(e)}'])

    return response

@login_required
@require_http_methods(["GET"])
def export_ai_vs_manual(request):
    sy = _get_sy(request.GET.get('school_year'))
    if not sy:
        return HttpResponse('No school year found.', status=400)

    qs = ProgramSelection.objects.filter(
        school_year=sy, admin_approved=True,
    ).select_related(
        'student', 'student__student_data', 'assigned_section',
    ).order_by('approved_by', 'student__student_data__last_name')

    response = _csv_response(f'ai_vs_manual_{sy.year_label.replace('-', '_')}.csv")

```

```

writer = csv.writer(response)
writer.writerow([
    'LRN', 'Last Name', 'First Name',
    'Program', 'Section', 'Approved By',
    'Processing Type', 'Approved At',
])

for ps in qs:
    sd = getattr(ps.student, 'student_data', None)
    processing = 'AI Automated' if (ps.approved_by or "").upper().__contains__('AI') else
'Manual'
    writer.writerow([
        ps.student.lrn,
        getattr(sd, 'last_name', "") or "",
        getattr(sd, 'first_name', "") or "",
        ps.selected_program_code or "",
        ps.assigned_section.name if ps.assigned_section else "",
        ps.approved_by or "",
        processing,
        ps.approved_at.strftime('%Y-%m-%d %H:%M') if ps.approved_at else "",
    ])

return response

@login_required
@require_http_methods(["GET"])
def export_no_section(request):
    sy = _get_sy(request.GET.get('school_year'))
    if not sy:
        return HttpResponse('No school year found.', status=400)

    program_code = request.GET.get('program', 'all')

    qs = ProgramSelection.objects.filter(
        school_year=sy, admin_approved=True, assigned_section__isnull=True,
    ).select_related('student', 'student__student_data')

    if program_code != 'all':
        qs = qs.filter(selected_program_code=program_code)

    qs = qs.order_by('selected_program_code', 'student__student_data__last_name')

    response = _csv_response(f"no_section_assigned_{sy.year_label.replace('-', '_')}.csv")
    writer = csv.writer(response)
    writer.writerow([
        'LRN', 'Last Name', 'First Name',
        'Program', 'Track', 'Approved By', 'Approved At',
    ])

```

```
)
```

```
for ps in qs:
```

```
    sd = getattr(ps.student, 'student_data', None)
```

```
    writer.writerow([
```

```
        ps.student.lrn,
```

```
        getattr(sd, 'last_name', "") or "",
```

```
        getattr(sd, 'first_name', "") or "",
```

```
        ps.selected_program_code or "",
```

```
        ps.regular_track or "",
```

```
        ps.approved_by or "",
```

```
        ps.approved_at.strftime('%Y-%m-%d %H:%M') if ps.approved_at else "",
```

```
    ])
```

```
return response
```

```
@login_required
```

```
@require_http_methods(["GET"])
```

```
def export_document_compliance(request):
```

```
    sy = _get_sy(request.GET.get('school_year'))
```

```
    if not sy:
```

```
        return HttpResponse('No school year found.', status=400)
```

```
    status_filter = request.GET.get('doc_status', 'all')
```

```
    qs = StudentEnrollment.objects.filter(
```

```
        school_year=sy,
```

```
    ).select_related('student', 'student__student_data')
```

```
    if status_filter == 'complete':
```

```
        qs = qs.filter(documents_completed=True)
```

```
    elif status_filter == 'missing':
```

```
        qs = qs.filter(documents_completed=False)
```

```
    qs = qs.order_by('student__student_data__last_name')
```

```
    response = _csv_response(f'document_compliance_{sy.year_label.replace('-', '_')}.csv")
```

```
    writer = csv.writer(response)
```

```
    writer.writerow([
```

```
        'LRN', 'Last Name', 'First Name',
```

```
        'Enrollee Type', 'Enrollment Status',
```

```
        'Documents Complete', 'Program',
```

```
    ])
```

```
for e in qs:
```

```
    sd = getattr(e.student, 'student_data', None)
```

```
    ps = ProgramSelection.objects.filter(student=e.student).first()
```

```

writer.writerow([
    e.student.lrn,
    getattr(sd, 'last_name', '') or '',
    getattr(sd, 'first_name', '') or '',
    e.enrollee_type,
    e.enrollment_status,
    'Yes' if e.documents_completed else 'No',
    ps.selected_program_code if ps else '',
])

```

```

return response

```

```

@login_required
@require_http_methods(["GET"])
def export_activity_log(request):
    sy = _get_sy(request.GET.get('school_year'))
    category = request.GET.get('category', 'all')

    response = _csv_response(f'activity_log_{sy.year_label.replace('-', '_') if sy else 'all'}.csv')
    writer = csv.writer(response)

```

```

try:
    from coordinator_app.models import CoordinatorActivityLog

```

```

    qs = CoordinatorActivityLog.objects.select_related(
        'user', 'program',
    ).order_by('-created_at')

```

```

    if sy and hasattr(sy, 'start_date') and sy.start_date:
        qs = qs.filter(created_at__year=sy.start_date.year)

```

```

    if category != 'all':
        qs = qs.filter(category=category)

```

```

writer.writerow([
    'Date', 'Time', 'User', 'Action',
    'Category', 'Program', 'Student LRN',
    'Student Name', 'Section', 'Description',
])

```

```

for log in qs:
    writer.writerow([
        log.created_at.strftime('%Y-%m-%d'),
        log.created_at.strftime('%H:%M:%S'),
        log.user.get_full_name() if log.user else '',
        log.action_display if hasattr(log, 'action_display') else log.action,
        log.category or '',
    ])

```



```

        log.program.code if log.program else ",
        log.student_lrn or ",
        log.student_name or ",
        log.section_name or ",
        log.description or ",
    ])

```

```

except Exception as e:
    writer.writerow([f'Error: {str(e)}'])

```

```

return response

```

```

@login_required
@require_http_methods(["GET"])
def export_enrollee_type_breakdown(request):
    sy = _get_sy(request.GET.get('school_year'))
    if not sy:
        return HttpResponse('No school year found.', status=400)

    qs = StudentEnrollment.objects.filter(
        school_year=sy,
    ).select_related('student', 'student__student_data', 'grade_level')

    enrollee_type = request.GET.get('enrollee_type', 'all')
    if enrollee_type != 'all':
        qs = qs.filter(enrollee_type=enrollee_type)

    qs = qs.order_by('enrollee_type', 'student__student_data__last_name')

    response = _csv_response(f'enrollee_breakdown_{sy.year_label.replace('-', '_')}.csv')
    writer = csv.writer(response)
    writer.writerow([
        'LRN', 'Last Name', 'First Name',
        'Enrollee Type', 'Grade Level',
        'Enrollment Status', 'Program',
    ])

```

```

for e in qs:
    sd = getattr(e.student, 'student_data', None)
    ps = ProgramSelection.objects.filter(student=e.student).first()
    writer.writerow([
        e.student.lrn,
        getattr(sd, 'last_name', "") or "",
        getattr(sd, 'first_name', "") or "",
        e.get_enrollee_type_display(),
        e.grade_level.name if e.grade_level else "",
        e.enrollment_status,
    ])

```

```

        ps.selected_program_code if ps else ",
    ])

    return response

@login_required
@require_http_methods(["GET"])
def export_move_requests(request):
    sy = _get_sy(request.GET.get('school_year'))

    response = _csv_response(f'move_requests_{sy.year_label.replace('-', '_') if sy else 'all'}.csv")
    writer = csv.writer(response)

    try:
        from admin_app.models import ProgramMoveRequest

        qs = ProgramMoveRequest.objects.select_related(
            'student', 'student__student_data',
            'requested_by', 'reviewed_by',
        ).order_by('-created_at')

        if sy:
            qs = qs.filter(student__enrollments__school_year=sy).distinct()

        status_filter = request.GET.get('status', 'all')
        if status_filter != 'all':
            qs = qs.filter(status=status_filter)

        writer.writerow([
            'LRN', 'Last Name', 'First Name',
            'From Program', 'To Program',
            'Requested By', 'Status',
            'Reviewed By', 'Reason',
            'Created At', 'Reviewed At',
        ])

        for r in qs:
            sd = getattr(r.student, 'student_data', None)
            writer.writerow([
                r.student.lrn,
                getattr(sd, 'last_name', "") or "",
                getattr(sd, 'first_name', "") or "",
                r.from_program_code or "",
                r.to_program_code or "",
                r.requested_by.get_full_name() if r.requested_by else "",
                r.status,
                r.reviewed_by.get_full_name() if r.reviewed_by else "",
            ])

```

```

        r.reason or ",
        r.created_at.strftime('%Y-%m-%d %H:%M') if r.created_at else ",
        r.reviewed_at.strftime('%Y-%m-%d %H:%M') if r.reviewed_at else ",
    ])

```

```

except Exception as e:
    writer.writerow([f'Error: {str(e)}'])

```

```

return response

```

Admin Management Commands

create_initial_data.py – Initialize system with base data

```

from django.core.management.base import BaseCommand
from admin_app.models import Position, Department, Program

```

```

class Command(BaseCommand):
    help = 'Creates initial positions, departments, and programs'

    def handle(self, *args, **kwargs):
        # Create Positions
        self.stdout.write(self.style.WARNING('\n 📄 Creating Positions...'))
        positions = [
            {'name': 'Teacher I', 'description': 'Entry-level teaching position'},
            {'name': 'Teacher II', 'description': 'Mid-level teaching position'},
            {'name': 'Teacher III', 'description': 'Senior teaching position'},
            {'name': 'Master Teacher I', 'description': 'Advanced teaching position'},
            {'name': 'Master Teacher II', 'description': 'Advanced teaching position'},
            {'name': 'Master Teacher III', 'description': 'Advanced teaching position'},
            {'name': 'Head Teacher I', 'description': 'Department head position'},
            {'name': 'Head Teacher II', 'description': 'Department head position'},
            {'name': 'Head Teacher III', 'description': 'Department head position'},
            {'name': 'Principal I', 'description': 'School principal'},
            {'name': 'Principal II', 'description': 'School principal'},
            {'name': 'Principal III', 'description': 'School principal'},
            {'name': 'School Head', 'description': 'School head administrator'},
        ]

        for pos_data in positions:
            pos, created = Position.objects.get_or_create(
                name=pos_data['name'],
                defaults={'description': pos_data['description']}
            )
            if created:
                self.stdout.write(self.style.SUCCESS(f' ✓ Created position: {pos.name}'))
            else:
                self.stdout.write(f' - Position already exists: {pos.name}')

```

```

# Create Departments
self.stdout.write(self.style.WARNING("\n 🏢 Creating Departments..."))
departments = [
    {'name': 'English Department', 'description': 'Handles English language and literature education'},
    {'name': 'Science Department', 'description': 'Covers all science subjects'},
    {'name': 'Mathematics Department', 'description': 'Mathematics and related subjects'},
    {'name': 'Filipino Department', 'description': 'Filipino language and culture'},
    {'name': 'Values Department', 'description': 'Values education and character formation'},
    {'name': 'TLE Department', 'description': 'Technology and Livelihood Education'},
    {'name': 'History Department', 'description': 'History and social studies'},
    {'name': 'MAPEH Department', 'description': 'Music, Arts, Physical Education, and Health'},
]

for dept_data in departments:
    dept, created = Department.objects.get_or_create(
        name=dept_data['name'],
        defaults={'description': dept_data['description']}
    )
    if created:
        self.stdout.write(self.style.SUCCESS(f' ✓ Created department: {dept.name}'))
    else:
        self.stdout.write(f' - Department already exists: {dept.name}')

# Create Programs
self.stdout.write(self.style.WARNING("\n 🎓 Creating Programs..."))
programs = [
    {'code': 'STE', 'name': 'Science, Technology, and Engineering', 'description': 'Focus on science and technology education'},
    {'code': 'STEM', 'name': 'Science, Technology, Engineering, and Mathematics', 'description': 'Advanced science and math program'},
    {'code': 'SPFL', 'name': 'Special Program in Foreign Language', 'description': 'Language immersion program'},
    {'code': 'SPTVE', 'name': 'Special Program in Technical-Vocational Education', 'description': 'Technical and vocational skills'},
    {'code': 'REGULAR', 'name': 'Regular Program', 'description': 'Standard curriculum with balanced workload'},
    {'code': 'TOP5', 'name': 'TOP 5 Program', 'description': 'Top performing students program'},
    {'code': 'HETERO', 'name': 'Heterogeneous', 'description': 'Mixed ability grouping'},
]

for prog_data in programs:
    prog, created = Program.objects.get_or_create(
        code=prog_data['code'],
        defaults={

```

```

        'name': prog_data['name'],
        'description': prog_data['description']
    }
)
if created:
    self.stdout.write(self.style.SUCCESS(f' ✓ Created program: {prog.code} -
{prog.name}'))
else:
    self.stdout.write(f' - Program already exists: {prog.code}')

self.stdout.write(self.style.SUCCESS(f'\n ✓ Initial data creation completed!\n'))

```

create_users.py – Create system users

```

from django.core.management.base import BaseCommand
from django.contrib.auth.models import User
from admin_app.models import UserProfile, Program

```

```

class Command(BaseCommand):
    help = 'Creates initial admin and coordinator users along with their profiles'

    def handle(self, *args, **kwargs):
        # Create admin user
        self.stdout.write(self.style.WARNING('Creating admin user...'))
        admin_user = User.objects.create_user(
            username='admin',
            password='admin123',
            first_name='Admin',
            last_name='User',
            email='admin@znhswest.edu'
        )
        UserProfile.objects.create(
            user=admin_user,
            user_type='admin'
        )
        self.stdout.write(self.style.SUCCESS(f' ✓ Created admin user: {admin_user.username}'))

        # Create coordinator user
        self.stdout.write(self.style.WARNING('Creating coordinator user...'))
        # Ensure the program exists (you would probably have this already)
        program, created = Program.objects.get_or_create(
            code='STEM',
            defaults={'name': 'Science, Technology, Engineering, and Mathematics', 'description':
'Advanced science and math program'}
        )

        coord_user = User.objects.create_user(
            username='stem_coord',

```

```

        password='stem123',
        first_name='Juan',
        last_name='Garcia',
        email='juan.garcia@znhswest.edu'
    )

    # Create UserProfile for the coordinator
    UserProfile.objects.create(
        user=coord_user,
        user_type='coordinator',
        program=program,
        employee_id='COORD123' # Assign a unique employee ID
    )
    self.stdout.write(self.style.SUCCESS(f'✓ Created coordinator user:
{coord_user.username}'))

    self.stdout.write(self.style.SUCCESS("\n✅ Initial users creation completed!\n"))

```

populate_buildings.py – Populate building records

```

from django.core.management.base import BaseCommand
from admin_app.models import Building, Room

class Command(BaseCommand):
    help = 'Populates the database with buildings and rooms'

    def handle(self, *args, **options):
        self.stdout.write('Populating buildings and rooms...')

        # Define buildings and their rooms
        buildings_data = {
            'Building 1': ['101', '102', '103', '104', '105'],
            'Building 2': ['201', '202', '203', '204', '205'],
            'Building 3': ['301', '302', '303', '304', '305'],
            'Building 4': ['Lab 1', 'Lab 2', 'Lab 3', 'Lecture 1', 'Lecture 2'],
            'Building 5': ['IT-101', 'IT-102', 'IT-201', 'IT-202', 'Server Room']
        }

        created_buildings = 0
        created_rooms = 0

        for building_name, room_numbers in buildings_data.items():
            # Get or create building
            building, building_created = Building.objects.get_or_create(name=building_name)
            if building_created:
                created_buildings += 1
                self.stdout.write(self.style.SUCCESS(f'Created building: {building_name}'))

```

```

# Create rooms for this building
for room_number in room_numbers:
    room, room_created = Room.objects.get_or_create(
        building=building,
        room_number=room_number
    )
    if room_created:
        created_rooms += 1
        self.stdout.write(f' - Created room: {room_number}')

self.stdout.write(self.style.SUCCESS(
    f'\nCompleted! Created {created_buildings} buildings and {created_rooms} rooms.'
))

```

populate_teachers.py – Populate teacher records

```

from django.core.management.base import BaseCommand
from admin_app.models import Teacher, Position, Department

```

```

class Command(BaseCommand):
    help = 'Populate the database with 10 sample teachers'

    def handle(self, *args, **kwargs):
        # Get or create positions
        position_t1, _ = Position.objects.get_or_create(
            name='Teacher I',
            defaults={'description': 'Entry-level teaching position'}
        )
        position_t2, _ = Position.objects.get_or_create(
            name='Teacher II',
            defaults={'description': 'Intermediate teaching position'}
        )
        position_t3, _ = Position.objects.get_or_create(
            name='Teacher III',
            defaults={'description': 'Advanced teaching position'}
        )
        position_mt1, _ = Position.objects.get_or_create(
            name='Master Teacher I',
            defaults={'description': 'Master teaching position'}
        )

        # Get or create departments
        dept_math, _ = Department.objects.get_or_create(
            name='Mathematics Department',
            defaults={'description': 'Department of Mathematics'}
        )
        dept_science, _ = Department.objects.get_or_create(
            name='Science Department',

```

```

        defaults={'description': 'Department of Science'}
    )
    dept_english, _ = Department.objects.get_or_create(
        name='English Department',
        defaults={'description': 'Department of English'}
    )
    dept_filipino, _ = Department.objects.get_or_create(
        name='Filipino Department',
        defaults={'description': 'Department of Filipino'}
    )
    dept_social, _ = Department.objects.get_or_create(
        name='Social Studies Department',
        defaults={'description': 'Department of Social Studies'}
    )

```

Sample teacher data

```

teachers_data = [
    {
        'first_name': 'Maria',
        'middle_name': 'Santos',
        'last_name': 'Cruz',
        'email': 'maria.cruz@znhswest.edu.ph',
        'position': position_t3,
        'department': dept_math,
        'address': '123 Zamboanga City',
    },
    {
        'first_name': 'Juan',
        'middle_name': 'Dela',
        'last_name': 'Reyes',
        'email': 'juan.reyes@znhswest.edu.ph',
        'position': position_mt1,
        'department': dept_science,
        'address': '456 Zamboanga City',
    },
    {
        'first_name': 'Ana',
        'middle_name': 'Lopez',
        'last_name': 'Garcia',
        'email': 'ana.garcia@znhswest.edu.ph',
        'position': position_t2,
        'department': dept_english,
        'address': '789 Zamboanga City',
    },
    {
        'first_name': 'Pedro',
        'middle_name': 'Ramos',

```



```
'last_name': 'Aquino',
'email': 'pedro.aquino@znhswest.edu.ph',
'position': position_t1,
'department': dept_filipino,
'address': '321 Zamboanga City',
},
{
  'first_name': 'Rosa',
  'middle_name': 'Mendoza',
  'last_name': 'Torres',
  'email': 'rosa.torres@znhswest.edu.ph',
  'position': position_t3,
  'department': dept_social,
  'address': '654 Zamboanga City',
},
{
  'first_name': 'Carlos',
  'middle_name': 'Flores',
  'last_name': 'Bautista',
  'email': 'carlos.bautista@znhswest.edu.ph',
  'position': position_t2,
  'department': dept_math,
  'address': '987 Zamboanga City',
},
{
  'first_name': 'Elena',
  'middle_name': 'Santiago',
  'last_name': 'Villanueva',
  'email': 'elena.villanueva@znhswest.edu.ph',
  'position': position_t1,
  'department': dept_science,
  'address': '147 Zamboanga City',
},
{
  'first_name': 'Roberto',
  'middle_name': 'Castro',
  'last_name': 'Fernandez',
  'email': 'roberto.fernandez@znhswest.edu.ph',
  'position': position_t3,
  'department': dept_english,
  'address': '258 Zamboanga City',
},
{
  'first_name': 'Carmen',
  'middle_name': 'Rivera',
  'last_name': 'Morales',
  'email': 'carmen.morales@znhswest.edu.ph',
```

```

        'position': position_mt1,
        'department': dept_filipino,
        'address': '369 Zamboanga City',
    },
    {
        'first_name': 'Miguel',
        'middle_name': 'Hernandez',
        'last_name': 'Pascual',
        'email': 'miguel.pascual@znhswest.edu.ph',
        'position': position_t2,
        'department': dept_social,
        'address': '741 Zamboanga City',
    },
]

created_count = 0
skipped_count = 0

for teacher_data in teachers_data:
    # Check if teacher with this email already exists
    if Teacher.objects.filter(email=teacher_data['email']).exists():
        self.stdout.write(
            self.style.WARNING(
                f"Skipped: {teacher_data['first_name']} {teacher_data['last_name']} (email already
exists)"
            )
        )
        skipped_count += 1
        continue

    # Create teacher
    teacher = Teacher.objects.create(**teacher_data)
    self.stdout.write(
        self.style.SUCCESS(
            f"Created: {teacher.get_full_name()} - {teacher.position.name}
({teacher.department.name})"
        )
    )
    created_count += 1

self.stdout.write(
    self.style.SUCCESS(
        f"\n✓ Successfully created {created_count} teacher(s)"
    )
)
if skipped_count > 0:
    self.stdout.write(

```

```

        self.style.WARNING(
            f'⚠ Skipped {skipped_count} teacher(s) (already exist)'
        )
    )
)

```

seed_grade_levels.py – Initialize grade levels

```

from django.core.management.base import BaseCommand
from admin_app.models import GradeLevel

```

```

class Command(BaseCommand):
    help = 'Seed Grade 7-10 into GradeLevel table'

    def handle(self, *args, **kwargs):
        grades = [
            {'code': 'G7', 'name': 'Grade 7'},
            {'code': 'G8', 'name': 'Grade 8'},
            {'code': 'G9', 'name': 'Grade 9'},
            {'code': 'G10', 'name': 'Grade 10'},
        ]
        for g in grades:
            obj, created = GradeLevel.objects.get_or_create(
                code=g['code'],
                defaults={'name': g['name'], 'is_active': True}
            )
            status = 'Created' if created else 'Already exists'
            self.stdout.write(f'{status}: {obj.name}')
            self.stdout.write(self.style.SUCCESS('Done!'))

```

set_grade7.py – Configure Grade 7 settings

```

"""

```

Management command: set_grade7

Sets grade_level = Grade 7 for ALL existing students and sections that currently have no grade level (or optionally all of them).

Usage:

```

python manage.py set_grade7
python manage.py set_grade7 --force      # overwrite even if already set
python manage.py set_grade7 --dry-run    # preview without saving
"""

```

```

from django.core.management.base import BaseCommand
from admin_app.models import GradeLevel, Section
from enrollment_app.models import Student

```

```

class Command(BaseCommand):
    help = 'Set grade_level = Grade 7 for all existing students and sections'

```

```

def add_arguments(self, parser):
    parser.add_argument(
        '--force',
        action='store_true',
        help='Overwrite grade_level even if already set (default: only update NULL entries)',
    )
    parser.add_argument(
        '--dry-run',
        action='store_true',
        help='Preview how many records will be updated without saving anything',
    )

```

```

def handle(self, *args, **options):
    force = options['force']
    dry_run = options['dry_run']

```

— 1. Find Grade 7

```

grade7 = GradeLevel.objects.filter(code='G7').first()
if not grade7:
    # Try by name if code is different
    grade7 = GradeLevel.objects.filter(name__icontains='grade 7').first()

if not grade7:
    self.stderr.write(self.style.ERROR(
        'Grade 7 not found in GradeLevel table.\n'
        'Please add it first via Admin → Settings → Others → Grade Level.'
    ))
    return

self.stdout.write(self.style.SUCCESS(
    f'Found Grade 7 → id={grade7.id}, code="{grade7.code}", name="{grade7.name}"
))

```

— 2. Update Sections

```

if force:
    section_qs = Section.objects.all()
else:
    section_qs = Section.objects.filter(grade_level__isnull=True)

section_count = section_qs.count()

if dry_run:
    self.stdout.write(f'[DRY RUN] Sections to update: {section_count}')
else:

```

```

        updated = section_qs.update(grade_level=grade7)
        self.stdout.write(self.style.SUCCESS(
            f'✓ Sections updated: {updated}'
        ))

```

— 3. Update Students

```

if force:
    student_qs = Student.objects.all()
else:
    student_qs = Student.objects.filter(grade_level__isnull=True)

student_count = student_qs.count()

if dry_run:
    self.stdout.write(f'[DRY RUN] Students to update: {student_count}')
    self.stdout.write(self.style.WARNING(
        '\nDry run complete. No changes saved. Remove --dry-run to apply.'
    ))
else:
    updated = student_qs.update(grade_level=grade7)
    self.stdout.write(self.style.SUCCESS(
        f'✓ Students updated: {updated}'
    ))
    self.stdout.write(self.style.SUCCESS(
        '\nDone! All targeted records now have Grade 7 assigned.'
    ))

```

link_sections_to_schoolyear.py – Associate sections with school years

```

"""

```

Management command to link existing sections to the current active school year

Usage: python manage.py link_sections_to_schoolyear

```

"""

```

```

from django.core.management.base import BaseCommand

```

```

from django.db import transaction

```

```

from admin_app.models import Section, SchoolYear

```

```

class Command(BaseCommand):

```

```

    help = 'Links all existing sections without a school year to the current active school year'

```

```

    def add_arguments(self, parser):

```

```

        parser.add_argument(

```

```

            '--school-year',

```

```

            type=str,

```

```
        help='School year label (e.g., 2025-2026) to link sections to. If not provided, uses the  
active school year.',
```

```
    )  
    parser.add_argument(  
        '--dry-run',  
        action='store_true',  
        help='Show what would be updated without actually updating',  
    )
```

```
def handle(self, *args, **options):  
    school_year_label = options.get('school_year')  
    dry_run = options.get('dry_run')
```

```
    try:
```

```
        # Get the target school year
```

```
        if school_year_label:
```

```
            school_year = SchoolYear.objects.get(year_label=school_year_label)
```

```
            self.stdout.write(self.style.SUCCESS(f'Using school year: {school_year.year_label}'))
```

```
        else:
```

```
            school_year = SchoolYear.objects.filter(is_active=True).first()
```

```
            if not school_year:
```

```
                self.stdout.write(self.style.ERROR('No active school year found. Please create one  
first or specify --school-year'))
```

```
            return
```

```
            self.stdout.write(self.style.SUCCESS(f'Using active school year:
```

```
{school_year.year_label}'))
```

```
        # Get sections without a school year
```

```
        sections_without_year = Section.objects.filter(school_year__isnull=True)
```

```
        count = sections_without_year.count()
```

```
        if count == 0:
```

```
            self.stdout.write(self.style.WARNING('No sections found without a school year. All  
sections are already linked!'))
```

```
            return
```

```
        self.stdout.write(f'\nFound {count} section(s) without a school year:')
```

```
        self.stdout.write('=' * 80)
```

```
        for section in sections_without_year:
```

```
            self.stdout.write(f' - {section.program.code} - {section.name} (Adviser:  
{section.adviser or "None"})')
```

```
        if dry_run:
```

```
            self.stdout.write('\n' + '=' * 80)
```

```
            self.stdout.write(self.style.WARNING('DRY RUN: No changes were made. Remove --  
dry-run to apply changes.'))
```

```

        return

    # Confirm before proceeding
    self.stdout.write('\n' + '=' * 80)
    confirm = input(f'\nLink these {count} section(s) to school year
"{school_year.year_label}"? (yes/no): ')

    if confirm.lower() not in ['yes', 'y']:
        self.stdout.write(self.style.WARNING('Operation cancelled.'))
        return

    # Update sections
    with transaction.atomic():
        updated_count = sections_without_year.update(school_year=school_year)

        self.stdout.write('\n' + '=' * 80)
        self.stdout.write(self.style.SUCCESS(f'✓ Successfully linked {updated_count}
section(s) to school year "{school_year.year_label}"'))

    # Show summary
    self.stdout.write('\nSummary by Program:')
    programs = sections_without_year.values_list('program__code', flat=True).distinct()
    for program_code in programs:
        program_count = Section.objects.filter(
            school_year=school_year,
            program__code=program_code
        ).count()
        self.stdout.write(f' - {program_code}: {program_count} section(s)')

except SchoolYear.DoesNotExist:
    self.stdout.write(self.style.ERROR(f'School year "{school_year_label}" not found.'))
    self.stdout.write('Available school years:')
    for sy in SchoolYear.objects.all():
        active_marker = ' (ACTIVE)' if sy.is_active else ''
        self.stdout.write(f' - {sy.year_label}{active_marker}')
except Exception as e:
    self.stdout.write(self.style.ERROR(f'Error: {str(e)}'))

```

reset_data.py – Reset system data

```

from django.core.management.base import BaseCommand
from django.db import connection
from enrollment_app.models import (
    Student, StudentData, Parent, Guardian, FamilyData,
    SurveyData, AcademicData, ProgramSelection, EnrollmentStatusLog
)

class Command(BaseCommand):

```

```
help = 'Delete all data from Enrollment App models and reset IDs to 1'
```

```
def handle(self, *args, **kwargs):
```

```
    try:
```

```
        self.stdout.write('Starting data deletion...')
```

```
        # Delete all data (order matters due to foreign keys)
```

```
        deleted_counts = {}
```

```
        deleted_counts['EnrollmentStatusLog'] = EnrollmentStatusLog.objects.all().delete()[0]
```

```
        deleted_counts['ProgramSelection'] = ProgramSelection.objects.all().delete()[0]
```

```
        deleted_counts['AcademicData'] = AcademicData.objects.all().delete()[0]
```

```
        deleted_counts['SurveyData'] = SurveyData.objects.all().delete()[0]
```

```
        deleted_counts['FamilyData'] = FamilyData.objects.all().delete()[0]
```

```
        deleted_counts['Guardian'] = Guardian.objects.all().delete()[0]
```

```
        deleted_counts['Parent'] = Parent.objects.all().delete()[0]
```

```
        deleted_counts['StudentData'] = StudentData.objects.all().delete()[0]
```

```
        deleted_counts['Student'] = Student.objects.all().delete()[0]
```

```
        # Display deletion summary
```

```
        self.stdout.write('\nDeleted records:')
```

```
        for model, count in deleted_counts.items():
```

```
            self.stdout.write(f' - {model}: {count}')
```

```
        # Reset the ID counters for PostgreSQL
```

```
        self.stdout.write('\nResetting ID sequences...')
```

```
        with connection.cursor() as cursor:
```

```
            sequences = [
```

```
                'enrollment_app_student_id_seq',
```

```
                'enrollment_app_studentdata_id_seq',
```

```
                'enrollment_app_parent_id_seq',
```

```
                'enrollment_app_guardian_id_seq',
```

```
                'enrollment_app_familydata_id_seq',
```

```
                'enrollment_app_surveydata_id_seq',
```

```
                'enrollment_app_academicdata_id_seq',
```

```
                'enrollment_app_programselection_id_seq',
```

```
                'enrollment_app_enrollmentstatuslog_id_seq',
```

```
            ]
```

```
        for seq in sequences:
```

```
            try:
```

```
                cursor.execute(f"ALTER SEQUENCE {seq} RESTART WITH 1;")
```

```
            except Exception as e:
```

```
                self.stdout.write(self.style.WARNING(f' ⚠ Could not reset {seq}: {e}'))
```

```
        self.stdout.write(self.style.SUCCESS('\n✅ All enrollment data deleted and IDs reset to
```

```
1.'))
```


except Exception as e:

```
self.stdout.write(self.style.ERROR(f'\n❌ Error occurred: {e}'))
```

ste_qualified.py – Generate STE qualification records

```
from django.core.management.base import BaseCommand
from coordinator_app.models import Qualified_for_ste # Update with your app name
from django.contrib.auth.models import User
```

```
class Command(BaseCommand):
```

```
    help = 'Insert student records into Qualified_for_ste'
```

```
    def handle(self, *args, **kwargs):
```

```
        students = [
            ('111223344556', 'Carlos', 'Garcia', '2010-03-11', 'PQR Primary School'),
            ('222334455667', 'Ana', 'Lopez', '2013-08-25', 'DEF School'),
            ('333445566778', 'Luis', 'Reyes', '2011-11-30', 'GHI Academy'),
            ('444556677889', 'Isabella', 'Morales', '2012-01-05', 'JKL School'),
            ('555667788990', 'Miguel', 'Torres', '2013-07-18', 'MNO Primary School'),
            ('666778899001', 'Sofia', 'Cruz', '2010-12-15', 'NOP School'),
            ('777889900112', 'David', 'Mendez', '2012-06-20', 'QRS Academy'),
            ('888990011223', 'Emma', 'Diaz', '2013-04-07', 'TUV Elementary'),
            ('999001122334', 'Oliver', 'Martinez', '2011-10-30', 'WXY School'),
            ('101234567890', 'Ava', 'Hernandez', '2012-02-25', 'ZAB Academy'),
            ('111234567891', 'James', 'Parker', '2013-09-14', 'CDE School'),
            ('121234567892', 'Mia', 'Rojas', '2010-05-17', 'FGH School'),
            ('131234567893', 'Ethan', 'Ramirez', '2011-03-28', 'IJK Academy'),
            ('141234567894', 'Zoe', 'Gonzalez', '2012-07-02', 'LMN School'),
            ('151234567895', 'Liam', 'Alvarez', '2013-08-18', 'OPQ School'),
            ('161234567896', 'Grace', 'Vargas', '2010-04-26', 'RST Primary School'),
            ('171234567897', 'Noah', 'Castillo', '2012-11-13', 'UVW Academy'),
            ('181234567898', 'Lily', 'Ortega', '2013-01-03', 'XYZ School'),
            ('191234567899', 'Elijah', 'Gutierrez', '2011-12-19', 'ABC Academy'),
            ('201234567890', 'Chloe', 'Sanchez', '2012-09-05', 'DEF School'),
            ('211234567891', 'Lucas', 'Torres', '2010-06-30', 'GHI School'),
            ('221234567892', 'Mila', 'Mora', '2013-10-22', 'JKL Academy'),
            ('231234567893', 'Isaiah', 'Hernandez', '2012-01-15', 'MNO School'),
            ('241234567894', 'Layla', 'Solis', '2011-11-16', 'PQR Academy'),
            ('251234567895', 'Gabriel', 'Palacios', '2010-02-11', 'STU School'),
            ('261234567896', 'Nora', 'Ruiz', '2013-05-09', 'VWX Academy'),
            ('271234567897', 'Cameron', 'Valenzuela', '2012-08-28', 'YZA School'),
            ('281234567898', 'Savannah', 'Trejo', '2011-09-17', 'BCD Academy'),
            ('291234567899', 'Eric', 'Bautista', '2013-04-12', 'EFG School'),
            ('301234567890', 'Kylie', 'Olivares', '2011-10-27', 'HIJ Primary School'),
            ('311234567891', 'Parker', 'Escobar', '2010-03-15', 'KLM Academy'),
            ('321234567892', 'Skylar', 'Luna', '2012-06-23', 'NOP School'),
            ('331234567893', 'Wyatt', 'Mendez', '2013-09-20', 'QRS Academy'),
```

```

('341234567894', 'Riley', 'Cervantes', '2012-05-08', 'TUV School'),
('351234567895', 'Levi', 'Salazar', '2011-02-14', 'WXY Academy'),
('361234567896', 'Madison', 'Martinez', '2010-04-25', 'ZAB School'),
('371234567897', 'Jaxon', 'Reyes', '2013-11-16', 'CDE Academy'),
('381234567898', 'Ella', 'Gonzalez', '2012-01-30', 'FGH School'),
('391234567899', 'Anthony', 'Pena', '2011-03-29', 'IJK Academy'),
('401234567890', 'Caroline', 'Hernandez', '2012-12-05', 'LMN School'),
('411234567891', 'Hunter', 'Gonzalez', '2010-11-18', 'OPQ Academy'),
('421234567892', 'Victoria', 'Morales', '2013-06-14', 'RST School'),
('431234567893', 'Matthew', 'Ramirez', '2012-02-09', 'UVW Academy'),
('441234567894', 'Addison', 'Diaz', '2011-07-17', 'XYZ School'),
('451234567895', 'Nathan', 'Sanchez', '2010-08-30', 'ABC Academy'),
('461234567896', 'Aubrey', 'Salinas', '2013-03-24', 'DEF School'),
('471234567897', 'Owen', 'Cortez', '2011-05-29', 'GHI Academy'),
('481234567898', 'Hailey', 'Soto', '2012-09-22', 'JKL School'),
('491234567899', 'Samuel', 'Rojas', '2013-01-06', 'MNO Primary School'),
('501234567890', 'Scarlett', 'Guerrero', '2010-10-10', 'PQR Academy'),
('511234567891', 'Dylan', 'Hernandez', '2012-04-01', 'TUV School'),
('521234567892', 'Alyssa', 'Mendez', '2013-05-11', 'WXY Academy'),
]

for student in students:
    student_lrn, first_name, last_name, dob, school_name = student
    qualified_record = Qualified_for_ste(
        student_lrn=student_lrn,
        exam_score=85.0, # Example score
        interview_score=90.0, # Example score
        status='qualified', # Example status
        updated_by=User.objects.get(id=5), # Assuming a user with ID 5
        remarks=f'Student from {school_name}.'
    )
    qualified_record.save()
    self.stdout.write(self.style.SUCCESS(f'Successfully inserted: {qualified_record}'))

self.stdout.write(self.style.SUCCESS("All records inserted successfully!"))

```

check_continuing_students.py – Check continuing student status

"""

Management command: check_continuing_students

=====

Checks which students from SY 2025-2026 are eligible to enroll
as continuing (old) students in SY 2026-2027.

Usage:

```

python manage.py check_continuing_students
python manage.py check_continuing_students --export csv
python manage.py check_continuing_students --status promoted

```

```
python manage.py check_continuing_students --verbose
```

Place this file in:

```
any_app/management/commands/check_continuing_students.py  
(e.g. admin_app/management/commands/check_continuing_students.py)
```

```
"""
```

```
from django.core.management.base import BaseCommand  
from django.db.models import Q  
from admin_app.models import SchoolYear  
from enrollment_app.models import Student, StudentEnrollment, StudentAcademicYearStatus,  
ProgramSelection  
import csv  
import sys
```

```
class Command(BaseCommand):  
    help = 'Check students eligible to continue enrollment in 2026-2027'
```

```
    def add_arguments(self, parser):  
        parser.add_argument(  
            '--export',  
            type=str,  
            choices=['csv'],  
            help='Export results to CSV (prints to stdout)',  
        )  
        parser.add_argument(  
            '--status',  
            type=str,  
            choices=['promoted', 'retained', 'pending', 'all'],  
            default='all',  
            help='Filter by academic year status (default: all)',  
        )  
        parser.add_argument(  
            '--verbose',  
            action='store_true',  
            help='Show detailed info per student',  
        )
```

```
    def handle(self, *args, **options):  
        # — Get the two school years —————  
        try:  
            sy_2025 = SchoolYear.objects.get(year_label='2025-2026')  
        except SchoolYear.DoesNotExist:  
            self.stderr.write(self.style.ERROR('School year 2025-2026 not found.'))  
            return
```

```
        try:
```

```

    sy_2026 = SchoolYear.objects.get(year_label='2026-2027')
except SchoolYear.DoesNotExist:
    self.stderr.write(self.style.ERROR('School year 2026-2027 not found.))
    return

self.stdout.write(self.style.NOTICE(
    f'\n{"*"*70}\n'
    f' Continuing Student Eligibility Check\n'
    f' From: {sy_2025.year_label} → To: {sy_2026.year_label}\n'
    f'{"*"*70}\n'
))

# — Step 1: Get all approved enrollments from 2025-2026 —————
approved_enrollments_2025 = StudentEnrollment.objects.filter(
    school_year=sy_2025,
    enrollment_status='approved',
).select_related('student', 'grade_level', 'student__student_data')

total_approved = approved_enrollments_2025.count()
self.stdout.write(f'Total approved enrollments in 2025-2026: {total_approved}')

# — Step 2: Build eligibility data —————
eligible = [] # promoted — definitely can continue
check_list = [] # no academic year status yet — needs review
ineligible = [] # retained, dropped, transferred, etc.
already_enrolled = [] # already have a 2026-2027 enrollment

for enrollment in approved_enrollments_2025:
    student = enrollment.student

    # Check if already enrolled in 2026-2027
    existing_2026 = StudentEnrollment.objects.filter(
        student=student,
        school_year=sy_2026,
    ).first()

    if existing_2026:
        already_enrolled.append({
            'ltn': student.ltn,
            'name': self._get_name(student),
            'grade_2025': enrollment.grade_level.name if enrollment.grade_level else 'N/A',
            'status_2026': existing_2026.enrollment_status,
            'enrollee_type_2026': existing_2026.enrollee_type,
        })
        continue

    # Check StudentAcademicYearStatus for 2025-2026

```

```

academic_status = StudentAcademicYearStatus.objects.filter(
    student=student,
    school_year=sy_2025,
).first()

# Get their program/section info
program_info = self._get_program_info(student, sy_2025)

row = {
    'ltn': student.ltn,
    'name': self._get_name(student),
    'grade_2025': enrollment.grade_level.name if enrollment.grade_level else 'N/A',
    'next_grade': self._next_grade(enrollment.grade_level),
    'program': program_info['program'],
    'section': program_info['section'],
    'final_status': academic_status.final_status if academic_status else 'NO RECORD',
    'overall_grade': str(academic_status.overall_grade) if academic_status and
academic_status.overall_grade else 'N/A',
    'remarks': academic_status.remarks if academic_status else "",
    'has_probation': self._check_probation(student),
}

if academic_status:
    if academic_status.final_status == 'promoted':
        eligible.append(row)
    else:
        ineligible.append(row)
else:
    # No academic year status recorded yet
    check_list.append(row)

# — Apply --status filter —————
status_filter = options['status']

# — Display Results —————
if options['export'] == 'csv':
    self._export_csv(eligible, check_list, ineligible, already_enrolled)
    return

# — Summary —————


---


self.stdout.write('\n' + '─'*70)
self.stdout.write(self.style.SUCCESS(
    f'  ELIGIBLE (Promoted) : {len(eligible)}'
))
self.stdout.write(self.style.WARNING(
    f'  NEEDS REVIEW (No status yet) : {len(check_list)}'
))

```

```

))
self.stdout.write(self.style.ERROR(
    f' ❌ INELIGIBLE (Not promoted) : {len(ineligible)}'
))
self.stdout.write(
    f' ⓘ ALREADY IN 2026-2027 : {len(already_enrolled)}'
)
self.stdout.write('─'*70 + '\n')

# — Eligible Students —————
if status_filter in ('all', 'promoted') and eligible:
    self.stdout.write(self.style.SUCCESS(
        f'\n ✅ ELIGIBLE FOR CONTINUING ENROLLMENT ({len(eligible)} students)\n'
    ))
    self._print_table(eligible, options['verbose'])

# — Needs Review
────────────────────────────────────────
if status_filter in ('all', 'pending') and check_list:
    self.stdout.write(self.style.WARNING(
        f'\n ⚠️ NO ACADEMIC STATUS RECORDED — NEEDS REVIEW ({len(check_list)}
students)\n'
    ))
    self.stdout.write(
        ' These students were approved in 2025-2026 but have no\n'
        ' StudentAcademicYearStatus record. They CANNOT be marked\n'
        ' as continuing until a status is recorded by their adviser.\n'
    )
    self._print_table(check_list, options['verbose'])

# — Ineligible
────────────────────────────────────────
if status_filter in ('all', 'retained') and ineligible:
    self.stdout.write(self.style.ERROR(
        f'\n ❌ NOT ELIGIBLE FOR CONTINUING ({len(ineligible)} students)\n'
    ))
    self._print_table(ineligible, options['verbose'])

# — Already enrolled —————
if already_enrolled:
    self.stdout.write(
        f'\n ⓘ ALREADY ENROLLED IN 2026-2027 ({len(already_enrolled)} students)\n'
    )
    if options['verbose']:
        for s in already_enrolled:
            self.stdout.write(

```

```

        f" {s['lrrn']} {s['name']:<30} "
        f"Status: {s['status_2026']} Type: {s['enrollee_type_2026']}"
    )

    self.stdout.write("\n" + '='*70 + '\n')
    self.stdout.write(self.style.SUCCESS(
        'Run with --export csv to get a downloadable spreadsheet.\n'
        'Run with --verbose to see section and program details.\n'
        'Run with --status promoted|retained|pending to filter.\n'
    ))

```

— Helpers

```

def _get_name(self, student):
    try:
        sd = student.student_data
        return f"{sd.last_name}, {sd.first_name}"
    except Exception:
        return 'N/A'

def _get_program_info(self, student, school_year):
    try:
        ps = ProgramSelection.objects.filter(
            student=student,
            school_year=school_year,
            admin_approved=True,
        ).select_related('assigned_section').first()
        if ps:
            return {
                'program': ps.selected_program_code or 'N/A',
                'section': ps.assigned_section.name if ps.assigned_section else 'Unassigned',
            }
    except Exception:
        pass
    return {'program': 'N/A', 'section': 'N/A'}

def _next_grade(self, grade_level):
    if not grade_level:
        return 'N/A'
    mapping = {
        'G7': 'Grade 8',
        'G8': 'Grade 9',
        'G9': 'Grade 10',
        'G10': 'Graduated',
    }
    return mapping.get(grade_level.code, f'After {grade_level.name}')

```

```

def _check_probation(self, student):
    try:
        from coordinator_app.models import ProbationRecord
        return ProbationRecord.objects.filter(student=student, is_active=True).exists()
    except Exception:
        return False

def _print_table(self, rows, verbose=False):
    if not rows:
        return
    header = f" {'LRN':<14} {'Name':<30} {'Grade':<10} {'Next Grade':<12} {'Final Status':<15}
{'Grade':<8}"
    if verbose:
        header += f" {'Program':<10} {'Section':<15} {'Probation':<10} {'Remarks'}"
    self.stdout.write(header)
    self.stdout.write(' ' + '-'*100)
    for r in rows:
        line = (
            f" {r['lrn']:<14} {r['name']:<30} {r['grade_2025']:<10} "
            f"{r['next_grade']:<12} {r['final_status']:<15} {r['overall_grade']:<8}"
        )
        if verbose:
            probation = ' ⚠️ YES' if r['has_probation'] else 'No'
            line += f" {r['program']:<10} {r['section']:<15} {probation:<10} {r.get('remarks','')}"
        self.stdout.write(line)

def _export_csv(self, eligible, check_list, ineligible, already_enrolled):
    writer = csv.writer(sys.stdout)
    writer.writerow([
        'LRN', 'Name', 'Grade 2025-2026', 'Next Grade',
        'Program', 'Section', 'Final Status', 'Overall Grade',
        'Has Probation', 'Eligible for Continuing', 'Remarks',
    ])
    for r in eligible:
        writer.writerow([
            r['lrn'], r['name'], r['grade_2025'], r['next_grade'],
            r['program'], r['section'], r['final_status'], r['overall_grade'],
            'Yes' if r['has_probation'] else 'No',
            'YES - PROMOTED',
            r.get('remarks', ''),
        ])
    for r in check_list:
        writer.writerow([
            r['lrn'], r['name'], r['grade_2025'], r['next_grade'],
            r['program'], r['section'], 'NO STATUS RECORDED', r['overall_grade'],
            'Yes' if r['has_probation'] else 'No',
        ])

```



```

        'NEEDS REVIEW',
        r.get('remarks', ""),
    ])
for r in ineligible:
    writer.writerow([
        r['ltn'], r['name'], r['grade_2025'], r['next_grade'],
        r['program'], r['section'], r['final_status'], r['overall_grade'],
        'Yes' if r['has_probation'] else 'No',
        'NO - NOT PROMOTED',
        r.get('remarks', ""),
    ])
for r in already_enrolled:
    writer.writerow([
        r['ltn'], r['name'], r['grade_2025'], 'N/A',
        'N/A', 'N/A', r['status_2026'], 'N/A',
        'N/A',
        f'ALREADY IN 2026-2027 ({r["enrollee_type_2026"]})',
        "",
    ])

```